

Rscript for the Kuppe2022 spatial dataset

Libraries and Data

```
In [3]: library(anndata)
library(EBImage)
```

```
In [4]: setwd("../")
getwd()
```

'/home/vincent/Documents/Projects/Endothelins/Manuscript2025/Analysis'

```
In [5]: # functions etc
func.dir=~/.Documents/Rlib/functions"
source("~/Documents/Rlib/functions/load_functions.r")
# additional functions
source("scripts/spatialDataProcessing.r")
```

Loading required package: RColorBrewer

Loading required package: wordcloud

The following functions contained errors and were not loaded properly, please check the code:

cell_types.r colour_scale_map.r text_overlay.r

```
In [6]: datasets = list.files("data/datasets", full=T)
datasets
```

'data/datasets/control_P1_10X001.h5ad' · 'data/datasets/control_P17_ACH002.h5ad' · 'data/datasets/control_P7_ACH003.h5ad' ·
'data/datasets/control_P8_ACH004.h5ad' · 'data/datasets/GT_IZ_P9_ACH0012.h5ad' · 'data/datasets/IZ_P10_ACH0014.h5ad' ·
'data/datasets/IZ_P15_ACH0015.h5ad' · 'data/datasets/IZ_P16_ACH0016.h5ad' · 'data/datasets/IZ_P3_10X0017.h5ad' · 'data/datasets/RZ_P3_10X0020.h5ad' ·
'data/datasets/RZ_P6_ACH0021.h5ad' · 'data/datasets/RZ_P6_ACH0022.h5ad' · 'data/datasets/RZ_P9_ACH0023.h5ad'

```
In [7]: # HARDCODE sample IDs here?
sample_id_list <- c( # these were manually identified in the h5ad objects
  "AKK006_II_healthycontrol_V19T05022_rep2", # "datasets/control_P1_10X001.h5ad"
  "Visium_1_CK279", # "data/spatialdatasets/control_P17_ACH002.h5ad"
  "Visium_3_CK281", # "data/spatialdatasets/control_P7_ACH003.h5ad"
  "Visium_4_CK282", # "data/spatialdatasets/control_P8_ACH004.h5ad"

  "Visium_9_CK287", # "data/spatialdatasets/GT_IZ_P9_ACH0012.h5ad"
  "Visium_7_CK285", # "data/spatialdatasets/IZ_P10_ACH0014.h5ad"
  "Visium_20_CK298", # "data/spatialdatasets/IZ_P15_ACH0015.h5ad"
  "Visium_15_CK293", # "data/spatialdatasets/IZ_P16_ACH0016.h5ad"
  "AKK002_Infarct_MI_acute_V19T05025_rep2", # "datasets/IZ_P3_10X0017.h5ad"

  "AKK002_Normal_MI_acute_V19T05026_rep1", # 'data/spatial/datasets/RZ_P3_10X0020.h5ad'
  "Visium_2_CK280", # 'data/spatial/datasets/RZ_P6_ACH0021.h5ad'
  "Visium_17_CK295", # 'data/spatial/datasets/RZ_P6_ACH0022.h5ad'
  "Visium_6_CK284" # 'data/spatial/datasets/RZ_P9_ACH0023.h5ad'
)
```

reading in datasets and pre-processing images

The short story here is that I had been working on a variety of functions for a bespoke shiny app. These functions involve pre-processing data (QC, normalisation of gene expression), rotation and checking of images (contrast, spot overlay), and sending processed data out to subdirectories for faster access.

The next steps for each sample do this, so we can pick up the data in the later sections for plotting.

Healthy Left Ventricle # 1 "AKK006_II_healthycontrol_V19T05022_rep2"

```
In [9]: # Healthy Left Ventricle # 2 "Visium_1_CK279"
x = anndata::read_h5ad(datasets[1])
x$obs$sample = names(x$uns$spatial)[1]

options(repr.plot.width = 10)
options(repr.plot.height = 10)

# run once to evaluate image orientation etc
preprocessImage(
  image_h5ad = x,
  rotation = 0,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = T,
  feature_names = x$var$feature_name,
  nCounts_column = "n_counts",
  nGenes_column = "n_genes",
  QC_threshold = 600,
  contrast_mod = 1.5
)

image_id = names(x$uns$spatial)[1]
image_data = Image(x$uns$spatial[[image_id]]$images$ hires)/255

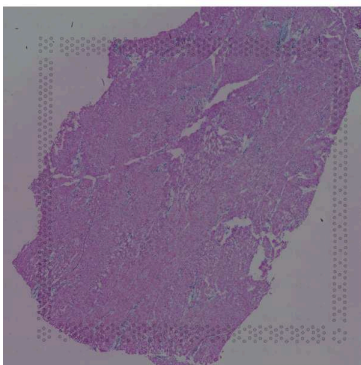
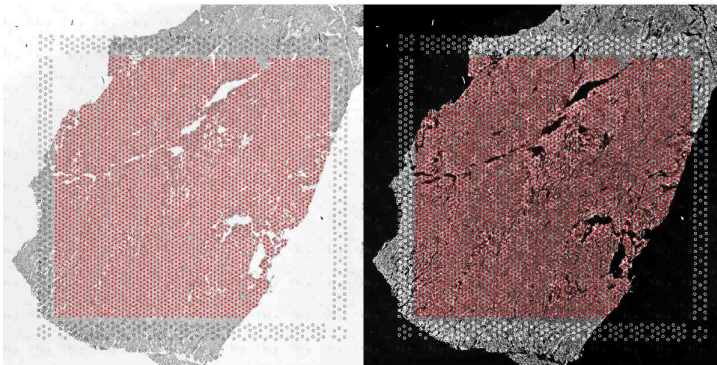
# find the scale factor
image_scale <- x$uns$spatial[[image_id]][["scalefactors"]][[paste("tissue_", "hires", "_scalef", sep="")]]
```

```
# Process Image:
# Color mode
colorMode(image_data) = "Color"

# plot the original image
display(image_data, method="raster", all=F)
```

1. Loading Spatial Dataset: AKK006_II_healthycontrol_V19T05022_rep2
2. Available sample IDs:
 - ... processing sample: AKK006_II_healthycontrol_V19T05022_rep2
3. "evaluate_image" set to TRUE:
 - ... Skipping spot matrix processing and gene expression writing
4. processing image(s).
 - ... pulling out image data from anndata file
 - ... applying selected transformations
5. Generating grey scale images and mask
 - ... "evaluate_image" is TRUE, plotting images for evaluation

IMAGE EVALUATION: Please check the plots to ensure adequate overlap of images and spots, as well as any adjustments needed for image orientation. Once you are happy, re-run the function with the argument "evaluate_image" set to FALSE. This will then process the gene expression matrix, and write the data to the directory for the shiny app to handle.



```
In [11]: # run again but flag evaluation as F
preprocessImage(
  image_h5ad = x,
  rotation = 0,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
```

```

    evaluate_image = F,
    feature_names = x$var$feature_name,
    nCounts_column = "n_counts",
    nGenes_column = "n_genes",
    QC_threshold = 600,
    contrast_mod = 1.5
  )

png(paste("data/spatial/", image_id, "/unprocessed.png", sep=""), height=2000, width=2000)
display(image_data, method="raster", all=F)
dev.off()

```

1. Loading Spatial Dataset: AKK006_II_healthycontrol_V19T05022_rep2
2. Available sample IDs:
 - ... processing sample: AKK006_II_healthycontrol_V19T05022_rep2
3. "evaluate_image" set to FALSE:
 - ... Transforming spot matrix

Warning message in asMethod(object):

"sparse->dense coercion: allocating vector of size 1.1 GiB"

```

... Picking up the gene lookup table from: "data/gene_lookup_table.csv"
... Picking up the spatial subset of gene lookup table from: "data/spatial/gene_lookup_spatial.csv"
... Writing the genes into their files:
... Writing the genes into their files:
A/Z
... Writing the genes into their files:
B/Z
... Writing the genes into their files:
C/Z
... Writing the genes into their files:
D/Z
... Writing the genes into their files:
E/Z
... Writing the genes into their files:
F/Z
... Writing the genes into their files:
G/Z
... Writing the genes into their files:
H/Z
... Writing the genes into their files:
I/Z
... Writing the genes into their files:
J/Z
... Writing the genes into their files:
K/Z
... Writing the genes into their files:
L/Z
... Writing the genes into their files:
M/Z
... Writing the genes into their files:
N/Z
... Writing the genes into their files:
O/Z
... Writing the genes into their files:
P/Z
... Writing the genes into their files:
Q/Z
... Writing the genes into their files:
R/Z
... Writing the genes into their files:
S/Z
... Writing the genes into their files:
T/Z
... Writing the genes into their files:
U/Z
... Writing the genes into their files:
V/Z
... Writing the genes into their files:
W/Z
... Writing the genes into their files:
X/Z
... Writing the genes into their files:
Y/Z
... Writing the genes into their files:
Z/Z

```

4. processing image(s).
 - ... pulling out image data from anndata file
 - ... applying selected transformations
5. Generating grey scale images and mask
6. writing image 1 (light version): data/spatial/AKK006_II_healthycontrol_V19T05022_rep2/images/1/overlay.png
 - ... writing image 2 (dark version): data/spatial/AKK006_II_healthycontrol_V19T05022_rep2/images/2/overlay.png
7. writing datapoint coordinates to: data/spatial/AKK006_II_healthycontrol_V19T05022_rep2/coordinates.csv

IMAGE EVALUATION: no image evaluation was carried out. If transformations are needed, please re-run the function with the argument "evaluate_image" set to TRUE (default). This will skip the gene matrix processing and plot the images, re-run iteratively using the "rotation" and "reflection" arguments until the desired orientation is achieved.

agg_record_521667323: 2

Healthy Left Ventricle # 2 "Visium_1_CK279"

```

In [12]: # Healthy Left Ventricle # 2 "Visium_1_CK279"
         x = anndata::read_h5ad(datasets[2])
         x$obs$sample = names(x$uns$spatial)[1]

         # run once to evaluate image orientation etc
         preprocessImage(

```

```

image_h5ad = x,
rotation = 0,
reflection = "none",
opacity_filter = 0.9,
sample_id = unique(x$obs$sample),
evaluate_image = T,
feature_names = x$var$feature_name,
nCounts_column = "n_counts",
nGenes_column = "n_genes",
QC_threshold = 600,
contrast_mod = 1
)

image_id = names(x$uns$spatial)[1]
image_data = Image(x$uns$spatial[[image_id]]$images$ hires)/255

# find the scale factor
image_scale <- x$uns$spatial[[image_id]][["scalefactors"]][[paste("tissue_", "hires", "_scalef", sep="")]]

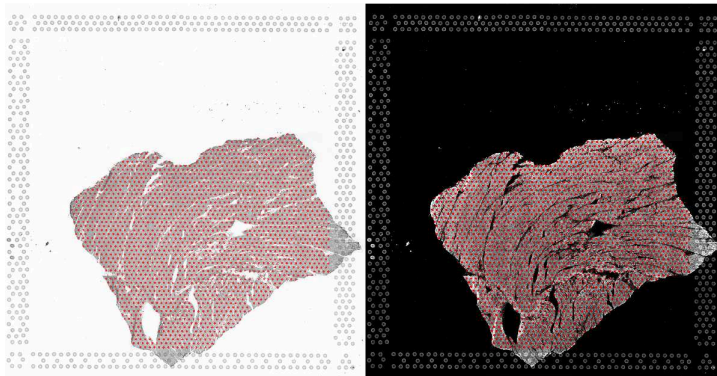
# Process Image:
# Color mode
colorMode(image_data) = "Color"

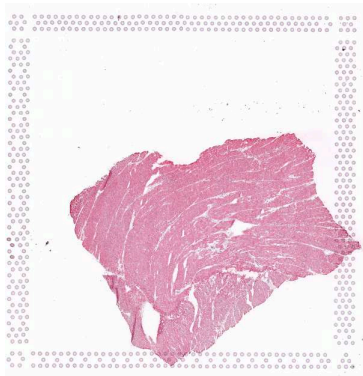
# plot the original image
display(image_data, method="raster", all=F)

```

1. Loading Spatial Dataset: Visium_1_CK279
2. Available sample IDs:
 - ... processing sample: Visium_1_CK279
3. "evaluate_image" set to TRUE:
 - ... Skipping spot matrix processing and gene expression writing
4. processing image(s).
 - ... pulling out image data from anndata file
 - ... applying selected transformations
5. Generating grey scale images and mask
 - ... "evaluate_image" is TRUE, plotting images for evaluation

IMAGE EVALUATION: Please check the plots to ensure adequate overlap of images and spots, as well as any adjustments needed for image orientation. Once you are happy, re-run the function with the argument "evaluate_image" set to FALSE. This will then process the gene expression matrix, and write the data to the directory for the shiny app to handle.





```
In [13]: # run again but flag evaluation as F
preprocessImage(
  image_h5ad = x,
  rotation = 0,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = F,
  feature_names = x$var$feature_name,
  nCounts_column = "n_counts",
  nGenes_column = "n_genes",
  QC_threshold = 600,
  contrast_mod = 1
)

png(paste("data/spatial/", image_id, "/unprocessed.png", sep=""), height=2000, width=2000)
display(image_data, method="raster", all=F)
dev.off()
```

```

1. Loading Spatial Dataset: Visium_1_CK279
2. Available sample IDs:
... processing sample: Visium_1_CK279
3. "evaluate_image" set to FALSE:
... Transforming spot matrix
... Picking up the gene lookup table from: "data/gene_lookup_table.csv"
... Picking up the spatial subset of gene lookup table from: "data/spatial/gene_lookup_spatial.csv"
... Writing the genes into their files:
... Writing the genes into their files:
A/Z
... Writing the genes into their files:
B/Z
... Writing the genes into their files:
C/Z
... Writing the genes into their files:
D/Z
... Writing the genes into their files:
E/Z
... Writing the genes into their files:
F/Z
... Writing the genes into their files:
G/Z
... Writing the genes into their files:
H/Z
... Writing the genes into their files:
I/Z
... Writing the genes into their files:
J/Z
... Writing the genes into their files:
K/Z
... Writing the genes into their files:
L/Z
... Writing the genes into their files:
M/Z
... Writing the genes into their files:
N/Z
... Writing the genes into their files:
O/Z
... Writing the genes into their files:
P/Z
... Writing the genes into their files:
Q/Z
... Writing the genes into their files:
R/Z
... Writing the genes into their files:
S/Z
... Writing the genes into their files:
T/Z
... Writing the genes into their files:
U/Z
... Writing the genes into their files:
V/Z
... Writing the genes into their files:
W/Z
... Writing the genes into their files:
X/Z
... Writing the genes into their files:
Y/Z
... Writing the genes into their files:
Z/Z

4. processing image(s).
... pulling out image data from anndata file
... applying selected transformations
5. Generating grey scale images and mask
6. writing image 1 (light version): data/spatial/Visium_1_CK279/images/1/overlay.png
... writing image 2 (dark version): data/spatial/Visium_1_CK279/images/2/overlay.png
7. writing datapoint coordinates to: data/spatial/Visium_1_CK279/coordinates.csv

```

IMAGE EVALUATION: no image evaluation was carried out. If transformations are needed, please re-run the function with the argument "evaluate_image" set to TRUE (default). This will skip the gene matrix processing and plot the images, re-run iteratively using the "rotation" and "reflection" arguments until the desired orientation is achieved.

agg_record_1999108986:2

Healthy Left Ventricle # 3 "Visium_3_CK281"

```

In [14]: # Healthy Left Ventricle # 3 "Visium_3_CK281"
x = anndata.read_h5ad(datasets[3])
x.obs.sample = names(x.uns$spatial)[1]

# run once to evaluate image orientation etc
preprocessImage(
  image_h5ad = x,
  rotation = 0,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x.obs.sample),
  evaluate_image = T,
  feature_names = x.var.feature_name,
  nCounts_column = "n_counts",
  nGenes_column = "n_genes",
  QC_threshold = 600,
  contrast_mod = 1
)

image_id = names(x.uns$spatial)[1]

```

```

image_data = Image(x$uns$spatial[[image_id]]$images$ hires)/255

# find the scale factor
image_scale <- x$uns$spatial[[image_id]][["scalefactors"]][[paste("tissue_", "hires", "_scalef", sep="")]]

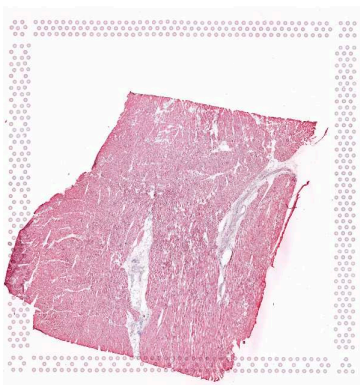
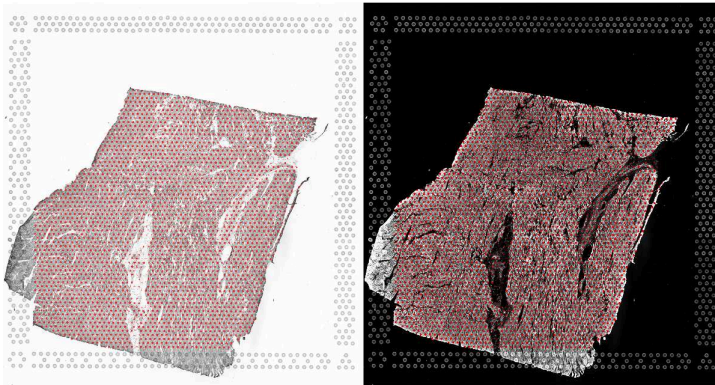
# Process Image:
# Color mode
colorMode(image_data) = "Color"

# plot the original image
display(image_data, method="raster", all=F)

```

1. Loading Spatial Dataset: Visium_3_CK281
2. Available sample IDs:
 - ... processing sample: Visium_3_CK281
3. "evaluate_image" set to TRUE:
 - ... Skipping spot matrix processing and gene expression writing
4. processing image(s).
 - ... pulling out image data from anndata file
 - ... applying selected transformations
5. Generating grey scale images and mask
 - ... "evaluate_image" is TRUE, plotting images for evaluation

IMAGE EVALUATION: Please check the plots to ensure adequate overlap of images and spots, as well as any adjustments needed for image orientation. Once you are happy, re-run the function with the argument "evaluate_image" set to FALSE. This will then process the gene expression matrix, and write the data to the directory for the shiny app to handle.



```
In [15]: # run again but flag evaluation as F
preprocessImage(
  image_h5ad = x,
  rotation = 0,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = F,
  feature_names = x$var$feature_name,
  nCounts_column = "n_counts",
  nGenes_column = "n_genes",
  QC_threshold = 600,
  contrast_mod = 1
)

png(paste("data/spatial/", image_id, "/unprocessed.png", sep=""), height=2000, width=2000)
display(image_data, method="raster", all=F)
dev.off()
```

1. Loading Spatial Dataset: Visium_3_CK281
2. Available sample IDs:
 - ... processing sample: Visium_3_CK281
3. "evaluate_image" set to FALSE:
 - ... Transforming spot matrix
 - ... Picking up the gene lookup table from: "data/gene_lookup_table.csv"
 - ... Picking up the spatial subset of gene lookup table from: "data/spatial/gene_lookup_spatial.csv"
 - ... Writing the genes into their files:
 - ... Writing the genes into their files:
- A/Z
 - ... Writing the genes into their files:
- B/Z
 - ... Writing the genes into their files:
- C/Z
 - ... Writing the genes into their files:
- D/Z
 - ... Writing the genes into their files:
- E/Z
 - ... Writing the genes into their files:
- F/Z
 - ... Writing the genes into their files:
- G/Z
 - ... Writing the genes into their files:
- H/Z
 - ... Writing the genes into their files:
- I/Z
 - ... Writing the genes into their files:
- J/Z
 - ... Writing the genes into their files:
- K/Z
 - ... Writing the genes into their files:
- L/Z
 - ... Writing the genes into their files:
- M/Z
 - ... Writing the genes into their files:
- N/Z
 - ... Writing the genes into their files:
- O/Z
 - ... Writing the genes into their files:
- P/Z
 - ... Writing the genes into their files:
- Q/Z
 - ... Writing the genes into their files:
- R/Z
 - ... Writing the genes into their files:
- S/Z
 - ... Writing the genes into their files:
- T/Z
 - ... Writing the genes into their files:
- U/Z
 - ... Writing the genes into their files:
- V/Z
 - ... Writing the genes into their files:
- W/Z
 - ... Writing the genes into their files:
- X/Z
 - ... Writing the genes into their files:
- Y/Z
 - ... Writing the genes into their files:
- Z/Z

4. processing image(s).
 - ... pulling out image data from anndata file
 - ... applying selected transformations
5. Generating grey scale images and mask
6. writing image 1 (light version): data/spatial/Visium_3_CK281/images/1/overlay.png
 - ... writing image 2 (dark version): data/spatial/Visium_3_CK281/images/2/overlay.png
7. writing datapoint coordinates to: data/spatial/Visium_3_CK281/coordinates.csv

IMAGE EVALUATION: no image evaluation was carried out. If transformations are needed, please re-run the function with the argument "evaluate_image" set to TRUE (default). This will skip the gene matrix processing and plot the images, re-run iteratively using the "rotation" and "reflection" arguments until the desired orientation is achieved.

agg_record_1521746628:2

```
In [16]: gc()
```

A matrix: 2 × 6 of type dbl

	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	2177288	116.3	4239520	226.5	3684547	196.8
Vcells	15127259	115.5	318113005	2427.1	469596127	3582.8

Healthy Left Ventricle # 4 "Visium_4_CK282"

```
In [17]: # Healthy Left Ventricle # 4 "Visium_4_CK282"
x = anndata.read_h5ad(datasets[4])
x$obs$sample = names(x$uns$spatial)[1]

# run once to evaluate image orientation etc
preprocessImage(
  image_h5ad = x,
  rotation = 0,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = T,
  feature_names = x$var$feature_name,
  nCounts_column = "n_counts",
  nGenes_column = "n_genes",
  QC_threshold = 600,
  contrast_mod = 1
)

image_id = names(x$uns$spatial)[1]
image_data = Image(x$uns$spatial[[image_id]]$images$ hires)/255

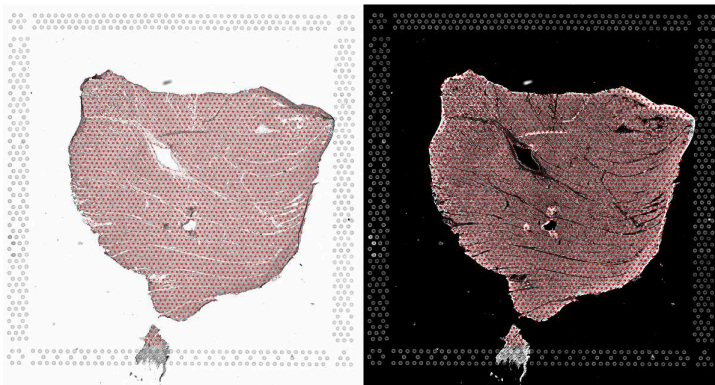
# find the scale factor
image_scale <- x$uns$spatial[[image_id]][["scalefactors"]][[paste("tissue_", "hires", "_scalef", sep="")]]

# Process Image:
# Color mode
colorMode(image_data) = "Color"

# plot the original image
display(image_data, method="raster", all=F)
```

1. Loading Spatial Dataset: Visium_4_CK282
2. Available sample IDs:
... processing sample: Visium_4_CK282
3. "evaluate_image" set to TRUE:
... Skipping spot matrix processing and gene expression writing
4. processing image(s).
... pulling out image data from anndata file
... applying selected transformations
5. Generating grey scale images and mask
... "evaluate_image" is TRUE, plotting images for evaluation

IMAGE EVALUATION: Please check the plots to ensure adequate overlap of images and spots, as well as any adjustments needed for image orientation. Once you are happy, re-run the function with the argument "evaluate_image" set to FALSE. This will then process the gene expression matrix, and write the data to the directory for the shiny app to handle.





```
In [18]: # run again but flag evaluation as F
preprocessImage(
  image_h5ad = x,
  rotation = 0,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = F,
  feature_names = x$var$feature_name,
  nCounts_column = "n_counts",
  nGenes_column = "n_genes",
  QC_threshold = 600,
  contrast_mod = 1
)

png(paste("data/spatial/", image_id, "/unprocessed.png", sep=""), height=2000, width=2000)
display(image_data, method="raster", all=F)
dev.off()
```

```

1. Loading Spatial Dataset: Visium_4_CK282
2. Available sample IDs:
... processing sample: Visium_4_CK282
3. "evaluate_image" set to FALSE:
... Transforming spot matrix
... Picking up the gene lookup table from: "data/gene_lookup_table.csv"
... Picking up the spatial subset of gene lookup table from: "data/spatial/gene_lookup_spatial.csv"
... Writing the genes into their files:
... Writing the genes into their files:
A/Z
... Writing the genes into their files:
B/Z
... Writing the genes into their files:
C/Z
... Writing the genes into their files:
D/Z
... Writing the genes into their files:
E/Z
... Writing the genes into their files:
F/Z
... Writing the genes into their files:
G/Z
... Writing the genes into their files:
H/Z
... Writing the genes into their files:
I/Z
... Writing the genes into their files:
J/Z
... Writing the genes into their files:
K/Z
... Writing the genes into their files:
L/Z
... Writing the genes into their files:
M/Z
... Writing the genes into their files:
N/Z
... Writing the genes into their files:
O/Z
... Writing the genes into their files:
P/Z
... Writing the genes into their files:
Q/Z
... Writing the genes into their files:
R/Z
... Writing the genes into their files:
S/Z
... Writing the genes into their files:
T/Z
... Writing the genes into their files:
U/Z
... Writing the genes into their files:
V/Z
... Writing the genes into their files:
W/Z
... Writing the genes into their files:
X/Z
... Writing the genes into their files:
Y/Z
... Writing the genes into their files:
Z/Z

4. processing image(s).
... pulling out image data from anndata file
... applying selected transformations
5. Generating grey scale images and mask
6. writing image 1 (light version): data/spatial/Visium_4_CK282/images/1/overlay.png
... writing image 2 (dark version): data/spatial/Visium_4_CK282/images/2/overlay.png
7. writing datapoint coordinates to: data/spatial/Visium_4_CK282/coordinates.csv

```

IMAGE EVALUATION: no image evaluation was carried out. If transformations are needed, please re-run the function with the argument "evaluate_image" set to TRUE (default). This will skip the gene matrix processing and plot the images, re-run iteratively using the "rotation" and "reflection" arguments until the desired orientation is achieved.

agg_record_418121744: 2

INFARCTED Left Ventricle # 1 "Visium_9_CK287" - good?

```

In [24]: # INFARCTED Left Ventricle # 1 "Visium_9_CK287" - good?
x = anndata::read_h5ad(datasets[5])
x$obs$sample = names(x$uns$spatial)[1]

preprocessImage(
  image_h5ad = x,
  rotation = 90,
  reflection = "horizontal",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = T,
  feature_names = x$var$feature_name,
  nCounts_column = "n_counts",
  nGenes_column = "n_genes",
  QC_threshold = 600,
  contrast_mod = 1
)

image_id = names(x$uns$spatial)[1]
image_data = Image(x$uns$spatial[[image_id]]$images$ hires)/255

```



```

# find the scale factor
image_scale <- x$uns$spatial[[image_id]][["scalefactors"]][["paste("tissue_", "hires", "_scalef", sep=")"]]

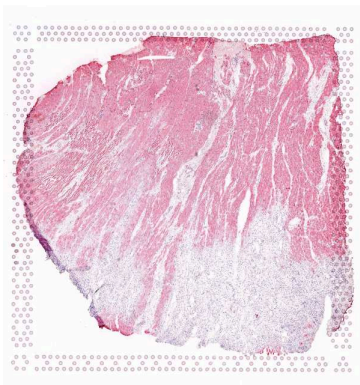
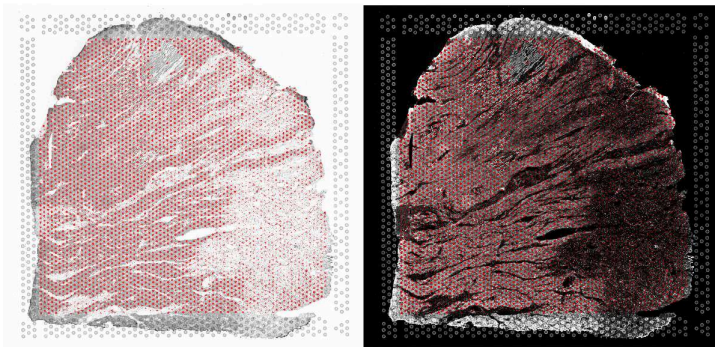
# Process Image:
# Color mode
colorMode(image_data) = "Color"

# plot the original image
display(image_data, method="raster", all=F)

```

1. Loading Spatial Dataset: Visium_9_CK287
2. Available sample IDs:
- ... processing sample: Visium_9_CK287
3. "evaluate_image" set to TRUE:
- ... Skipping spot matrix processing and gene expression writing
4. processing image(s).
- ... pulling out image data from anndata file
- ... applying selected transformations
5. Generating grey scale images and mask
- ... "evaluate_image" is TRUE, plotting images for evaluation

IMAGE EVALUATION: Please check the plots to ensure adequate overlap of images and spots, as well as any adjustments needed for image orientation. Once you are happy, re-run the function with the argument "evaluate_image" set to FALSE. This will then process the gene expression matrix, and write the data to the directory for the shiny app to handle.



```

In [25]: # run again but flag evaluation as F
preprocessImage(
  image_h5ad = x,

```

```

rotation = 90,
reflection = "horizontal",
opacity_filter = 0.9,
sample_id = unique(x$obs$sample),
evaluate_image = F,
feature_names = x$var$feature_name,
nCounts_column = "n_counts",
nGenes_column = "n_genes",
QC_threshold = 600,
contrast_mod = 1
)

png(paste("data/spatial/", image_id, "/unprocessed.png", sep=""), height=2000, width=2000)
display(image_data, method="raster", all=F)
dev.off()

```

```

1. Loading Spatial Dataset: Visium_9_CK287
2. Available sample IDs:
... processing sample: Visium_9_CK287
3. "evaluate_image" set to FALSE:
... Transforming spot matrix

```

Warning message in asMethod(object):

"sparse->dense coercion: allocating vector of size 1.1 GiB"

```

... Picking up the gene lookup table from: "data/gene_lookup_table.csv"
... Picking up the spatial subset of gene lookup table from: "data/spatial/gene_lookup_spatial.csv"
... Writing the genes into their files:
... Writing the genes into their files:
A/Z
... Writing the genes into their files:
B/Z
... Writing the genes into their files:
C/Z
... Writing the genes into their files:
D/Z
... Writing the genes into their files:
E/Z
... Writing the genes into their files:
F/Z
... Writing the genes into their files:
G/Z
... Writing the genes into their files:
H/Z
... Writing the genes into their files:
I/Z
... Writing the genes into their files:
J/Z
... Writing the genes into their files:
K/Z
... Writing the genes into their files:
L/Z
... Writing the genes into their files:
M/Z
... Writing the genes into their files:
N/Z
... Writing the genes into their files:
O/Z
... Writing the genes into their files:
P/Z
... Writing the genes into their files:
Q/Z
... Writing the genes into their files:
R/Z
... Writing the genes into their files:
S/Z
... Writing the genes into their files:
T/Z
... Writing the genes into their files:
U/Z
... Writing the genes into their files:
V/Z
... Writing the genes into their files:
W/Z
... Writing the genes into their files:
X/Z
... Writing the genes into their files:
Y/Z
... Writing the genes into their files:
Z/Z

```

```

4. processing image(s).
... pulling out image data from anndata file
... applying selected transformations
5. Generating grey scale images and mask
6. writing image 1 (light version): data/spatial/Visium_9_CK287/images/1/overlay.png
... writing image 2 (dark version): data/spatial/Visium_9_CK287/images/2/overlay.png
7. writing datapoint coordinates to: data/spatial/Visium_9_CK287/coordinates.csv

```

IMAGE EVALUATION: no image evaluation was carried out. If transformations are needed, please re-run the function with the argument "evaluate_image" set to TRUE (default). This will skip the gene matrix processing and plot the images, re-run iteratively using the "rotation" and "reflection" arguments until the desired orientation is achieved.

agg_record_867092732: 2

INFARCTED Left Ventricle # 2 "Visium_7_CK285"

```

In [26]: # INFARCTED Left Ventricle # 2 "Visium_7_CK285"
         x = anndata::read_h5ad(datasets[6])
         x$obs$sample = names(x$uns$spatial)[1]

```

```

preprocessImage(
  image_h5ad = x,
  rotation = 0,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = T,
  feature_names = x$var$feature_name,
  nCounts_column = "n_counts",
  nGenes_column = "n_genes",
  QC_threshold = 600,
  contrast_mod = 1
)

image_id = names(x$uns$spatial)[1]
image_data = Image(x$uns$spatial[[image_id]]$images$ hires)/255

# find the scale factor
image_scale <- x$uns$spatial[[image_id]][["scalefactors"]][["paste("tissue_", "hires", "_scalef", sep=")"]]

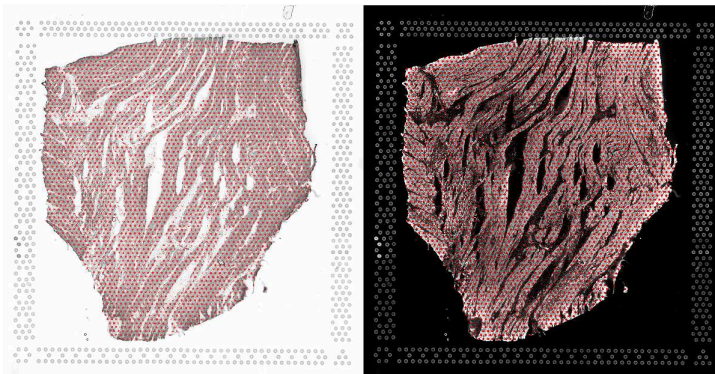
# Process Image:
# Color mode
colorMode(image_data) = "Color"

# plot the original image
display(image_data, method="raster", all=F)

```

1. Loading Spatial Dataset: Visium_7_CK285
2. Available sample IDs:
... processing sample: Visium_7_CK285
3. "evaluate_image" set to TRUE:
... Skipping spot matrix processing and gene expression writing
4. processing image(s).
... pulling out image data from anndata file
... applying selected transformations
5. Generating grey scale images and mask
... "evaluate_image" is TRUE, plotting images for evaluation

IMAGE EVALUATION: Please check the plots to ensure adequate overlap of images and spots, as well as any adjustments needed for image orientation. Once you are happy, re-run the function with the argument "evaluate_image" set to FALSE. This will then process the gene expression matrix, and write the data to the directory for the shiny app to handle.





```
In [27]: # run again but flag evaluation as F
preprocessImage(
  image_h5ad = x,
  rotation = 0,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = F,
  feature_names = x$var$feature_name,
  nCounts_column = "n_counts",
  nGenes_column = "n_genes",
  QC_threshold = 600,
  contrast_mod = 1
)

png(paste("data/spatial/", image_id, "/unprocessed.png", sep=""), height=2000, width=2000)
display(image_data, method="raster", all=F)
dev.off()
```

```

1. Loading Spatial Dataset:  Visium_7_CK285
2. Available sample IDs:
... processing sample:  Visium_7_CK285
3. "evaluate_image" set to FALSE:
... Transforming spot matrix
... Picking up the gene lookup table from: "data/gene_lookup_table.csv"
... Picking up the spatial subset of gene lookup table from: "data/spatial/gene_lookup_spatial.csv"
... Writing the genes into their files:
... Writing the genes into their files:
A/Z
... Writing the genes into their files:
B/Z
... Writing the genes into their files:
C/Z
... Writing the genes into their files:
D/Z
... Writing the genes into their files:
E/Z
... Writing the genes into their files:
F/Z
... Writing the genes into their files:
G/Z
... Writing the genes into their files:
H/Z
... Writing the genes into their files:
I/Z
... Writing the genes into their files:
J/Z
... Writing the genes into their files:
K/Z
... Writing the genes into their files:
L/Z
... Writing the genes into their files:
M/Z
... Writing the genes into their files:
N/Z
... Writing the genes into their files:
O/Z
... Writing the genes into their files:
P/Z
... Writing the genes into their files:
Q/Z
... Writing the genes into their files:
R/Z
... Writing the genes into their files:
S/Z
... Writing the genes into their files:
T/Z
... Writing the genes into their files:
U/Z
... Writing the genes into their files:
V/Z
... Writing the genes into their files:
W/Z
... Writing the genes into their files:
X/Z
... Writing the genes into their files:
Y/Z
... Writing the genes into their files:
Z/Z

4. processing image(s).
... pulling out image data from anndata file
... applying selected transformations
5. Generating grey scale images and mask
6. writing image 1 (light version):  data/spatial/Visium_7_CK285/images/1/overlay.png
... writing image 2 (dark version):  data/spatial/Visium_7_CK285/images/2/overlay.png
7. writing datapoint coordinates to:  data/spatial/Visium_7_CK285/coordinates.csv

```

IMAGE EVALUATION: no image evaluation was carried out. If transformations are needed, please re-run the function with the argument "evaluate_image" set to TRUE (default). This will skip the gene matrix processing and plot the images, re-run iteratively using the "rotation" and "reflection" arguments until the desired orientation is achieved.

agg_record_1180723703: 2

In [28]: `gc()`

A matrix: 2 × 6 of type dbl

	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	2187749	116.9	4239520	226.5	3684547	196.8
Vcells	15487444	118.2	388688249	2965.5	545479862	4161.7

INFARCTED Left Ventricle # 3 "Visium_20_CK298"

```

In [30]: # INFARCTED Left Ventricle # 3 "Visium_20_CK298"
x = anndata::read_h5ad(datasets[7])
x$obs$sample = names(x$uns$spatial)[1]

preprocessImage(
  image_h5ad = x,
  rotation = 0,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = T,
  feature_names = x$var$feature_name,
  nCounts_column = "n_counts",

```

```

nGenes_column = "n_genes",
QC_threshold = 600,
contrast_mod = 1
)

image_id = names(x$uns$spatial)[1]
image_data = Image(x$uns$spatial[[image_id]]$images$ hires)/255

# find the scale factor
image_scale <- x$uns$spatial[[image_id]][["scalefactors"]][[paste("tissue_", "hires", "_scalef", sep="")]]

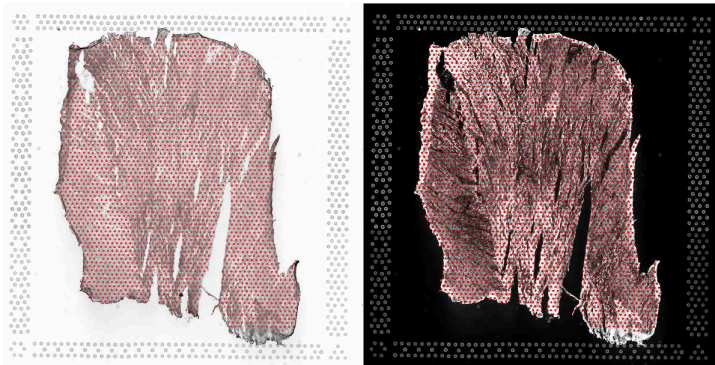
# Process Image:
# Color mode
colorMode(image_data) = "Color"

# plot the original image
display(image_data, method="raster", all=F)

```

1. Loading Spatial Dataset: Visium_20_CK298
2. Available sample IDs:
 - ... processing sample: Visium_20_CK298
3. "evaluate_image" set to TRUE:
 - ... Skipping spot matrix processing and gene expression writing
4. processing image(s).
 - ... pulling out image data from anndata file
 - ... applying selected transformations
5. Generating grey scale images and mask
 - ... "evaluate_image" is TRUE, plotting images for evaluation

IMAGE EVALUATION: Please check the plots to ensure adequate overlap of images and spots, as well as any adjustments needed for image orientation. Once you are happy, re-run the function with the argument "evaluate_image" set to FALSE. This will then process the gene expression matrix, and write the data to the directory for the shiny app to handle.





```
In [31]: # run again but flag evaluation as F
preprocessImage(
  image_h5ad = x,
  rotation = 0,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = F,
  feature_names = x$var$feature_name,
  nCounts_column = "n_counts",
  nGenes_column = "n_genes",
  QC_threshold = 600,
  contrast_mod = 1
)

png(paste("data/spatial/", image_id, "/unprocessed.png", sep=""), height=2000, width=2000)
display(image_data, method="raster", all=F)
dev.off()
```

```

1. Loading Spatial Dataset: Visium_20_CK298
2. Available sample IDs:
... processing sample: Visium_20_CK298
3. "evaluate_image" set to FALSE:
... Transforming spot matrix
... Picking up the gene lookup table from: "data/gene_lookup_table.csv"
... Picking up the spatial subset of gene lookup table from: "data/spatial/gene_lookup_spatial.csv"
... Writing the genes into their files:
... Writing the genes into their files:
A/Z
... Writing the genes into their files:
B/Z
... Writing the genes into their files:
C/Z
... Writing the genes into their files:
D/Z
... Writing the genes into their files:
E/Z
... Writing the genes into their files:
F/Z
... Writing the genes into their files:
G/Z
... Writing the genes into their files:
H/Z
... Writing the genes into their files:
I/Z
... Writing the genes into their files:
J/Z
... Writing the genes into their files:
K/Z
... Writing the genes into their files:
L/Z
... Writing the genes into their files:
M/Z
... Writing the genes into their files:
N/Z
... Writing the genes into their files:
O/Z
... Writing the genes into their files:
P/Z
... Writing the genes into their files:
Q/Z
... Writing the genes into their files:
R/Z
... Writing the genes into their files:
S/Z
... Writing the genes into their files:
T/Z
... Writing the genes into their files:
U/Z
... Writing the genes into their files:
V/Z
... Writing the genes into their files:
W/Z
... Writing the genes into their files:
X/Z
... Writing the genes into their files:
Y/Z
... Writing the genes into their files:
Z/Z

4. processing image(s).
... pulling out image data from anndata file
... applying selected transformations
5. Generating grey scale images and mask
6. writing image 1 (light version): data/spatial/Visium_20_CK298/images/1/overlay.png
... writing image 2 (dark version): data/spatial/Visium_20_CK298/images/2/overlay.png
7. writing datapoint coordinates to: data/spatial/Visium_20_CK298/coordinates.csv

IMAGE EVALUATION: no image evaluation was carried out. If transformations are needed, please re-run the function with the argument
"evaluate_image" set to TRUE (default). This will skip the gene matrix processing and plot the images, re-run iteratively using the
"rotation" and "reflection" arguments until the desired orientation is achieved.

```

agg_record_254050905:2

In [32]: `gc()`

A matrix: 2 × 6 of type dbl

	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	2187749	116.9	4239520	226.5	3684547	196.8
Vcells	15751444	120.2	334651446	2553.2	545479862	4161.7

INFARCTED Left Ventricle # 4 "Visium_15_CK293" - BAD one, there is a terrible distribution of spots

```

In [33]: # INFARCTED Left Ventricle # 4 "Visium_15_CK293" - BAD one, there is a terrible distribution of spots across this tissue...
# some regions have low UMIs
x = anndata::read_h5ad(datasets[8])
x$obs$sample = names(x$uns$spatial)[1]

preprocessImage(
  image_h5ad = x,
  rotation = 270,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = T,
  feature_names = x$var$feature_name,

```



```

nCounts_column = "n_counts",
nGenes_column = "n_genes",
QC_threshold = 600,
contrast_mod = 1
)

image_id = names(x$uns$spatial)[1]
image_data = Image(x$uns$spatial[[image_id]]$images$ hires)/255

# find the scale factor
image_scale <- x$uns$spatial[[image_id]][["scalefactors"]][[paste("tissue_", "hires", "_scalef", sep="")]]

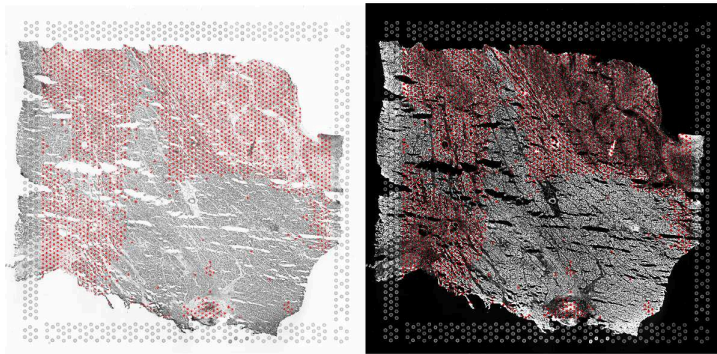
# Process Image:
# Color mode
colorMode(image_data) = "Color"

# plot the original image
display(image_data, method="raster", all=F)

```

1. Loading Spatial Dataset: Visium_15_CK293
2. Available sample IDs:
 - ... processing sample: Visium_15_CK293
3. "evaluate_image" set to TRUE:
 - ... Skipping spot matrix processing and gene expression writing
4. processing image(s).
 - ... pulling out image data from anndata file
 - ... applying selected transformations
5. Generating grey scale images and mask
 - ... "evaluate_image" is TRUE, plotting images for evaluation

IMAGE EVALUATION: Please check the plots to ensure adequate overlap of images and spots, as well as any adjustments needed for image orientation. Once you are happy, re-run the function with the argument "evaluate_image" set to FALSE. This will then process the gene expression matrix, and write the data to the directory for the shiny app to handle.





```
In [34]: # run again but flag evaluation as F
preprocessImage(
  image_h5ad = x,
  rotation = 270,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = F,
  feature_names = x$var$feature_name,
  nCounts_column = "n_counts",
  nGenes_column = "n_genes",
  QC_threshold = 600,
  contrast_mod = 1
)

png(paste("data/spatial/", image_id, "/unprocessed.png", sep=""), height=2000, width=2000)
display(image_data, method="raster", all=F)
dev.off()
```

```

1. Loading Spatial Dataset: Visium_15_CK293
2. Available sample IDs:
... processing sample: Visium_15_CK293
3. "evaluate_image" set to FALSE:
... Transforming spot matrix
... Picking up the gene lookup table from: "data/gene_lookup_table.csv"
... Picking up the spatial subset of gene lookup table from: "data/spatial/gene_lookup_spatial.csv"
... Writing the genes into their files:
... Writing the genes into their files:
A/Z
... Writing the genes into their files:
B/Z
... Writing the genes into their files:
C/Z
... Writing the genes into their files:
D/Z
... Writing the genes into their files:
E/Z
... Writing the genes into their files:
F/Z
... Writing the genes into their files:
G/Z
... Writing the genes into their files:
H/Z
... Writing the genes into their files:
I/Z
... Writing the genes into their files:
J/Z
... Writing the genes into their files:
K/Z
... Writing the genes into their files:
L/Z
... Writing the genes into their files:
M/Z
... Writing the genes into their files:
N/Z
... Writing the genes into their files:
O/Z
... Writing the genes into their files:
P/Z
... Writing the genes into their files:
Q/Z
... Writing the genes into their files:
R/Z
... Writing the genes into their files:
S/Z
... Writing the genes into their files:
T/Z
... Writing the genes into their files:
U/Z
... Writing the genes into their files:
V/Z
... Writing the genes into their files:
W/Z
... Writing the genes into their files:
X/Z
... Writing the genes into their files:
Y/Z
... Writing the genes into their files:
Z/Z

4. processing image(s).
... pulling out image data from anndata file
... applying selected transformations
5. Generating grey scale images and mask
6. writing image 1 (light version): data/spatial/Visium_15_CK293/images/1/overlay.png
... writing image 2 (dark version): data/spatial/Visium_15_CK293/images/2/overlay.png
7. writing datapoint coordinates to: data/spatial/Visium_15_CK293/coordinates.csv

```

IMAGE EVALUATION: no image evaluation was carried out. If transformations are needed, please re-run the function with the argument "evaluate_image" set to TRUE (default). This will skip the gene matrix processing and plot the images, re-run iteratively using the "rotation" and "reflection" arguments until the desired orientation is achieved.

agg_record_164948217:2

INFARCTED Left Ventricle # 5 "AKK002_Infarct_MI_acute_V19T05025_rep2" - Has the obvious SMC patch

```

In [35]: # INFARCTED Left Ventricle # 5 "AKK002_Infarct_MI_acute_V19T05025_rep2" - Has the obvious SMC patch
x = anndata::read_h5ad(datasets[9])
x$obs$sample = names(x$uns$spatial)[1]

preprocessImage(
  image_h5ad = x,
  rotation = 0,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = T,
  feature_names = x$var$feature_name,
  nCounts_column = "n_counts",
  nGenes_column = "n_genes",
  QC_threshold = 600,
  contrast_mod = 3 # reaaaally low contrast on this slide
)

image_id = names(x$uns$spatial)[1]
image_data = Image(x$uns$spatial[[image_id]]$images$ hires)/255

```

```

# find the scale factor
image_scale <- x$uns$spatial[[image_id]][["scalefactors"]][["paste("tissue_", "hires", "_scalef", sep=")"]]

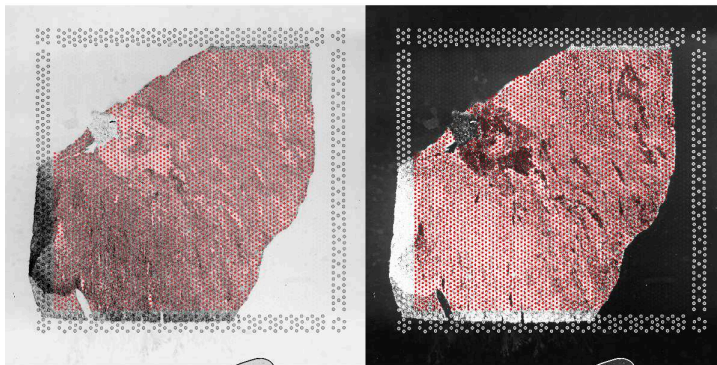
# Process Image:
# Color mode
colorMode(image_data) = "Color"

# plot the original image
display(image_data, method="raster", all=F)

```

1. Loading Spatial Dataset: AKK002_Infarct_MI_acute_V19T05025_rep2
2. Available sample IDs:
- ... processing sample: AKK002_Infarct_MI_acute_V19T05025_rep2
3. "evaluate_image" set to TRUE:
- ... Skipping spot matrix processing and gene expression writing
4. processing image(s).
- ... pulling out image data from anndata file
- ... applying selected transformations
5. Generating grey scale images and mask
- ... "evaluate_image" is TRUE, plotting images for evaluation

IMAGE EVALUATION: Please check the plots to ensure adequate overlap of images and spots, as well as any adjustments needed for image orientation. Once you are happy, re-run the function with the argument "evaluate_image" set to FALSE. This will then process the gene expression matrix, and write the data to the directory for the shiny app to handle.



```

In [36]: # run again but flag evaluation as F
preprocessImage(
  image_h5ad = x,

```

```

rotation = 0,
reflection = "none",
opacity_filter = 0.9,
sample_id = unique(x$obs$sample),
evaluate_image = F,
feature_names = x$var$feature_name,
nCounts_column = "n_counts",
nGenes_column = "n_genes",
QC_threshold = 600,
contrast_mod = 3
)

png(paste("data/spatial/", image_id, "/unprocessed.png", sep=""), height=2000, width=2000)
display(image_data, method="raster", all=F)
dev.off()

```

```

1. Loading Spatial Dataset: AKK002_Infarct_MI_acute_V19T05025_rep2
2. Available sample IDs:
... processing sample: AKK002_Infarct_MI_acute_V19T05025_rep2
3. "evaluate_image" set to FALSE:
... Transforming spot matrix
... Picking up the gene lookup table from: "data/gene_lookup_table.csv"
... Picking up the spatial subset of gene lookup table from: "data/spatial/gene_lookup_spatial.csv"
... Writing the genes into their files:
... Writing the genes into their files:
A/Z
... Writing the genes into their files:
B/Z
... Writing the genes into their files:
C/Z
... Writing the genes into their files:
D/Z
... Writing the genes into their files:
E/Z
... Writing the genes into their files:
F/Z
... Writing the genes into their files:
G/Z
... Writing the genes into their files:
H/Z
... Writing the genes into their files:
I/Z
... Writing the genes into their files:
J/Z
... Writing the genes into their files:
K/Z
... Writing the genes into their files:
L/Z
... Writing the genes into their files:
M/Z
... Writing the genes into their files:
N/Z
... Writing the genes into their files:
O/Z
... Writing the genes into their files:
P/Z
... Writing the genes into their files:
Q/Z
... Writing the genes into their files:
R/Z
... Writing the genes into their files:
S/Z
... Writing the genes into their files:
T/Z
... Writing the genes into their files:
U/Z
... Writing the genes into their files:
V/Z
... Writing the genes into their files:
W/Z
... Writing the genes into their files:
X/Z
... Writing the genes into their files:
Y/Z
... Writing the genes into their files:
Z/Z

4. processing image(s).
... pulling out image data from anndata file
... applying selected transformations
5. Generating grey scale images and mask
6. writing image 1 (light version): data/spatial/AKK002_Infarct_MI_acute_V19T05025_rep2/images/1/overlay.png
... writing image 2 (dark version): data/spatial/AKK002_Infarct_MI_acute_V19T05025_rep2/images/2/overlay.png
7. writing datapoint coordinates to: data/spatial/AKK002_Infarct_MI_acute_V19T05025_rep2/coordinates.csv

IMAGE EVALUATION: no image evaluation was carried out. If transformations are needed, please re-run the function with the argument
"evaluate_image" set to TRUE (default). This will skip the gene matrix processing and plot the images, re-run iteratively using the
"rotation" and "reflection" arguments until the desired orientation is achieved.

```

agg_record_876441125: 2

In [37]: `gc()`

A matrix: 2 × 6 of type dbl

	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	2187749	116.9	4239520	226.5	3684547	196.8
Vcells	15871444	121.1	402033644	3067.3	545479862	4161.7

INFARCTED RZ Left Ventricle # 1 "AKK002_Normal_MI_acute_V19T05026_rep1"

```
In [38]: # INFARCTED RZ Left Ventricle # 1 "AKK002_Normal_MI_acute_V19T05026_rep1"
x = anndata::read_h5ad(datasets[10])
x$obs$sample = names(x$uns$spatial)[1]

preprocessImage(
  image_h5ad = x,
  rotation = 0,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = T,
  feature_names = x$var$feature_name,
  nCounts_column = "n_counts",
  nGenes_column = "n_genes",
  QC_threshold = 600,
  contrast_mod = 1.5
)

image_id = names(x$uns$spatial)[1]
image_data = Image(x$uns$spatial[[image_id]]$images$ hires)/255

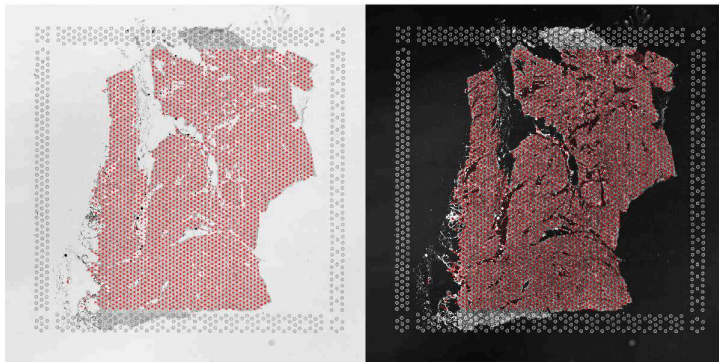
# find the scale factor
image_scale <- x$uns$spatial[[image_id]][["scalefactors"]][[paste("tissue_", "hires", "_scalef", sep="")]]

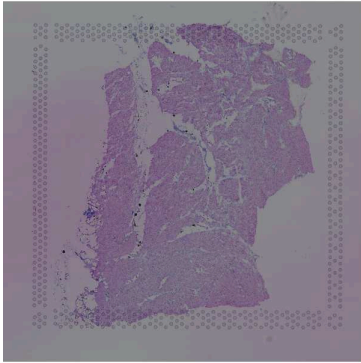
# Process Image:
# Color mode
colorMode(image_data) = "Color"

# plot the original image
display(image_data, method="raster", all=F)
```

1. Loading Spatial Dataset: AKK002_Normal_MI_acute_V19T05026_rep1
2. Available sample IDs:
 - ... processing sample: AKK002_Normal_MI_acute_V19T05026_rep1
3. "evaluate_image" set to TRUE:
 - ... Skipping spot matrix processing and gene expression writing
4. processing image(s).
 - ... pulling out image data from anndata file
 - ... applying selected transformations
5. Generating grey scale images and mask
 - ... "evaluate_image" is TRUE, plotting images for evaluation

IMAGE EVALUATION: Please check the plots to ensure adequate overlap of images and spots, as well as any adjustments needed for image orientation. Once you are happy, re-run the function with the argument "evaluate_image" set to FALSE. This will then process the gene expression matrix, and write the data to the directory for the shiny app to handle.





```
In [39]: # run again but flag evaluation as F
preprocessImage(
  image_h5ad = x,
  rotation = 0,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = F,
  feature_names = x$var$feature_name,
  nCounts_column = "n_counts",
  nGenes_column = "n_genes",
  QC_threshold = 600,
  contrast_mod = 1.5
)

png(paste("data/spatial/", image_id, "/unprocessed.png", sep=""), height=2000, width=2000)
display(image_data, method="raster", all=F)
dev.off()
```

```

1. Loading Spatial Dataset: AKK002_Normal_MI_acute_V19T05026_rep1
2. Available sample IDs:
... processing sample: AKK002_Normal_MI_acute_V19T05026_rep1
3. "evaluate_image" set to FALSE:
... Transforming spot matrix
... Picking up the gene lookup table from: "data/gene_lookup_table.csv"
... Picking up the spatial subset of gene lookup table from: "data/spatial/gene_lookup_spatial.csv"
... Writing the genes into their files:
... Writing the genes into their files:
A/Z
... Writing the genes into their files:
B/Z
... Writing the genes into their files:
C/Z
... Writing the genes into their files:
D/Z
... Writing the genes into their files:
E/Z
... Writing the genes into their files:
F/Z
... Writing the genes into their files:
G/Z
... Writing the genes into their files:
H/Z
... Writing the genes into their files:
I/Z
... Writing the genes into their files:
J/Z
... Writing the genes into their files:
K/Z
... Writing the genes into their files:
L/Z
... Writing the genes into their files:
M/Z
... Writing the genes into their files:
N/Z
... Writing the genes into their files:
O/Z
... Writing the genes into their files:
P/Z
... Writing the genes into their files:
Q/Z
... Writing the genes into their files:
R/Z
... Writing the genes into their files:
S/Z
... Writing the genes into their files:
T/Z
... Writing the genes into their files:
U/Z
... Writing the genes into their files:
V/Z
... Writing the genes into their files:
W/Z
... Writing the genes into their files:
X/Z
... Writing the genes into their files:
Y/Z
... Writing the genes into their files:
Z/Z

4. processing image(s).
... pulling out image data from anndata file
... applying selected transformations
5. Generating grey scale images and mask
6. writing image 1 (light version): data/spatial/AKK002_Normal_MI_acute_V19T05026_rep1/images/1/overlay.png
... writing image 2 (dark version): data/spatial/AKK002_Normal_MI_acute_V19T05026_rep1/images/2/overlay.png
7. writing datapoint coordinates to: data/spatial/AKK002_Normal_MI_acute_V19T05026_rep1/coordinates.csv

```

IMAGE EVALUATION: no image evaluation was carried out. If transformations are needed, please re-run the function with the argument "evaluate_image" set to TRUE (default). This will skip the gene matrix processing and plot the images, re-run iteratively using the "rotation" and "reflection" arguments until the desired orientation is achieved.

agg_record_680385228: 2

INFARCTED RZ # 2 "Visium_2_CK280"

```

In [40]: # INFARCTED LZ left Ventricle # 2 "Visium_2_CK280"
x = anndata::read_h5ad(datasets[11])
x$obs$sample = names(x$uns$spatial)[1]

preprocessImage(
  image_h5ad = x,
  rotation = 0,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = T,
  feature_names = x$var$feature_name,
  nCounts_column = "nCount_Spatial",
  nGenes_column = "nFeature_Spatial",
  QC_threshold = 600,
  contrast_mod = 1
)

image_id = names(x$uns$spatial)[1]
image_data = Image(x$uns$spatial[[image_id]]$images$ hires)/255

```



```

# find the scale factor
image_scale <- x$uns$spatial[[image_id]][["scalefactors"]][["paste("tissue_", "hires", "_scalef", sep=")"]]

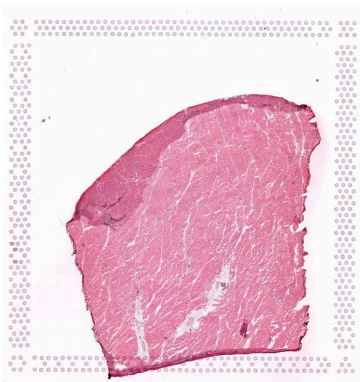
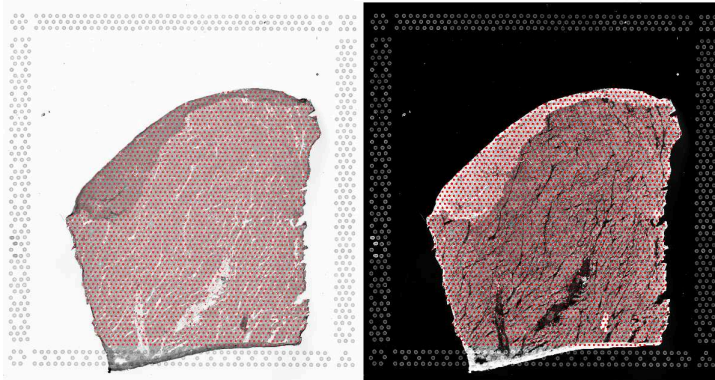
# Process Image:
# Color mode
colorMode(image_data) = "Color"

# plot the original image
display(image_data, method="raster", all=F)

```

1. Loading Spatial Dataset: Visium_2_CK280
2. Available sample IDs:
- ... processing sample: Visium_2_CK280
3. "evaluate_image" set to TRUE:
- ... Skipping spot matrix processing and gene expression writing
4. processing image(s).
- ... pulling out image data from anndata file
- ... applying selected transformations
5. Generating grey scale images and mask
- ... "evaluate_image" is TRUE, plotting images for evaluation

IMAGE EVALUATION: Please check the plots to ensure adequate overlap of images and spots, as well as any adjustments needed for image orientation. Once you are happy, re-run the function with the argument "evaluate_image" set to FALSE. This will then process the gene expression matrix, and write the data to the directory for the shiny app to handle.



```

In [41]: # run again but flag evaluation as F
preprocessImage(
  image_h5ad = x,

```

```

rotation = 0,
reflection = "none",
opacity_filter = 0.9,
sample_id = unique(x$obs$sample),
evaluate_image = F,
feature_names = x$var$feature_name,
nCounts_column = "nCount_Spatial",
nGenes_column = "nFeature_Spatial",
QC_threshold = 600,
contrast_mod = 1
)

png(paste("data/spatial/", image_id, "/unprocessed.png", sep=""), height=2000, width=2000)
display(image_data, method="raster", all=F)
dev.off()

```

```

1. Loading Spatial Dataset: Visium_2_CK280
2. Available sample IDs:
... processing sample: Visium_2_CK280
3. "evaluate_image" set to FALSE:
... Transforming spot matrix
... Picking up the gene lookup table from: "data/gene_lookup_table.csv"
... Picking up the spatial subset of gene lookup table from: "data/spatial/gene_lookup_spatial.csv"
... Writing the genes into their files:
... Writing the genes into their files:
A/Z
... Writing the genes into their files:
B/Z
... Writing the genes into their files:
C/Z
... Writing the genes into their files:
D/Z
... Writing the genes into their files:
E/Z
... Writing the genes into their files:
F/Z
... Writing the genes into their files:
G/Z
... Writing the genes into their files:
H/Z
... Writing the genes into their files:
I/Z
... Writing the genes into their files:
J/Z
... Writing the genes into their files:
K/Z
... Writing the genes into their files:
L/Z
... Writing the genes into their files:
M/Z
... Writing the genes into their files:
N/Z
... Writing the genes into their files:
O/Z
... Writing the genes into their files:
P/Z
... Writing the genes into their files:
Q/Z
... Writing the genes into their files:
R/Z
... Writing the genes into their files:
S/Z
... Writing the genes into their files:
T/Z
... Writing the genes into their files:
U/Z
... Writing the genes into their files:
V/Z
... Writing the genes into their files:
W/Z
... Writing the genes into their files:
X/Z
... Writing the genes into their files:
Y/Z
... Writing the genes into their files:
Z/Z

4. processing image(s).
... pulling out image data from anndata file
... applying selected transformations
5. Generating grey scale images and mask
6. writing image 1 (light version): data/spatial/Visium_2_CK280/images/1/overlay.png
... writing image 2 (dark version): data/spatial/Visium_2_CK280/images/2/overlay.png
7. writing datapoint coordinates to: data/spatial/Visium_2_CK280/coordinates.csv

```

IMAGE EVALUATION: no image evaluation was carried out. If transformations are needed, please re-run the function with the argument "evaluate_image" set to TRUE (default). This will skip the gene matrix processing and plot the images, re-run iteratively using the "rotation" and "reflection" arguments until the desired orientation is achieved.

agg_record_1541248798: 2

INFARCTED RZ Left Ventricle # 3 "Visium_17_CK295"

```

In [42]: # INFARCTED RZ Left Ventricle # 3 "Visium_17_CK295"
x = anndata::read_h5ad(datasets[12])
x$obs$sample = names(x$uns$spatial)[1]

preprocessImage(
  image_h5ad = x,
  rotation = 0,

```

```

reflection = "none",
opacity_filter = 0.9,
sample_id = unique(x$obs$sample),
evaluate_image = T,
feature_names = x$var$feature_name,
nCounts_column = "n_counts",
nGenes_column = "n_genes",
QC_threshold = 600,
contrast_mod = 1
)

image_id = names(x$uns$spatial)[1]
image_data = Image(x$uns$spatial[[image_id]]$images$ hires)/255

# find the scale factor
image_scale <- x$uns$spatial[[image_id]][["scalefactors"]][[paste("tissue_", "hires", "_scalef", sep="")]]

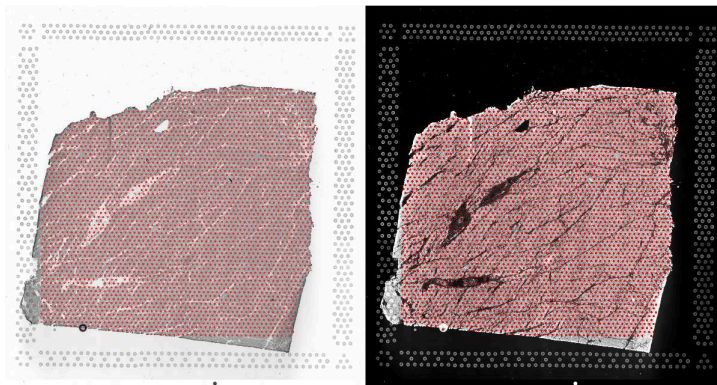
# Process Image:
# Color mode
colorMode(image_data) = "Color"

# plot the original image
display(image_data, method="raster", all=F)

```

1. Loading Spatial Dataset: Visium_17_CK295
2. Available sample IDs:
... processing sample: Visium_17_CK295
3. "evaluate_image" set to TRUE:
... Skipping spot matrix processing and gene expression writing
4. processing image(s).
... pulling out image data from anndata file
... applying selected transformations
5. Generating grey scale images and mask
... "evaluate_image" is TRUE, plotting images for evaluation

IMAGE EVALUATION: Please check the plots to ensure adequate overlap of images and spots, as well as any adjustments needed for image orientation. Once you are happy, re-run the function with the argument "evaluate_image" set to FALSE. This will then process the gene expression matrix, and write the data to the directory for the shiny app to handle.





```
In [43]: # run again but flag evaluation as F
preprocessImage(
  image_h5ad = x,
  rotation = 0,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = F,
  feature_names = x$var$feature_name,
  nCounts_column = "n_counts",
  nGenes_column = "n_genes",
  QC_threshold = 600,
  contrast_mod = 1
)

png(paste("data/spatial/", image_id, "/unprocessed.png", sep=""), height=2000, width=2000)
display(image_data, method="raster", all=F)
dev.off()
```

```

1. Loading Spatial Dataset: Visium_17_CK295
2. Available sample IDs:
... processing sample: Visium_17_CK295
3. "evaluate_image" set to FALSE:
... Transforming spot matrix
... Picking up the gene lookup table from: "data/gene_lookup_table.csv"
... Picking up the spatial subset of gene lookup table from: "data/spatial/gene_lookup_spatial.csv"
... Writing the genes into their files:
... Writing the genes into their files:
A/Z
... Writing the genes into their files:
B/Z
... Writing the genes into their files:
C/Z
... Writing the genes into their files:
D/Z
... Writing the genes into their files:
E/Z
... Writing the genes into their files:
F/Z
... Writing the genes into their files:
G/Z
... Writing the genes into their files:
H/Z
... Writing the genes into their files:
I/Z
... Writing the genes into their files:
J/Z
... Writing the genes into their files:
K/Z
... Writing the genes into their files:
L/Z
... Writing the genes into their files:
M/Z
... Writing the genes into their files:
N/Z
... Writing the genes into their files:
O/Z
... Writing the genes into their files:
P/Z
... Writing the genes into their files:
Q/Z
... Writing the genes into their files:
R/Z
... Writing the genes into their files:
S/Z
... Writing the genes into their files:
T/Z
... Writing the genes into their files:
U/Z
... Writing the genes into their files:
V/Z
... Writing the genes into their files:
W/Z
... Writing the genes into their files:
X/Z
... Writing the genes into their files:
Y/Z
... Writing the genes into their files:
Z/Z

4. processing image(s).
... pulling out image data from anndata file
... applying selected transformations
5. Generating grey scale images and mask
6. writing image 1 (light version): data/spatial/Visium_17_CK295/images/1/overlay.png
... writing image 2 (dark version): data/spatial/Visium_17_CK295/images/2/overlay.png
7. writing datapoint coordinates to: data/spatial/Visium_17_CK295/coordinates.csv

```

IMAGE EVALUATION: no image evaluation was carried out. If transformations are needed, please re-run the function with the argument "evaluate_image" set to TRUE (default). This will skip the gene matrix processing and plot the images, re-run iteratively using the "rotation" and "reflection" arguments until the desired orientation is achieved.

agg_record_2057391633:2

INFARCTED RZ Left Ventricle # 4 "Visium_6_CK284"

```

In [44]: # INFARCTED RZ Left Ventricle # 4 "Visium_6_CK284"
x = anndata::read_h5ad(datasets[13])
x$obs$sample = names(x$uns$spatial)[1]

preprocessImage(
  image_h5ad = x,
  rotation = 0,
  reflection = "none",
  opacity_filter = 0.9,
  sample_id = unique(x$obs$sample),
  evaluate_image = T,
  feature_names = x$var$feature_name,
  nCounts_column = "n_counts",
  nGenes_column = "n_genes",
  QC_threshold = 600,
  contrast_mod = 1
)

image_id = names(x$uns$spatial)[1]
image_data = Image(x$uns$spatial[[image_id]]$images$ hires)/255

```

```

# find the scale factor
image_scale <- x$uns$spatial[[image_id]][["scalefactors"]][paste("tissue_", "hires", "_scalef", sep="")]

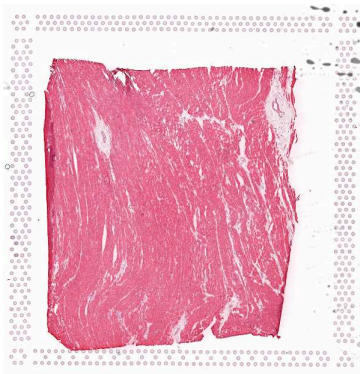
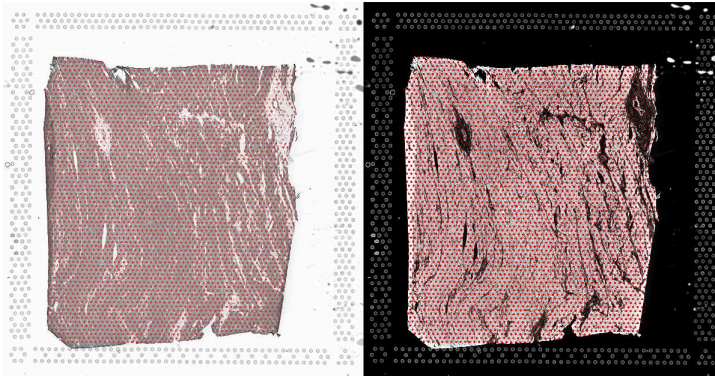
# Process Image:
# Color mode
colorMode(image_data) = "Color"

# plot the original image
display(image_data, method="raster", all=F)

```

1. Loading Spatial Dataset: Visium_6_CK284
2. Available sample IDs:
- ... processing sample: Visium_6_CK284
3. "evaluate_image" set to TRUE:
- ... Skipping spot matrix processing and gene expression writing
4. processing image(s).
- ... pulling out image data from anndata file
- ... applying selected transformations
5. Generating grey scale images and mask
- ... "evaluate_image" is TRUE, plotting images for evaluation

IMAGE EVALUATION: Please check the plots to ensure adequate overlap of images and spots, as well as any adjustments needed for image orientation. Once you are happy, re-run the function with the argument "evaluate_image" set to FALSE. This will then process the gene expression matrix, and write the data to the directory for the shiny app to handle.



```

In [45]: # run again but flag evaluation as F
preprocessImage(
  image_h5ad = x,

```

```

rotation = 0,
reflection = "none",
opacity_filter = 0.9,
sample_id = unique(x$obs$sample),
evaluate_image = F,
feature_names = x$var$feature_name,
nCounts_column = "n_counts",
nGenes_column = "n_genes",
QC_threshold = 600,
contrast_mod = 1
)

png(paste("data/spatial/", image_id, "/unprocessed.png", sep=""), height=2000, width=2000)
display(image_data, method="raster", all=F)
dev.off()

```

```

1. Loading Spatial Dataset: Visium_6_CK284
2. Available sample IDs:
... processing sample: Visium_6_CK284
3. "evaluate_image" set to FALSE:
... Transforming spot matrix
... Picking up the gene lookup table from: "data/gene_lookup_table.csv"
... Picking up the spatial subset of gene lookup table from: "data/spatial/gene_lookup_spatial.csv"
... Writing the genes into their files:
... Writing the genes into their files:
A/Z
... Writing the genes into their files:
B/Z
... Writing the genes into their files:
C/Z
... Writing the genes into their files:
D/Z
... Writing the genes into their files:
E/Z
... Writing the genes into their files:
F/Z
... Writing the genes into their files:
G/Z
... Writing the genes into their files:
H/Z
... Writing the genes into their files:
I/Z
... Writing the genes into their files:
J/Z
... Writing the genes into their files:
K/Z
... Writing the genes into their files:
L/Z
... Writing the genes into their files:
M/Z
... Writing the genes into their files:
N/Z
... Writing the genes into their files:
O/Z
... Writing the genes into their files:
P/Z
... Writing the genes into their files:
Q/Z
... Writing the genes into their files:
R/Z
... Writing the genes into their files:
S/Z
... Writing the genes into their files:
T/Z
... Writing the genes into their files:
U/Z
... Writing the genes into their files:
V/Z
... Writing the genes into their files:
W/Z
... Writing the genes into their files:
X/Z
... Writing the genes into their files:
Y/Z
... Writing the genes into their files:
Z/Z

4. processing image(s).
... pulling out image data from anndata file
... applying selected transformations
5. Generating grey scale images and mask
6. writing image 1 (light version): data/spatial/Visium_6_CK284/images/1/overlay.png
... writing image 2 (dark version): data/spatial/Visium_6_CK284/images/2/overlay.png
7. writing datapoint coordinates to: data/spatial/Visium_6_CK284/coordinates.csv

IMAGE EVALUATION: no image evaluation was carried out. If transformations are needed, please re-run the function with the argument
"evaluate_image" set to TRUE (default). This will skip the gene matrix processing and plot the images, re-run iteratively using the
"rotation" and "reflection" arguments until the desired orientation is achieved.

```

agg_record_1901438561:2

In [46]: `gc()`

	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	2187749	116.9	4239520	226.5	3684547	196.8
Vcells	15415471	117.7	386473412	2948.6	545479862	4161.7

Again, as these functions were part of an ongoing shiny app code base, I just pull up the functions that I was working on to access the pre-processed data, and plot the values over the images.

```
gc()
```

	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	2162270	115.5	4269140	228	2907270	155.3
Vcells	406477694	3101.2	614065586	4685	425114087	3243.4

```
In [ ]: plot.cols[[1]][[1]] = colorRampPalette(c("black", "red"))(6)
plot.cols[[2]][[1]] = colorRampPalette(c("white", "red"))(6)
plot.cols[[1]][[2]] = colorRampPalette(c(alt.cols))(6)
plot.cols[[2]][[2]] = colorRampPalette(c(alt.cols))(6)
```

```
In [64]: options(repr.plot.width = 10)
options(repr.plot.height = 10)
gc()
#par(mfrow=c(2,2))
gene.r = "EDN1"
for(i in 1:13){
  samples2plot = sample_id_list[c(i)]
  pdf(paste("figures/panels/results/Kuppe2022/", gene.r, "_", dataset_names[i], "R.pdf", sep=""), height=10, width=10)
  plotVisium1(
    rgb.samples_list = VisiumData(gene.r, sample_id_list = samples2plot),
    light_or_dark = input$light_or_dark,
    plot_size = input$plot_size,
    mask_overlay = input$mask_overlay,
    min.col = input$min.col,
    opacity_count_dependent = sum(input$opacity_count_dependent),
    opacity = input$opacity,
    dot_or_not = input$dot_or_not,
    sample_id_list = samples2plot
  )

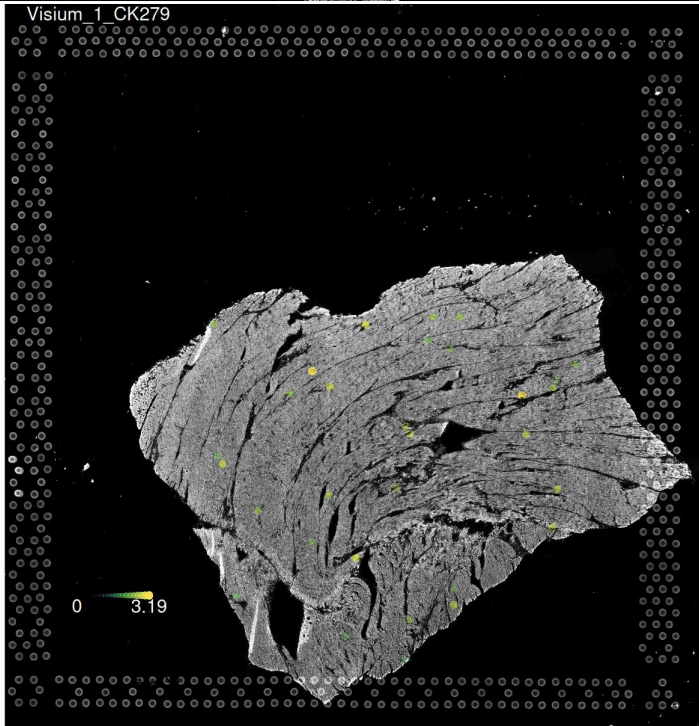
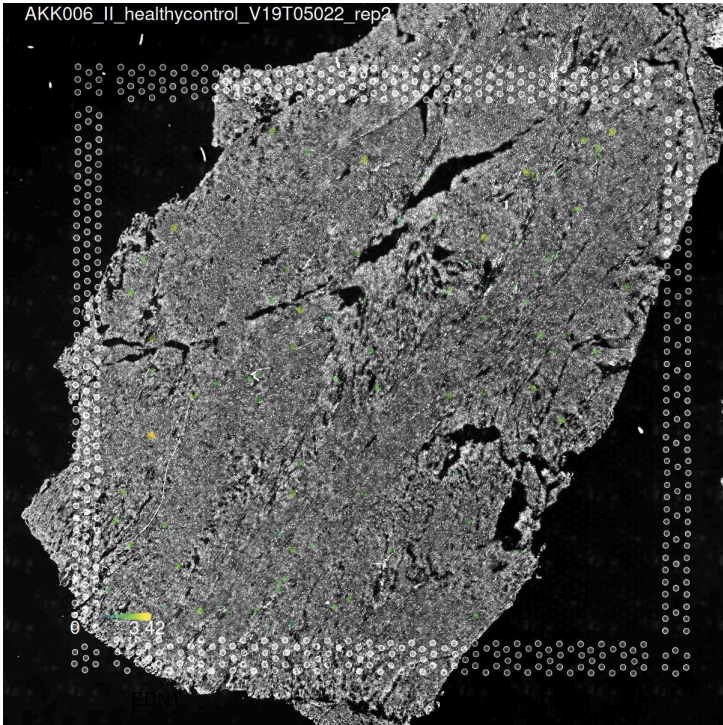
  dev.off()
  plotVisium1(
    rgb.samples_list = VisiumData(gene.r, sample_id_list = samples2plot),
    light_or_dark = input$light_or_dark,
    plot_size = input$plot_size,
    mask_overlay = input$mask_overlay,
    min.col = input$min.col,
    opacity_count_dependent = sum(input$opacity_count_dependent),
    opacity = input$opacity,
```

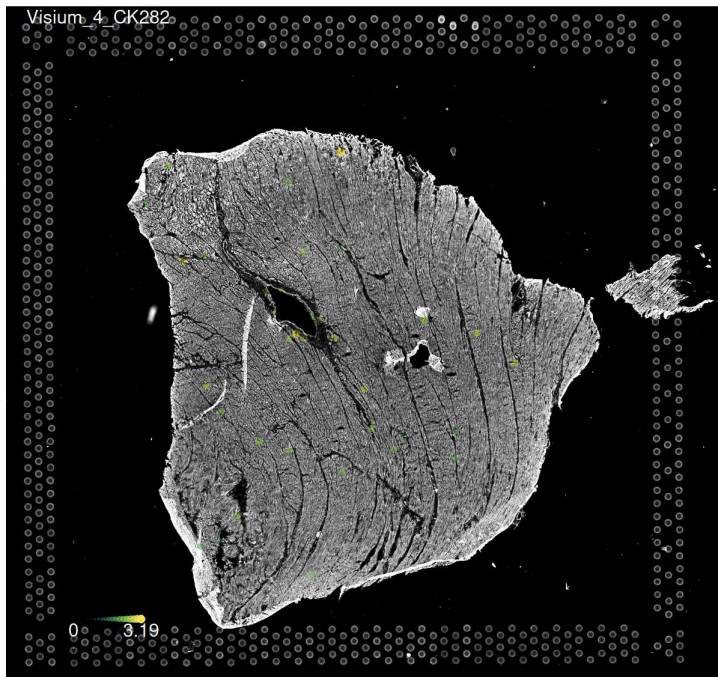
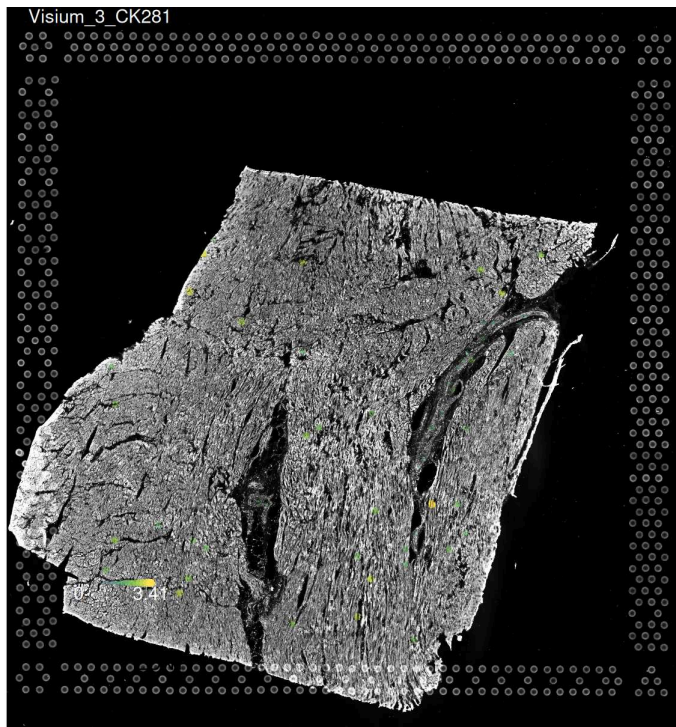


```
dot_or_not = input$dot_or_not,
sample_id_list = samples2plot
)
}
```

A matrix: 2 × 6 of type dbl

	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	2247957	120.1	4239520	226.5	4066359	217.2
Vcells	418208941	3190.7	1027113686	7836.3	1027113686	7836.3





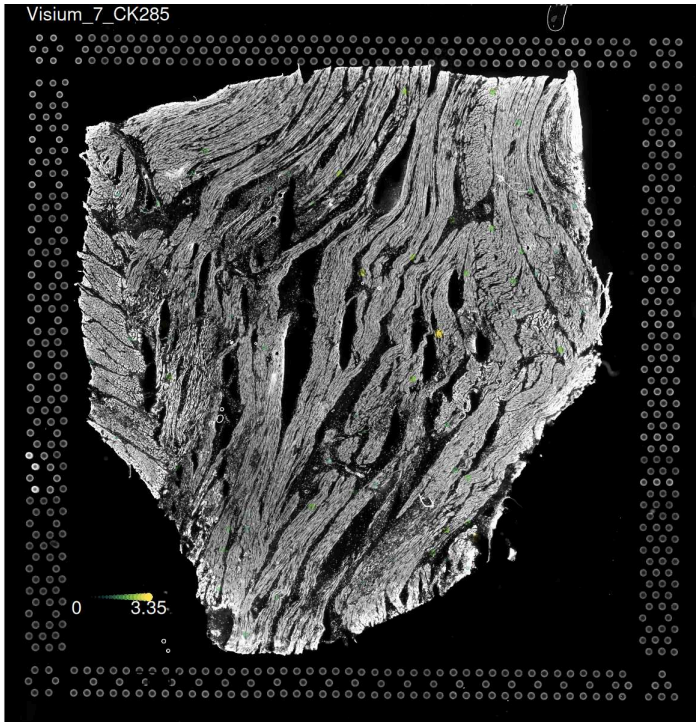
F0N1

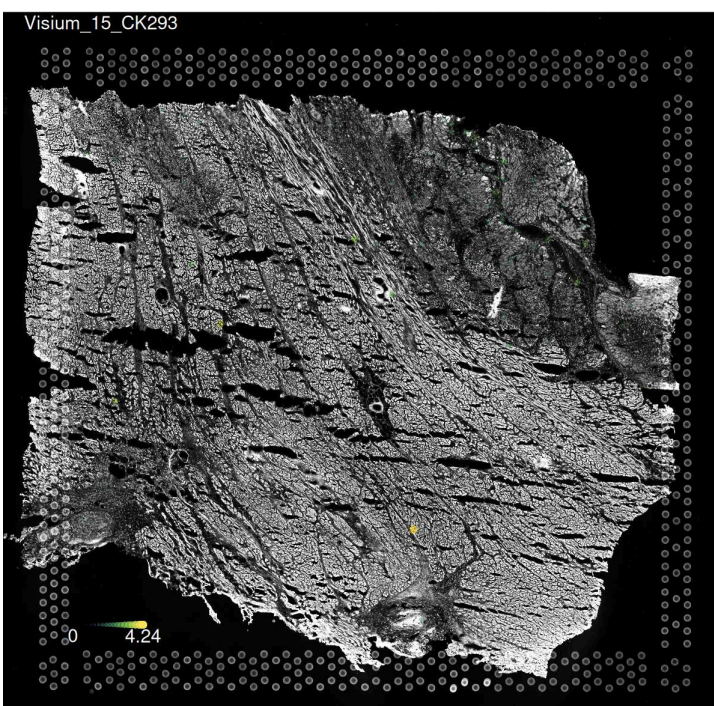
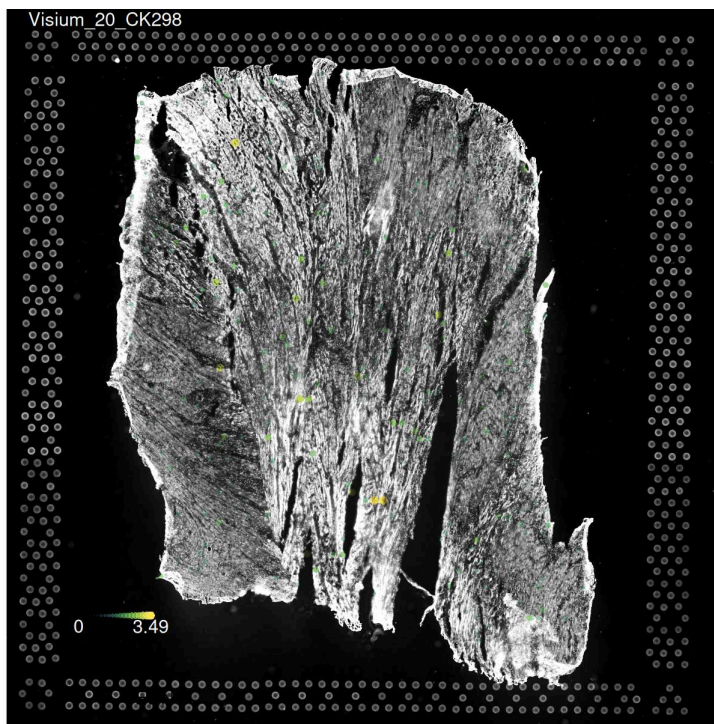
Visium_9_CK287



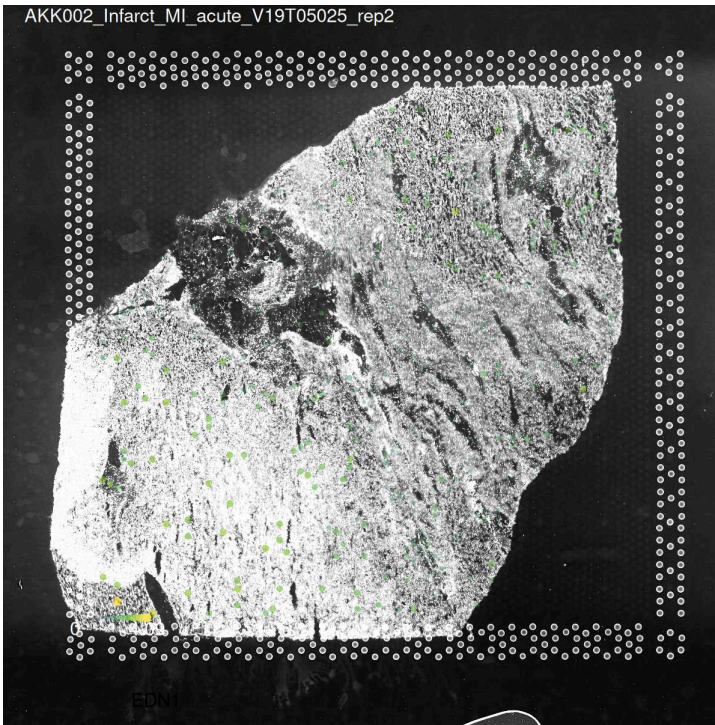
EDN1

Visium_7_CK285

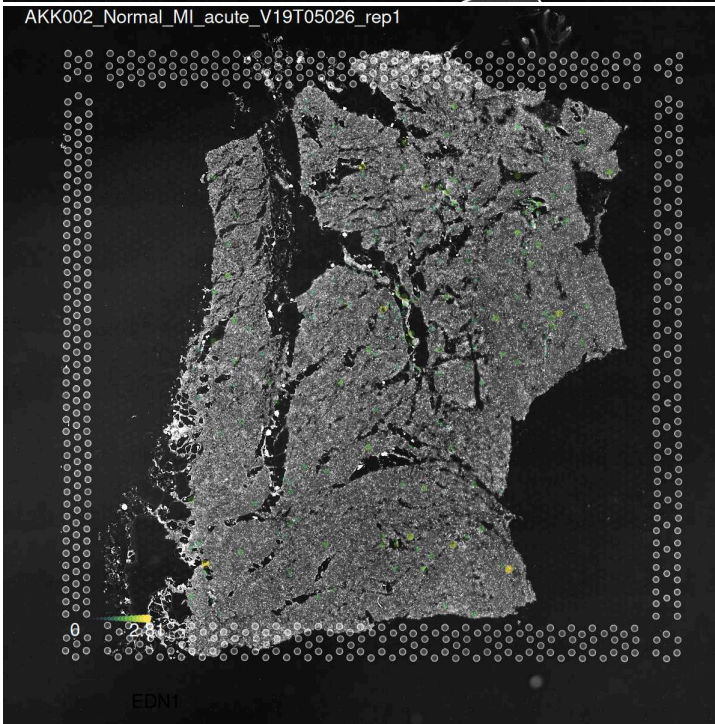




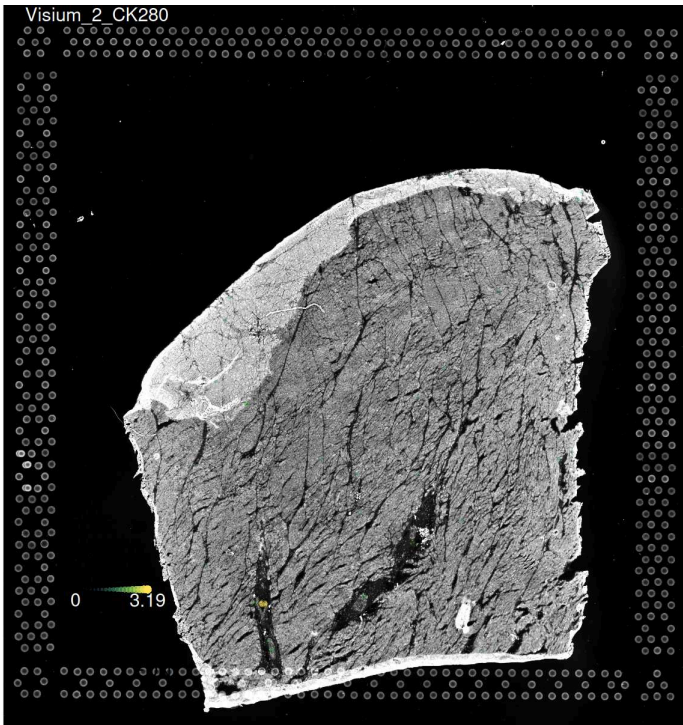
AKK002_Infarct_MI_acute_V19T05025_rep2



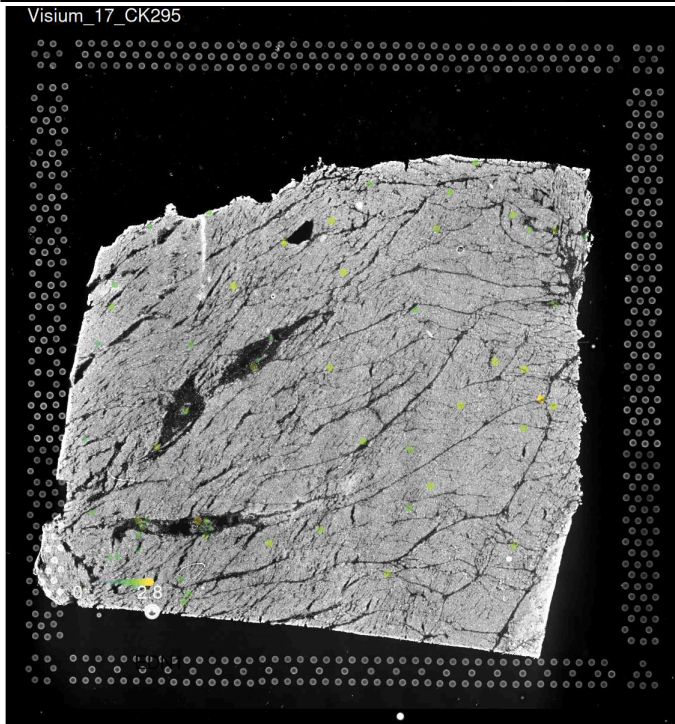
AKK002_Normal_MI_acute_V19T05026_rep1

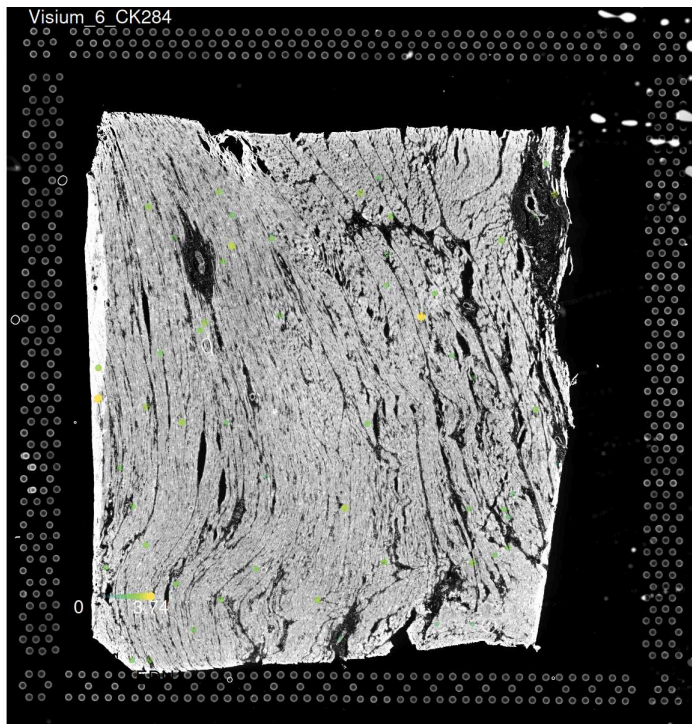


Visium_2_CK280



Visium_17_CK295



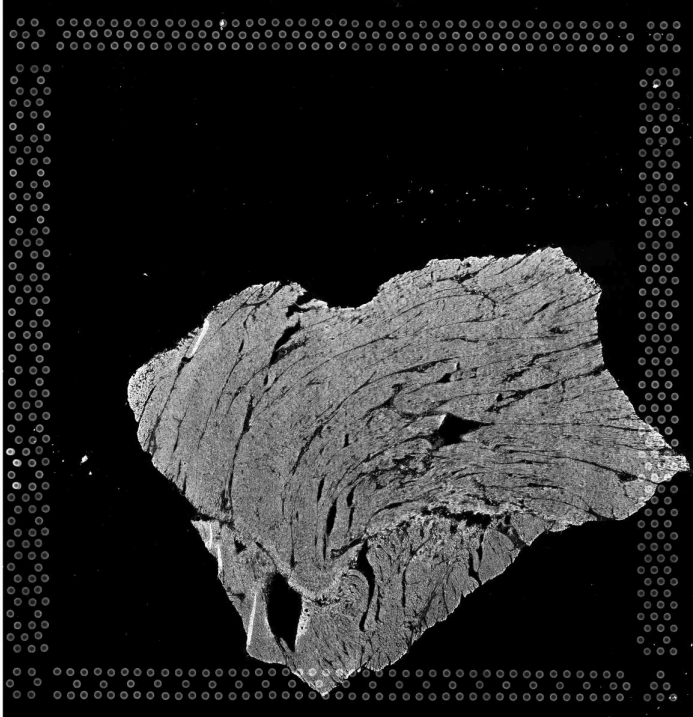
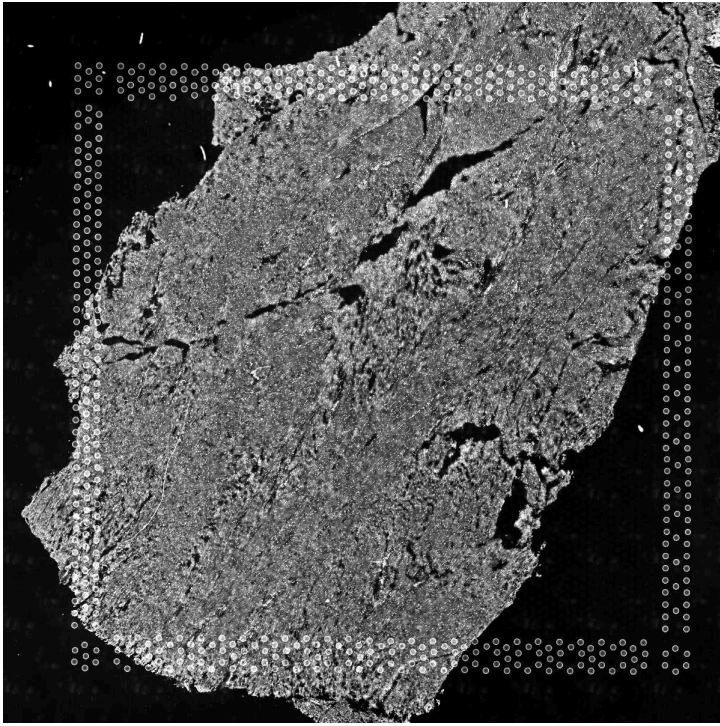


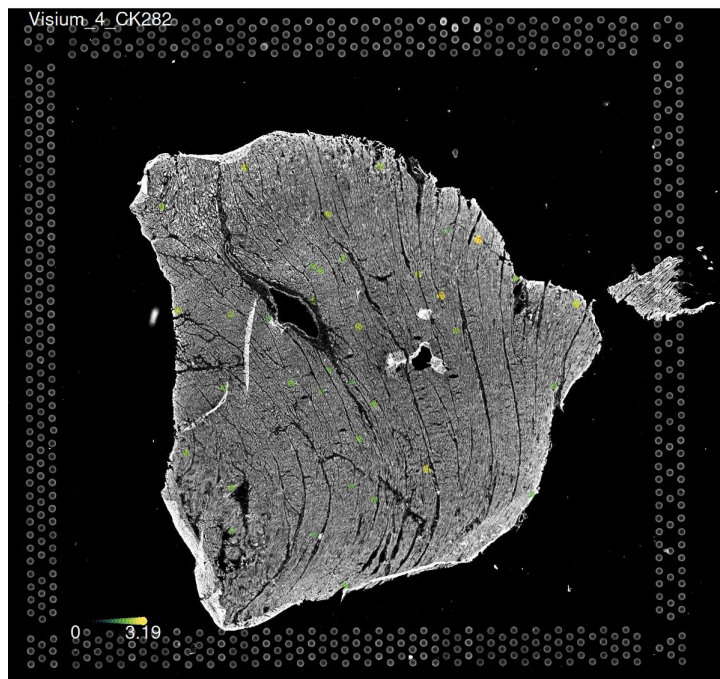
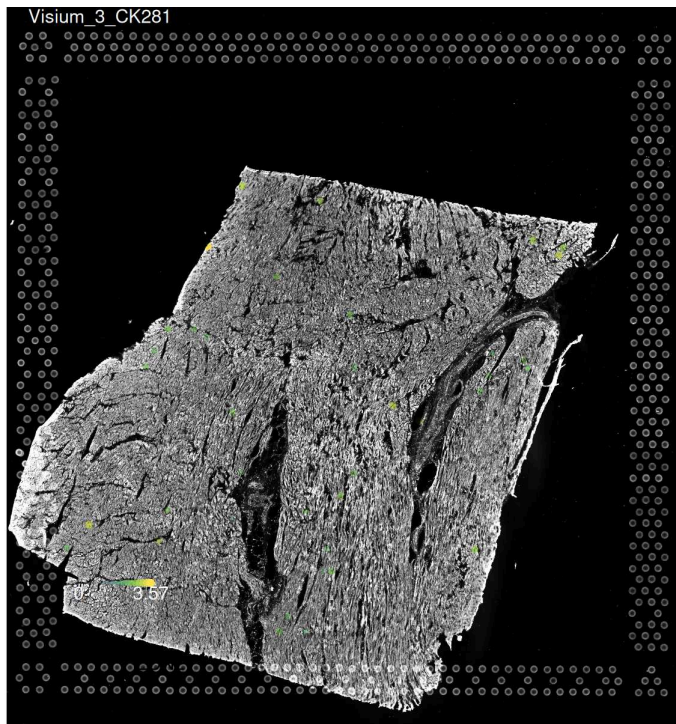
EDN2 only - all

```
In [65]: options(repr.plot.width = 10)
options(repr.plot.height = 10)
gc()
#par(mfrow=c(2,2))
gene.r = "EDN2"
for(i in 1:13){
  samples2plot = sample_id_list[c(i)]
  pdf(paste("figures/panels/results/Kuppe2022/", gene.r, "_", dataset_names[i], "R.pdf", sep=""), height=10, width=10)
  plotVisium1(
    rgb.samples_list = VisiumData1(gene.r, sample_id_list = samples2plot),
    light_or_dark = input$light_or_dark,
    plot_size = input$plot_size,
    mask_overlay = input$mask_overlay,
    min.col = input$min.col,
    opacity_count_dependent = sum(input$opacity_count_dependent),
    opacity = input$opacity,
    dot_or_not = input$dot_or_not,
    sample_id_list = samples2plot
  )
  dev.off()
  plotVisium1(
    rgb.samples_list = VisiumData1(gene.r, sample_id_list = samples2plot),
    light_or_dark = input$light_or_dark,
    plot_size = input$plot_size,
    mask_overlay = input$mask_overlay,
    min.col = input$min.col,
    opacity_count_dependent = sum(input$opacity_count_dependent),
    opacity = input$opacity,
    dot_or_not = input$dot_or_not,
    sample_id_list = samples2plot
  )
}
```

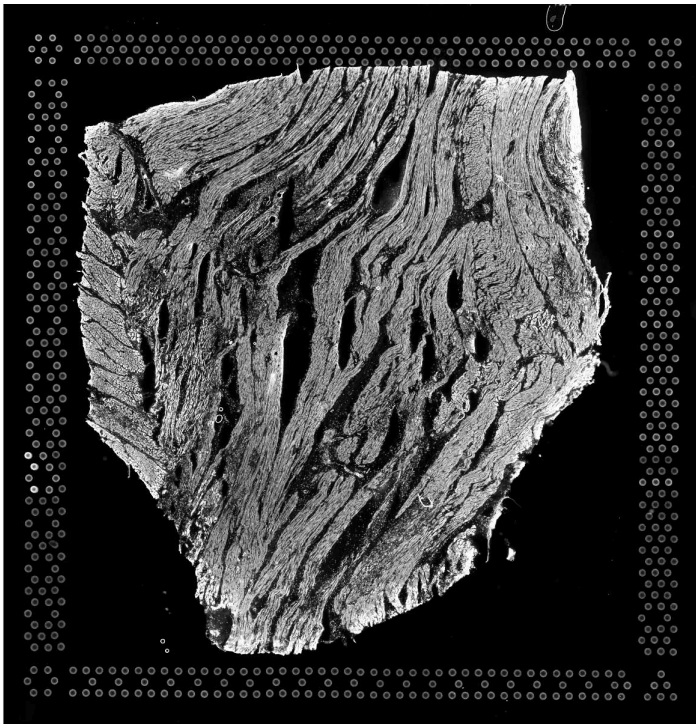
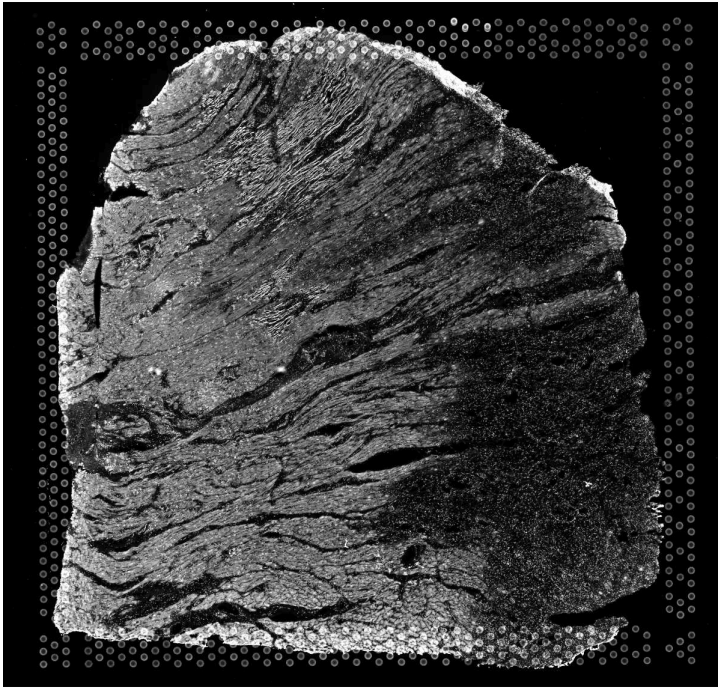
A matrix: 2 x 6 of type dbl

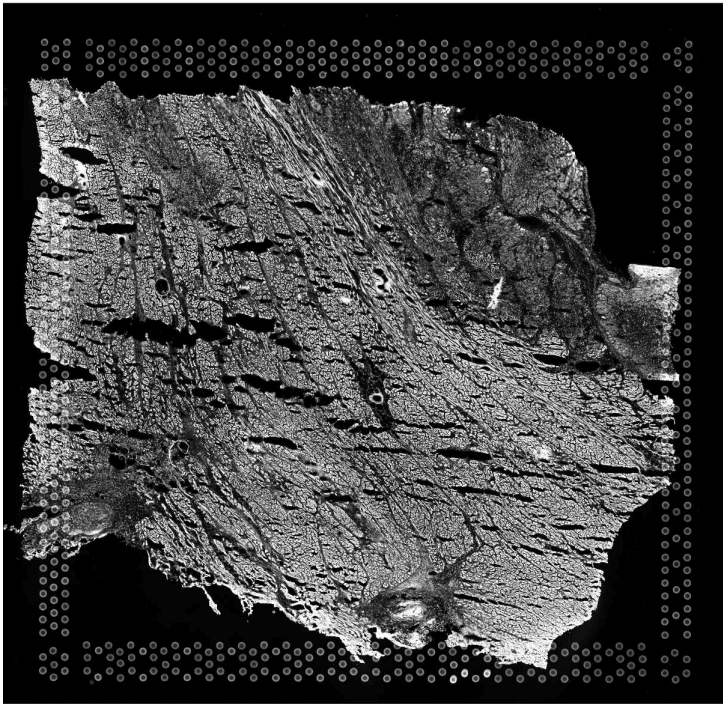
	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	2247968	120.1	4239520	226.5	4066359	217.2
Vcells	418208952	3190.7	1030865598	7864.9	1030865598	7864.9

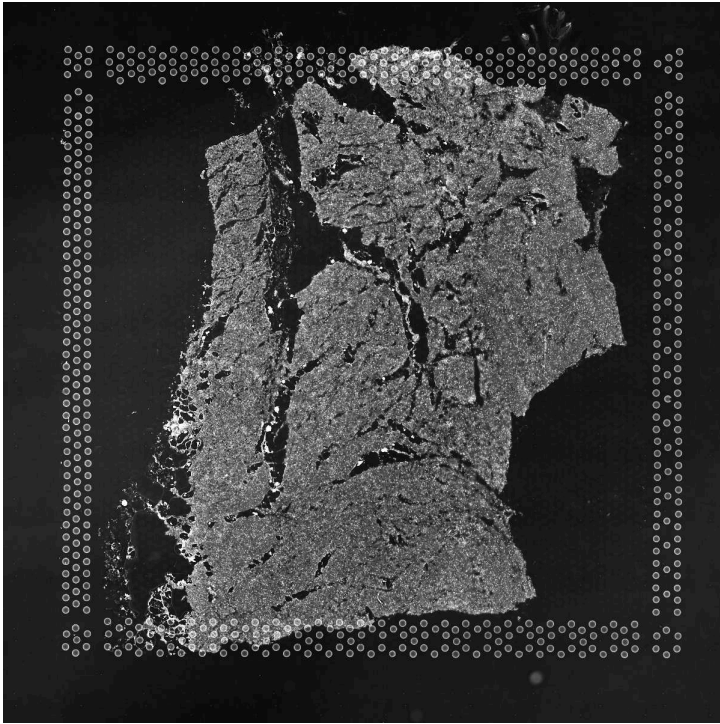


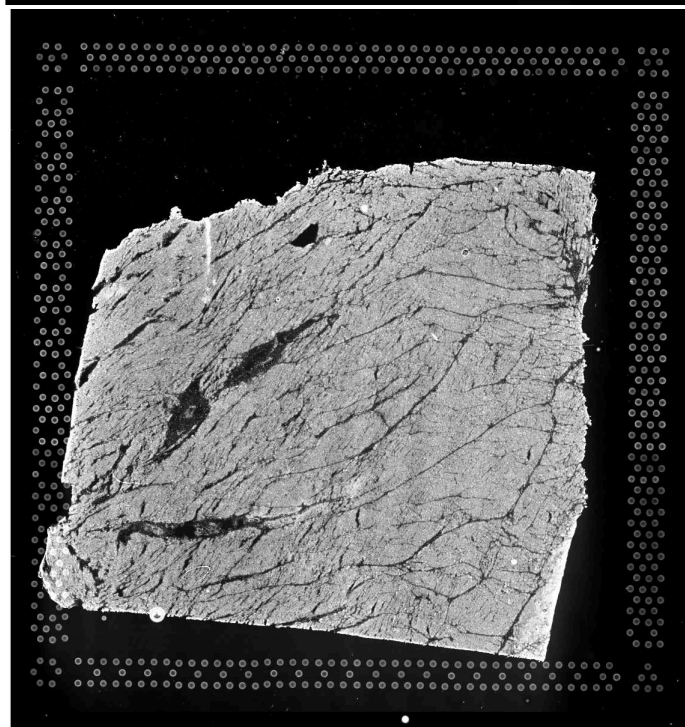
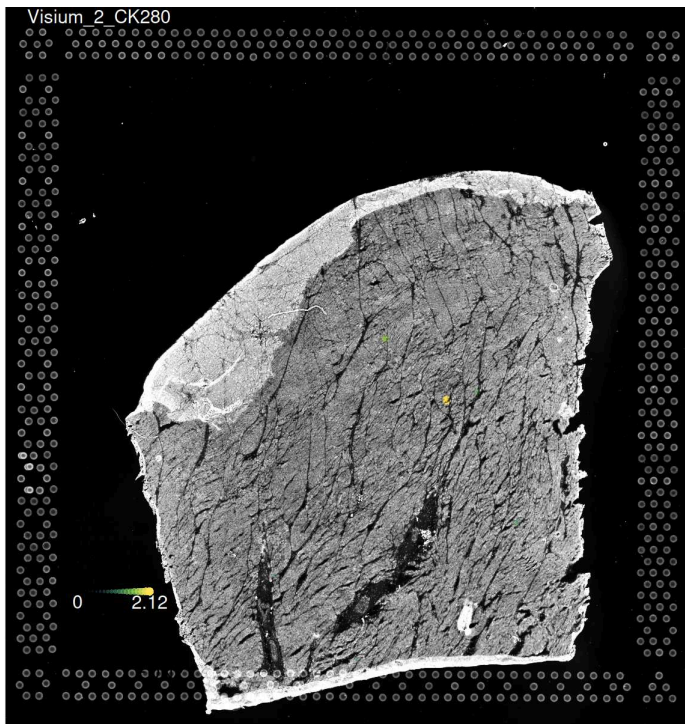


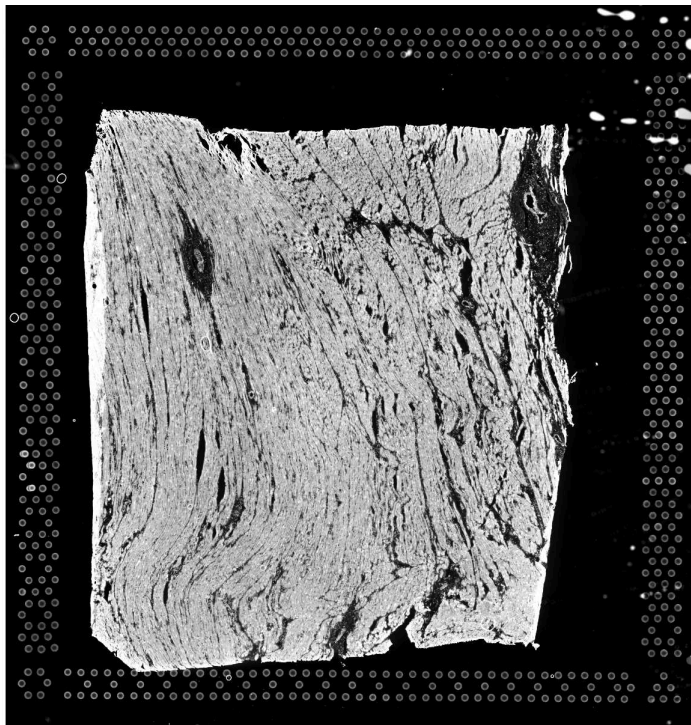
FN22









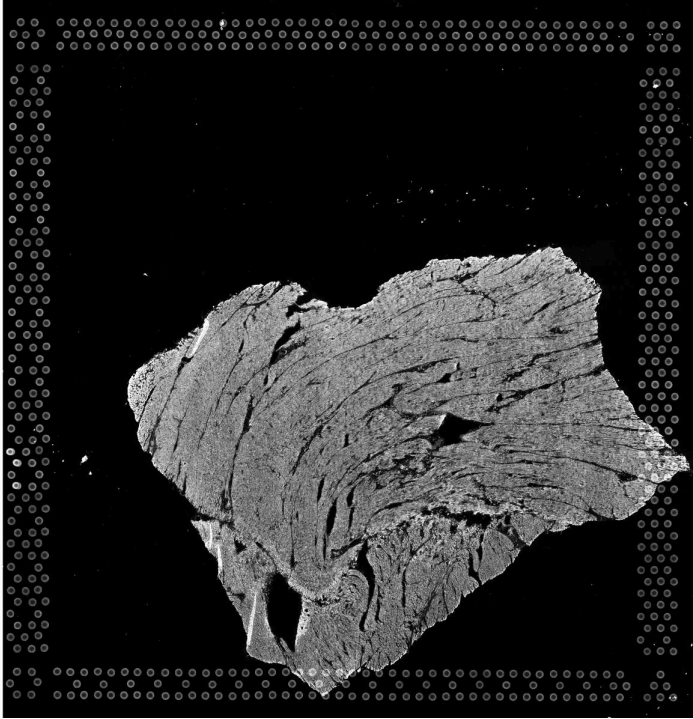
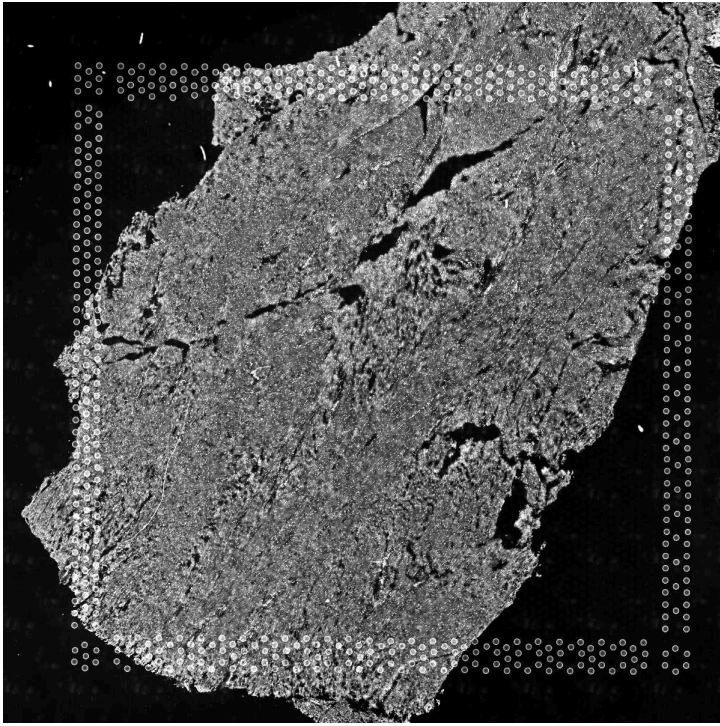


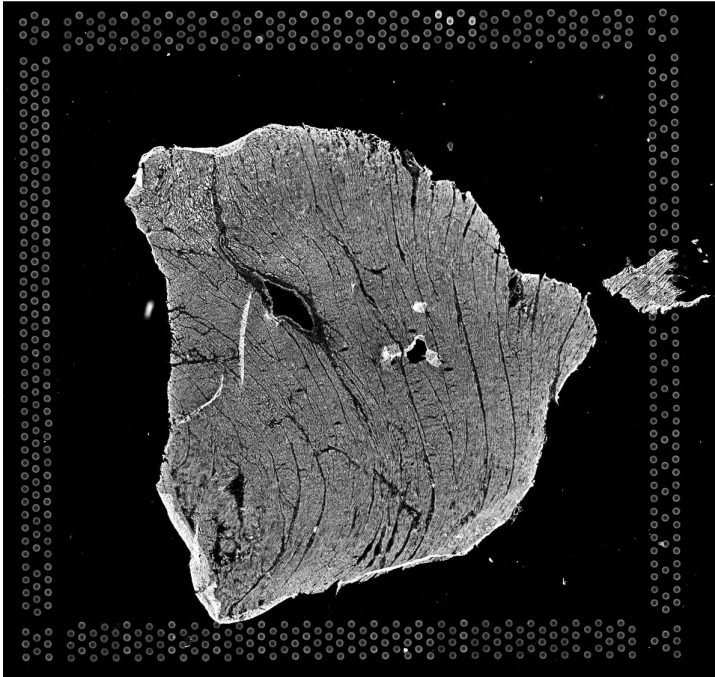
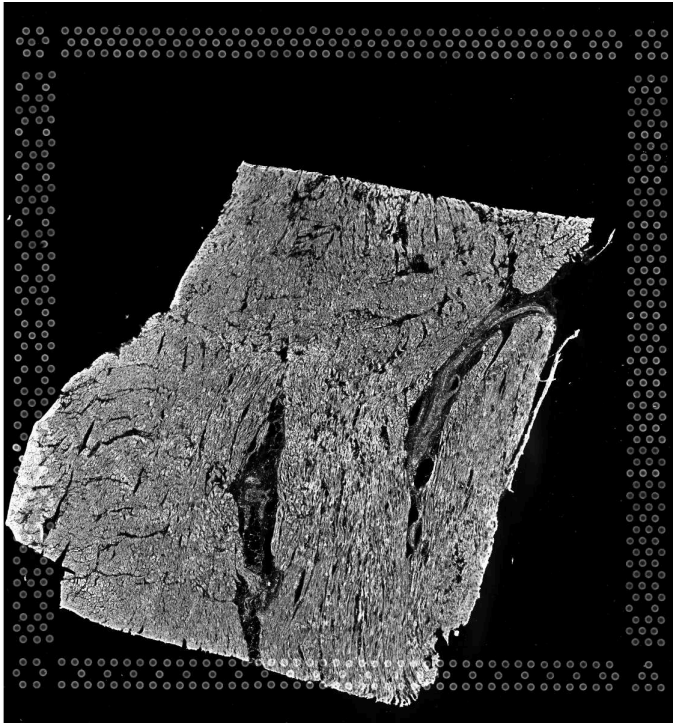
EDN3 only - all

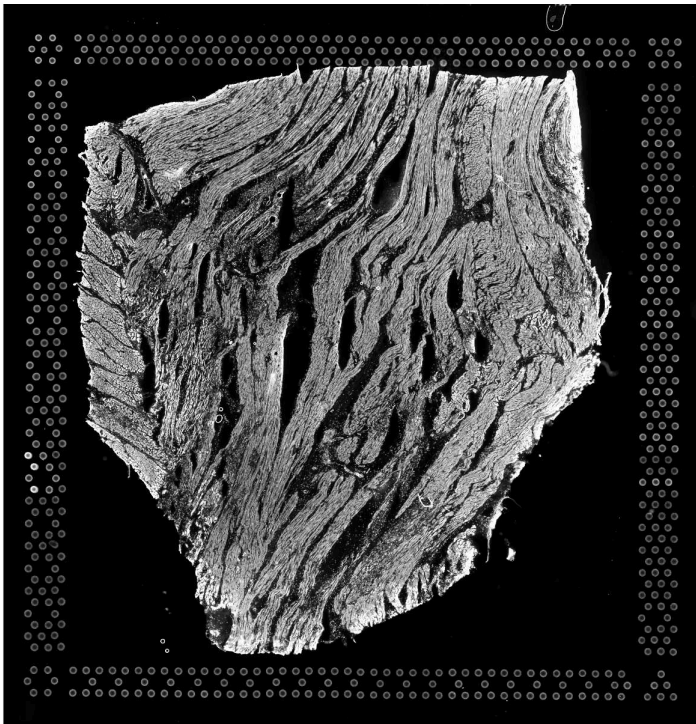
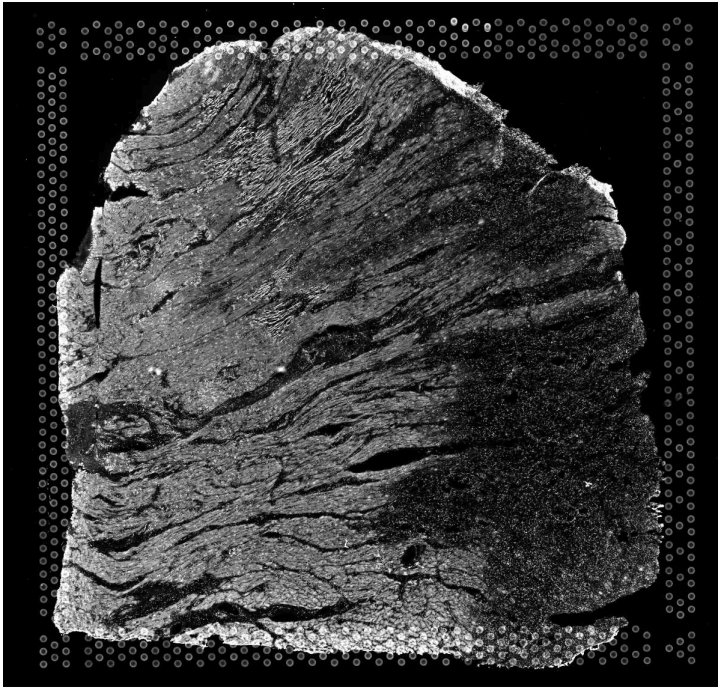
```
In [66]: options(repr.plot.width = 10)
options(repr.plot.height = 10)
gc()
#par(mfrow=c(2,2))
gene.r = "EDN3"
for(i in 1:13){
  samples2plot = sample_id_list[c(i)]
  pdf(paste("figures/panels/results/Kuppe2022/", gene.r, "_", dataset_names[i], "R.pdf", sep=""), height=10, width=10)
  plotVisium1(
    rgb.samples_list = VisiumData1(gene.r, sample_id_list = samples2plot),
    light_or_dark = input$light_or_dark,
    plot_size = input$plot_size,
    mask_overlay = input$mask_overlay,
    min.col = input$min.col,
    opacity_count_dependent = sum(input$opacity_count_dependent),
    opacity = input$opacity,
    dot_or_not = input$dot_or_not,
    sample_id_list = samples2plot
  )
  dev.off()
  plotVisium1(
    rgb.samples_list = VisiumData1(gene.r, sample_id_list = samples2plot),
    light_or_dark = input$light_or_dark,
    plot_size = input$plot_size,
    mask_overlay = input$mask_overlay,
    min.col = input$min.col,
    opacity_count_dependent = sum(input$opacity_count_dependent),
    opacity = input$opacity,
    dot_or_not = input$dot_or_not,
    sample_id_list = samples2plot
  )
}
```

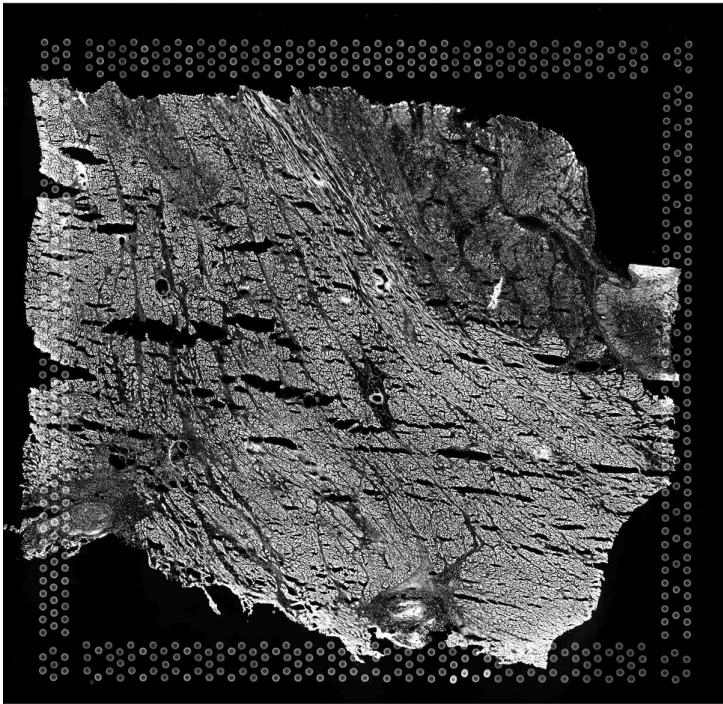
A matrix: 2 x 6 of type dbl

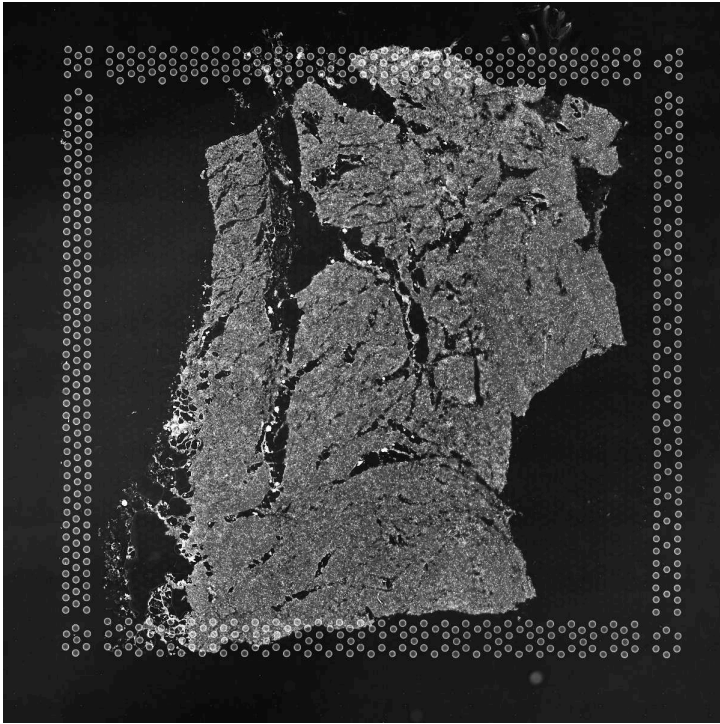
	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	2247965	120.1	4239520	226.5	4066359	217.2
Vcells	418208947	3190.7	1046510150	7984.3	1046510150	7984.3

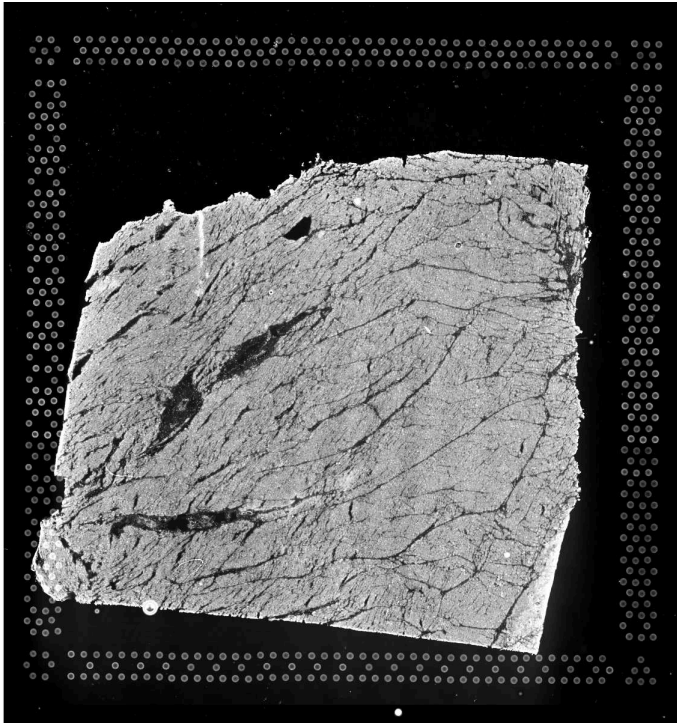
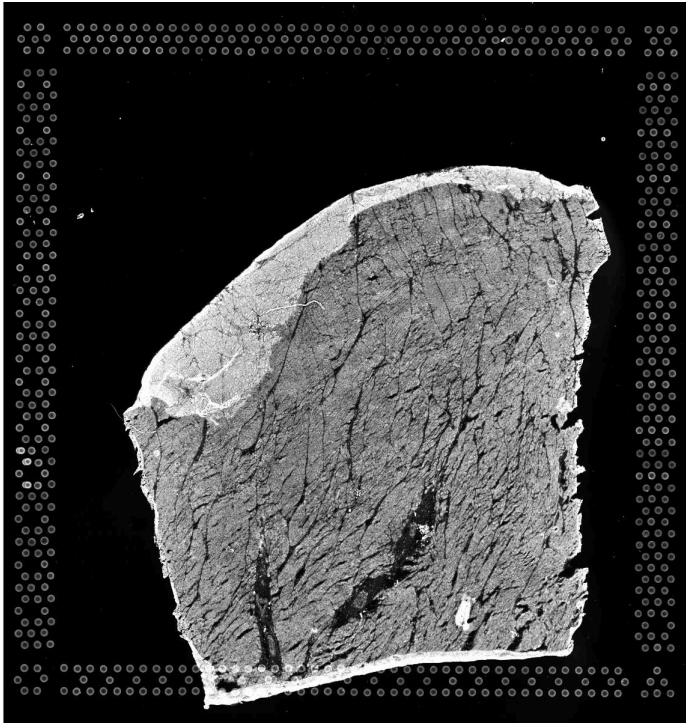


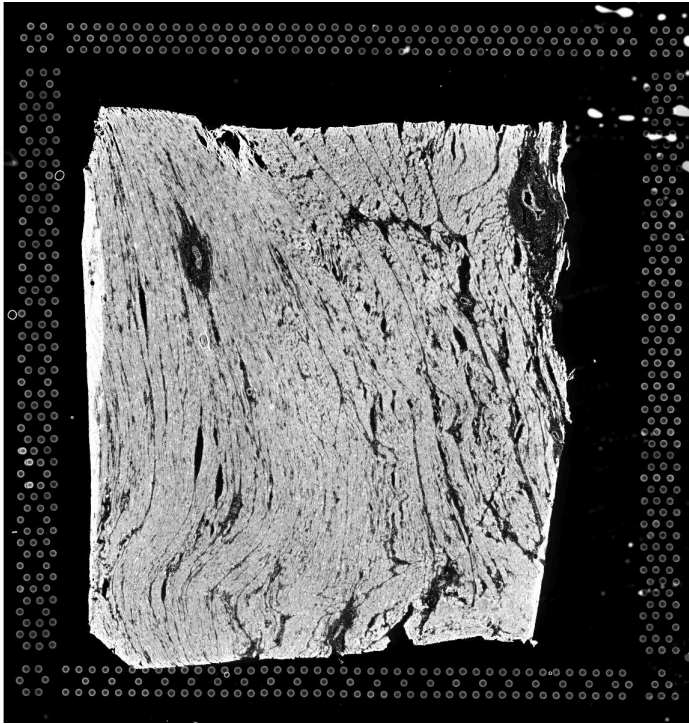










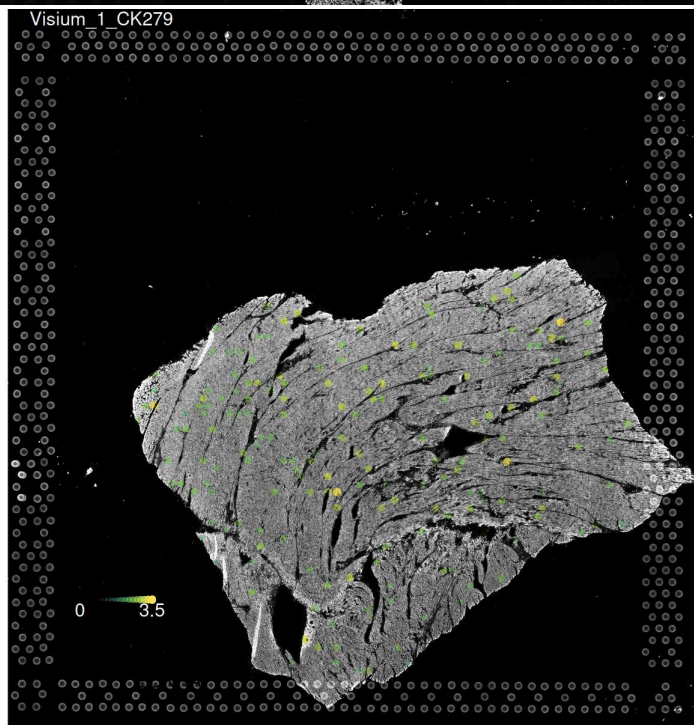
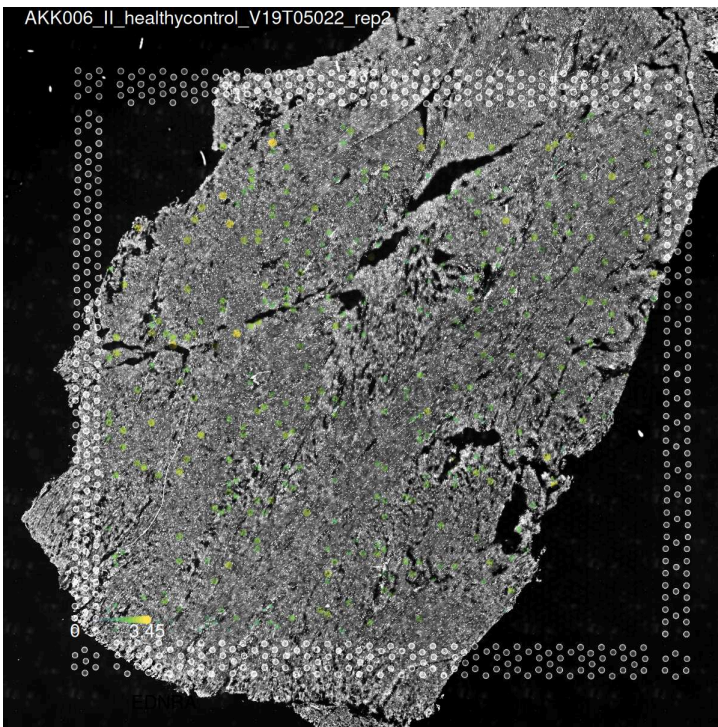


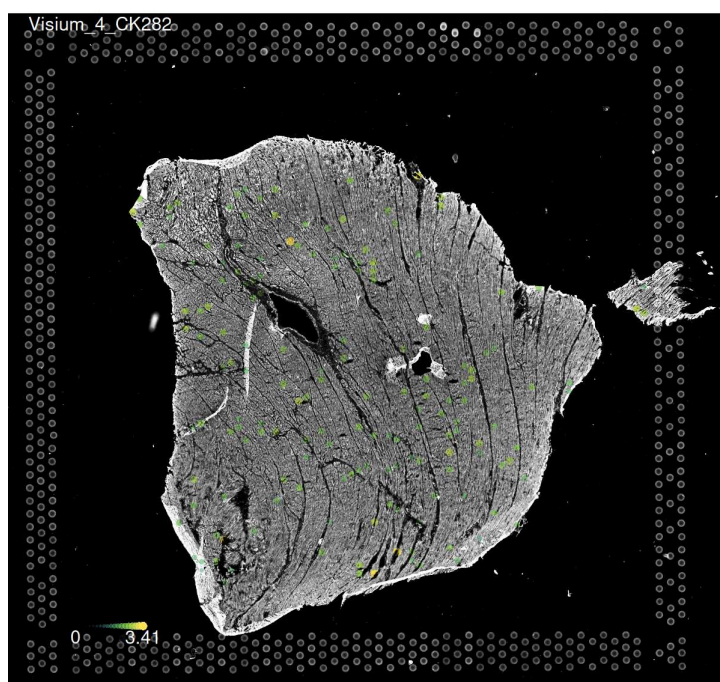
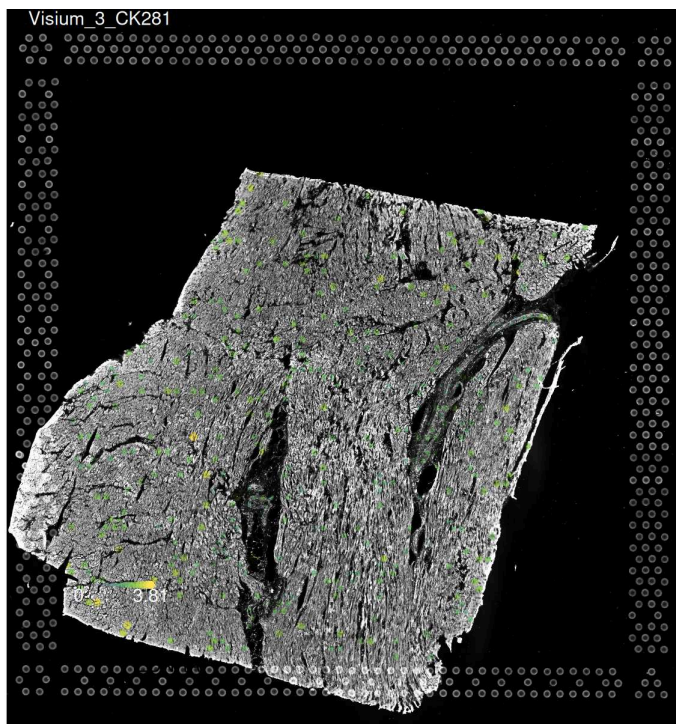
EDNRA only - all

```
In [12]: options(repr.plot.width = 10)
options(repr.plot.height = 10)
gc()
#par(mfrow=c(2,2))
gene.r = "EDNRA"
for(i in 1:13){
  samples2plot = sample_id_list[c(i)]
  pdf(paste("figures/panels/results/Kuppe2022/", gene.r, "_", dataset_names[i], "R.pdf", sep=""), height=10, width=10)
  plotVisium1(
    rgb.samples_list = VisiumData1(gene.r, sample_id_list = samples2plot),
    light_or_dark = input$light_or_dark,
    plot_size = input$plot_size,
    mask_overlay = input$mask_overlay,
    min.col = input$min.col,
    opacity_count_dependent = sum(input$opacity_count_dependent),
    opacity = input$opacity,
    dot_or_not = input$dot_or_not,
    sample_id_list = samples2plot
  )
  dev.off()
  plotVisium1(
    rgb.samples_list = VisiumData1(gene.r, sample_id_list = samples2plot),
    light_or_dark = input$light_or_dark,
    plot_size = input$plot_size,
    mask_overlay = input$mask_overlay,
    min.col = input$min.col,
    opacity_count_dependent = sum(input$opacity_count_dependent),
    opacity = input$opacity,
    dot_or_not = input$dot_or_not,
    sample_id_list = samples2plot
  )
}
```

A matrix: 2 × 6 of type dbl

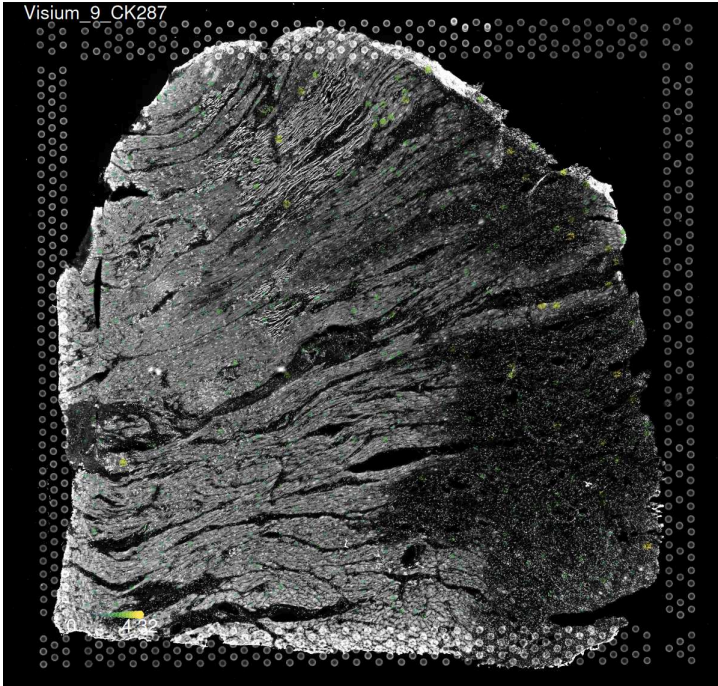
	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	2167256	115.8	4263800	227.8	2894821	154.7
Vcells	406490182	3101.3	614038292	4684.8	425104946	3243.3





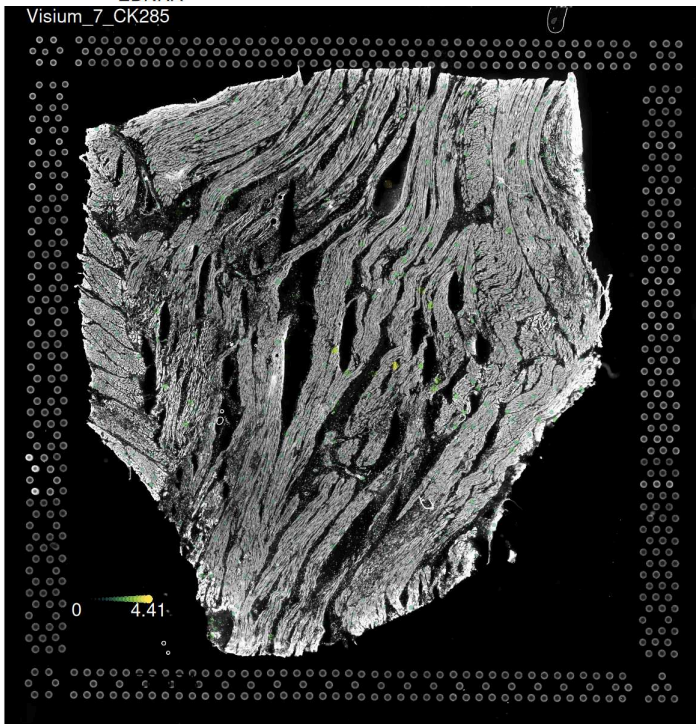
F0NRA

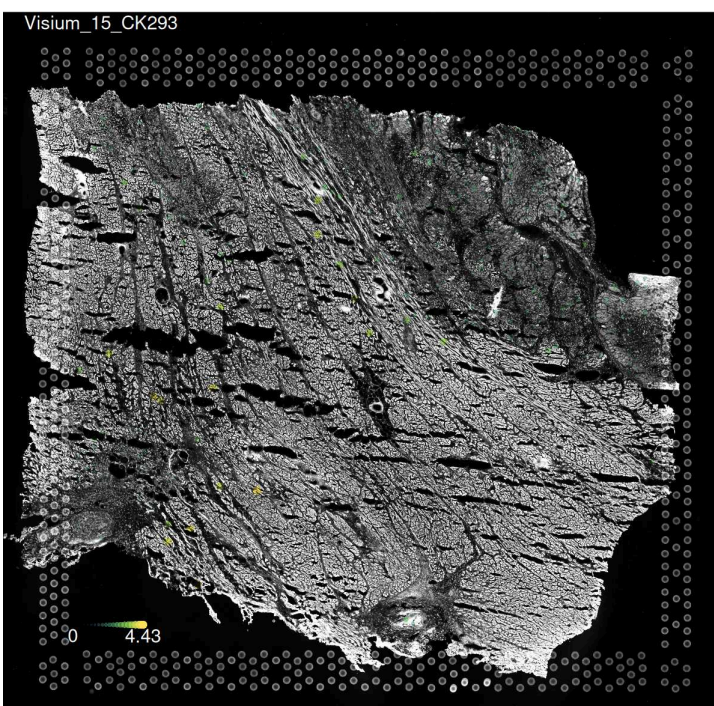
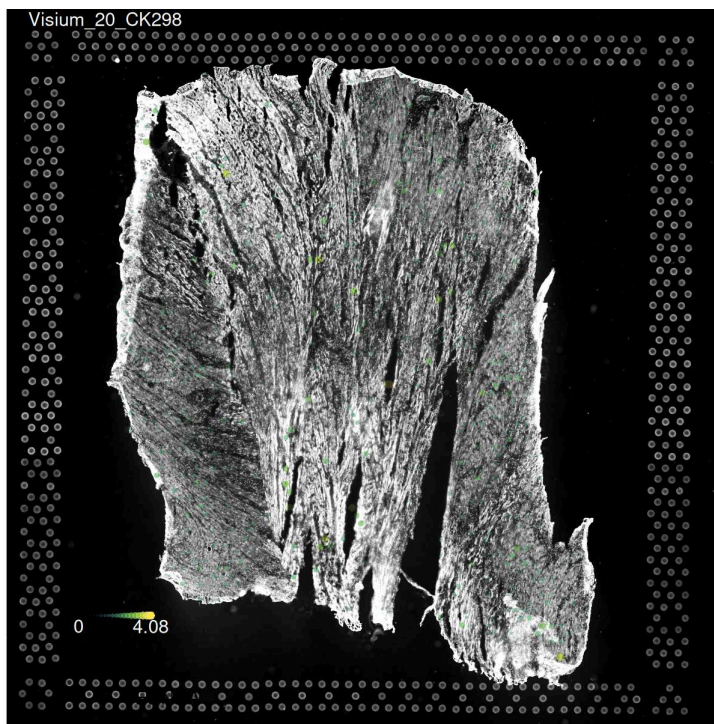
Visium_9_CK287



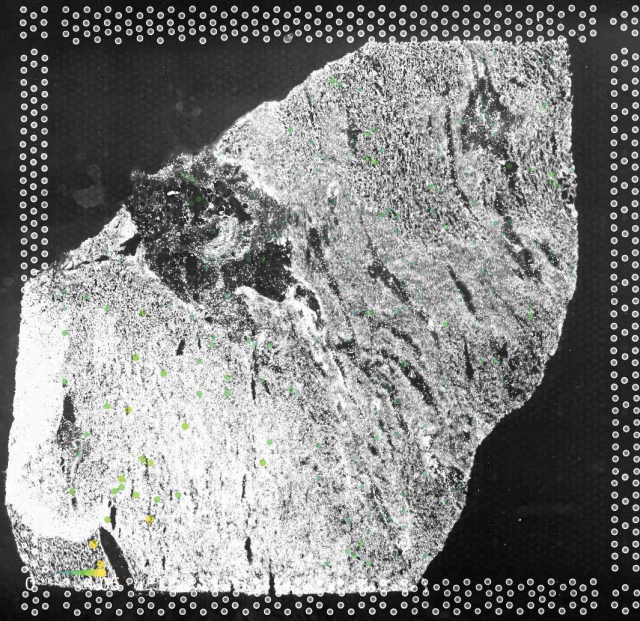
EDNRA

Visium_7_CK285



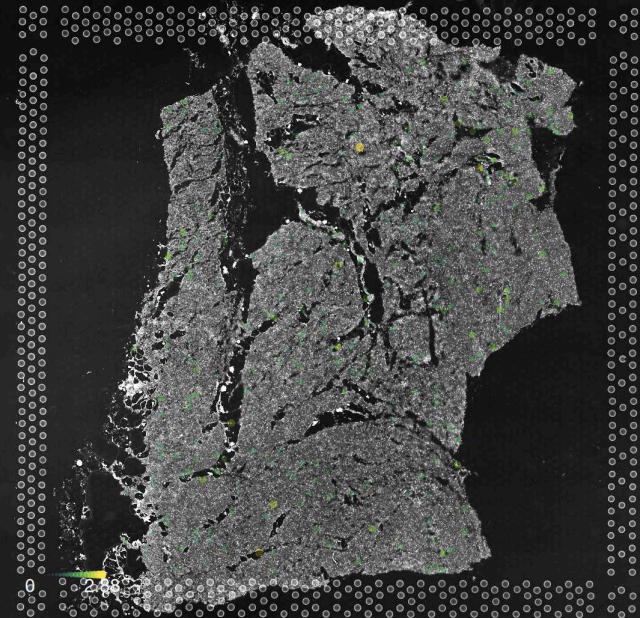


AKK002_Infarct_MI_acute_V19T05025_rep2



EDNRA

AKK002_Normal_MI_acute_V19T05026_rep1

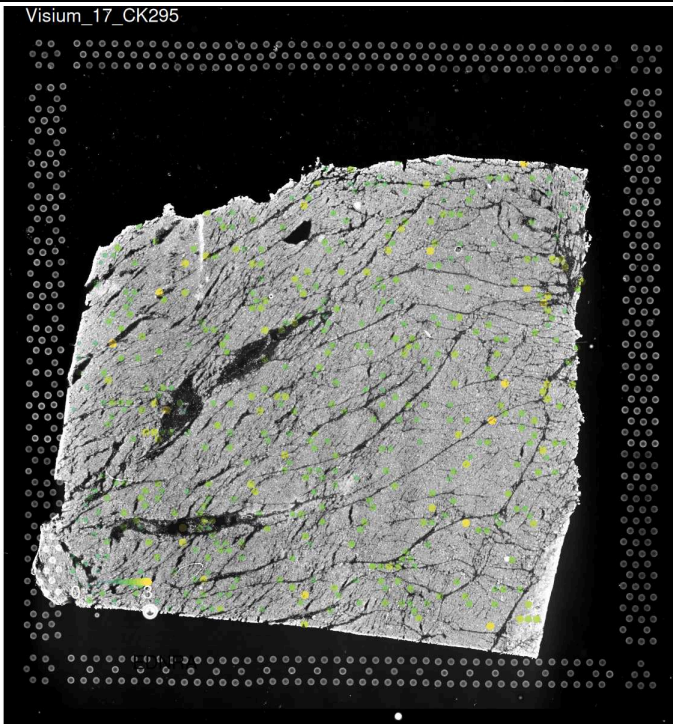


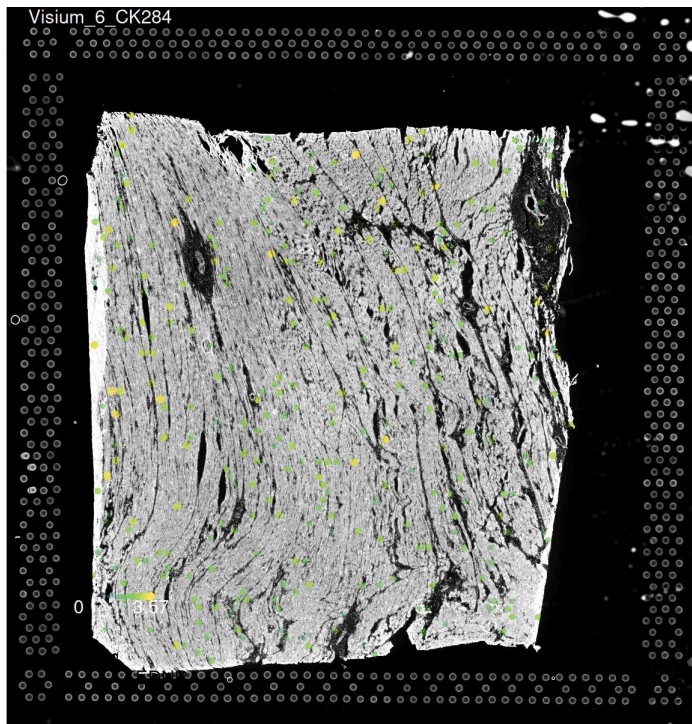
EDNRA

Visium_2_CK280



Visium_17_CK295



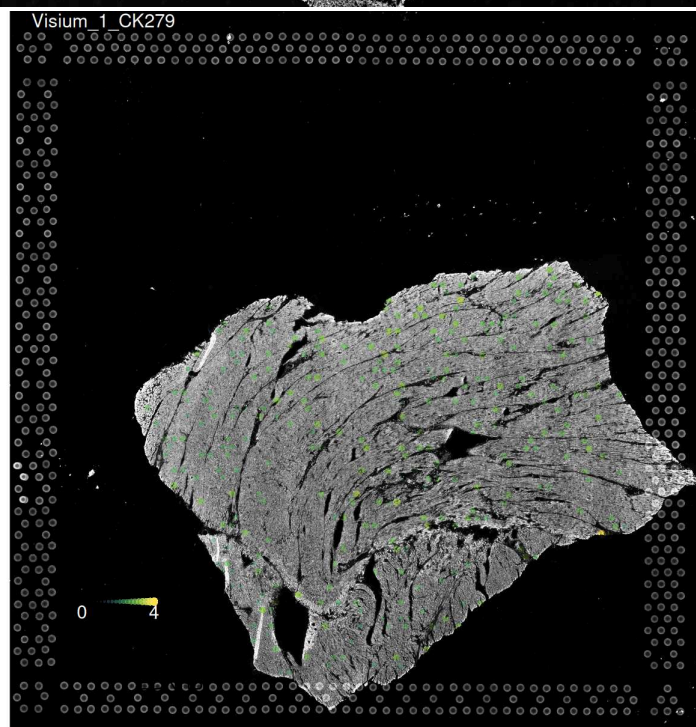
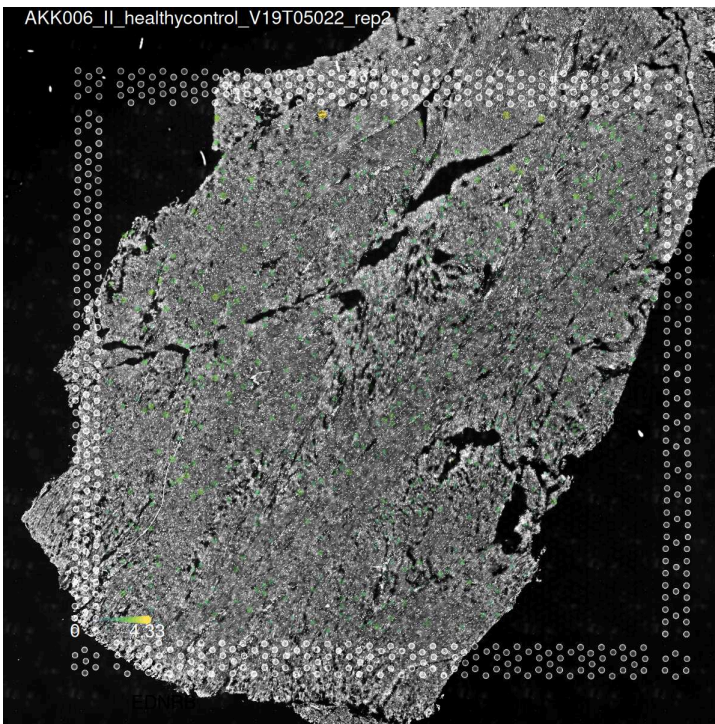


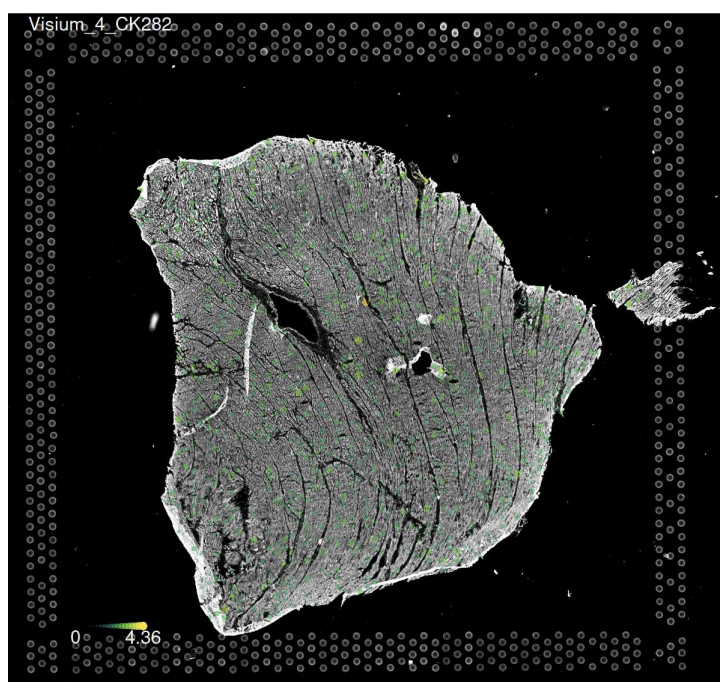
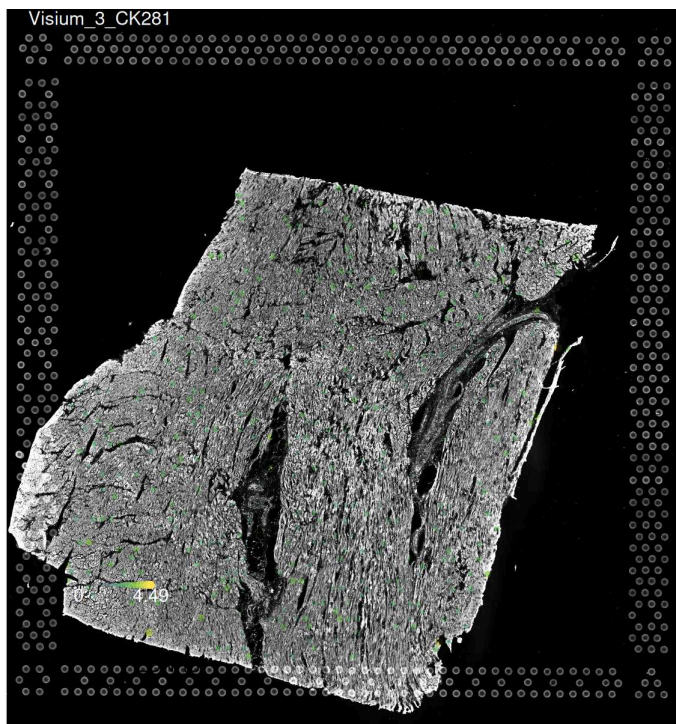
EDNRB only - all

```
In [13]: options(repr.plot.width = 10)
options(repr.plot.height = 10)
gc()
#par(mfrow=c(2,2))
gene.r = "EDNRB"
for(i in 1:13){
  samples2plot = sample_id_list[c(i)]
  pdf(paste("figures/panels/results/Kuppe2022/", gene.r, "_", dataset_names[i], "R.pdf", sep=""), height=10, width=10)
  plotVisium1(
    rgb.samples_list = VisiumData1(gene.r, sample_id_list = samples2plot),
    light_or_dark = input$light_or_dark,
    plot_size = input$plot_size,
    mask_overlay = input$mask_overlay,
    min.col = input$min.col,
    opacity_count_dependent = sum(input$opacity_count_dependent),
    opacity = input$opacity,
    dot_or_not = input$dot_or_not,
    sample_id_list = samples2plot
  )
  dev.off()
  plotVisium1(
    rgb.samples_list = VisiumData1(gene.r, sample_id_list = samples2plot),
    light_or_dark = input$light_or_dark,
    plot_size = input$plot_size,
    mask_overlay = input$mask_overlay,
    min.col = input$min.col,
    opacity_count_dependent = sum(input$opacity_count_dependent),
    opacity = input$opacity,
    dot_or_not = input$dot_or_not,
    sample_id_list = samples2plot
  )
}
```

A matrix: 2 × 6 of type dbl

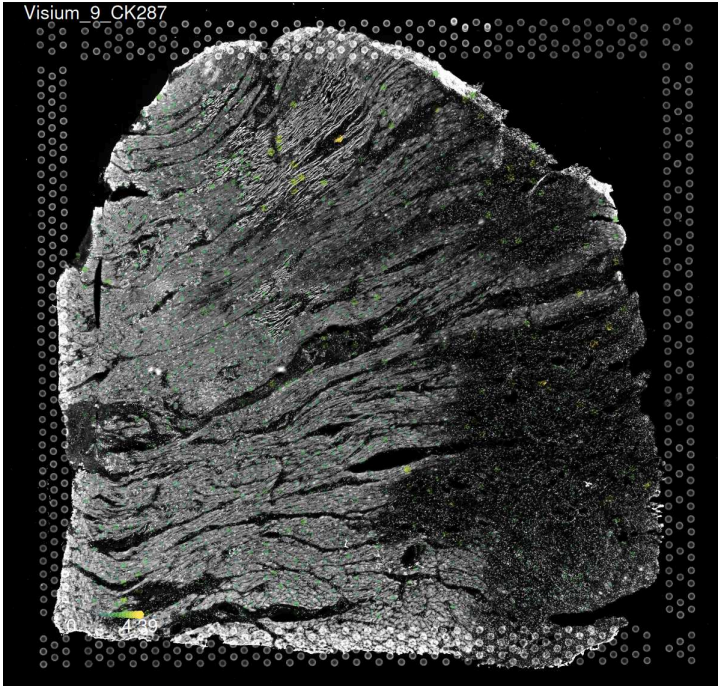
	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	2187973	116.9	4263800	227.8	3586903	191.6
Vcells	406541879	3101.7	747798152	5705.3	747798152	5705.3





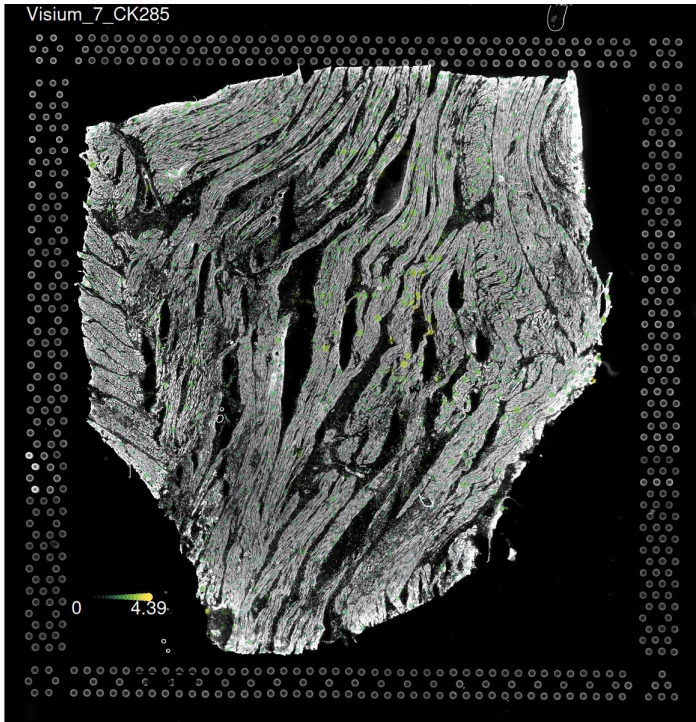
FDNRR

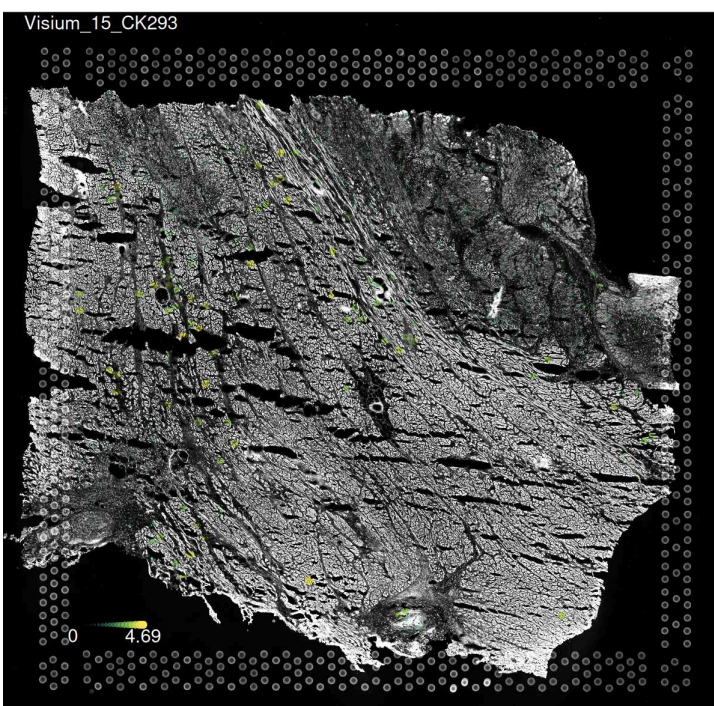
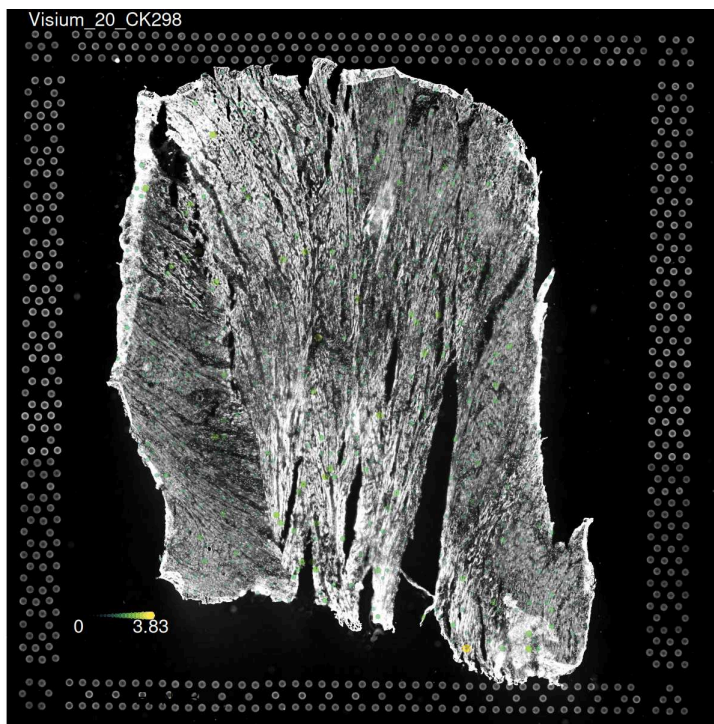
Visium_9_CK287



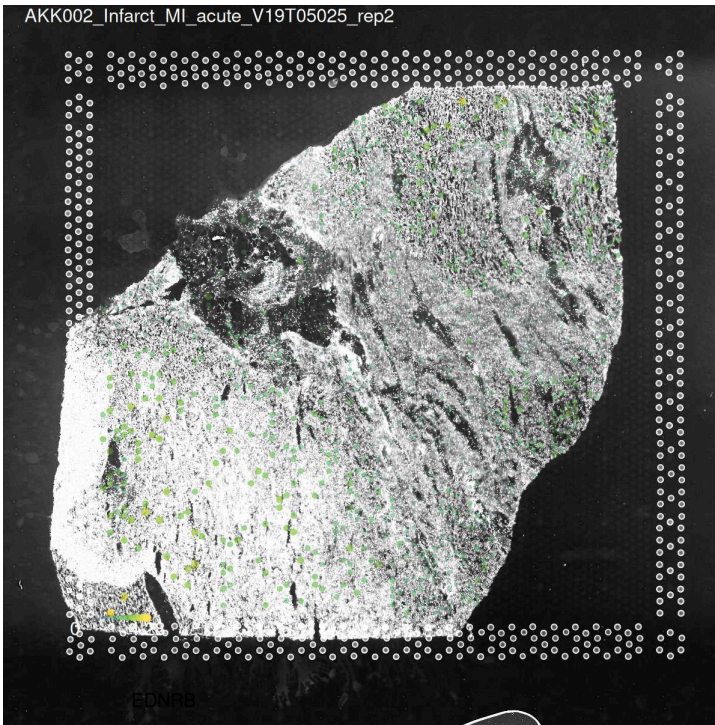
EDNRB

Visium_7_CK285

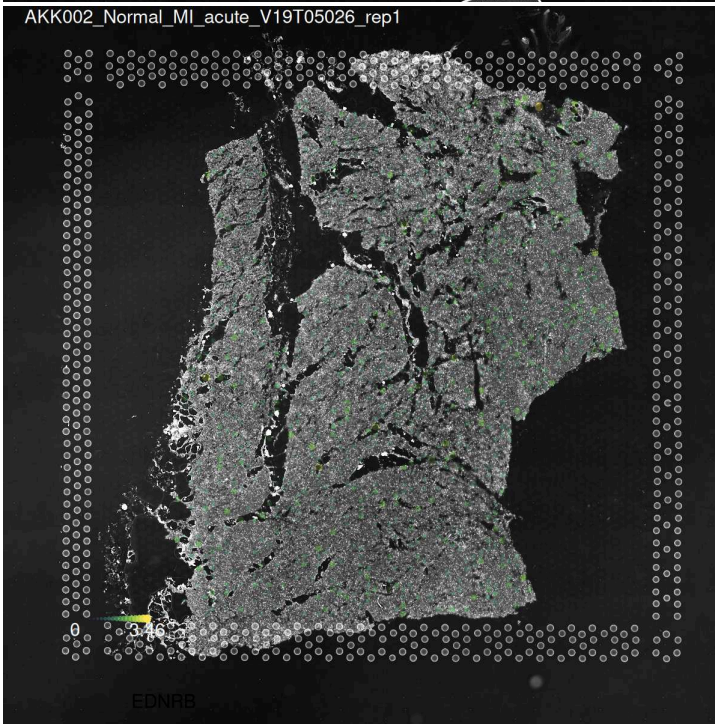




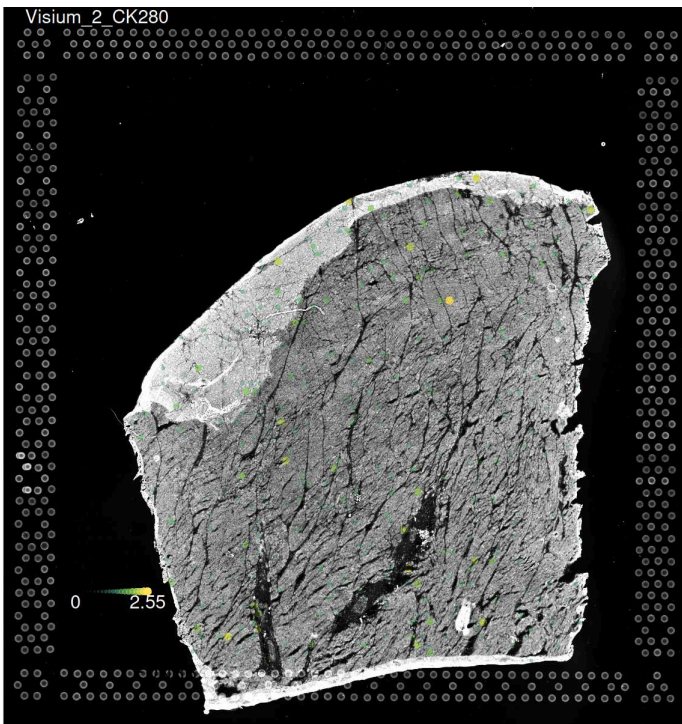
AKK002_Infarct_MI_acute_V19T05025_rep2



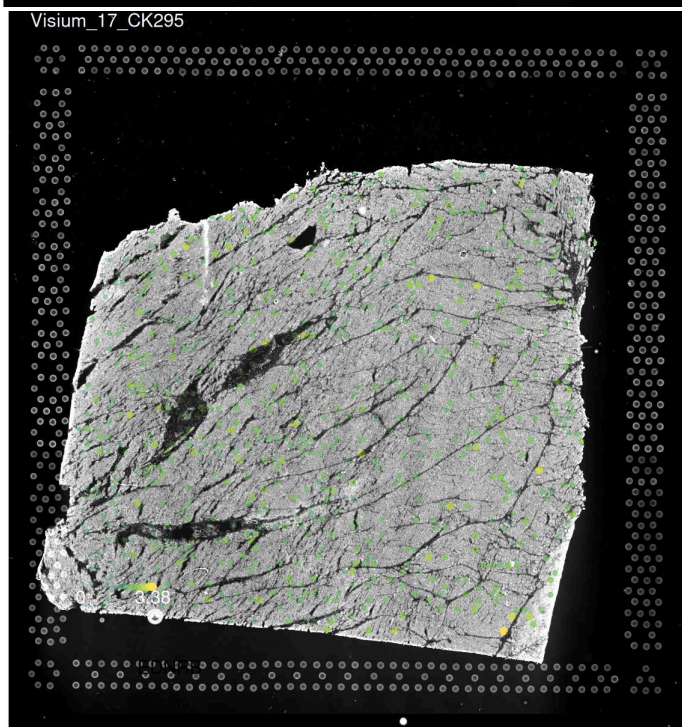
AKK002_Normal_MI_acute_V19T05026_rep1

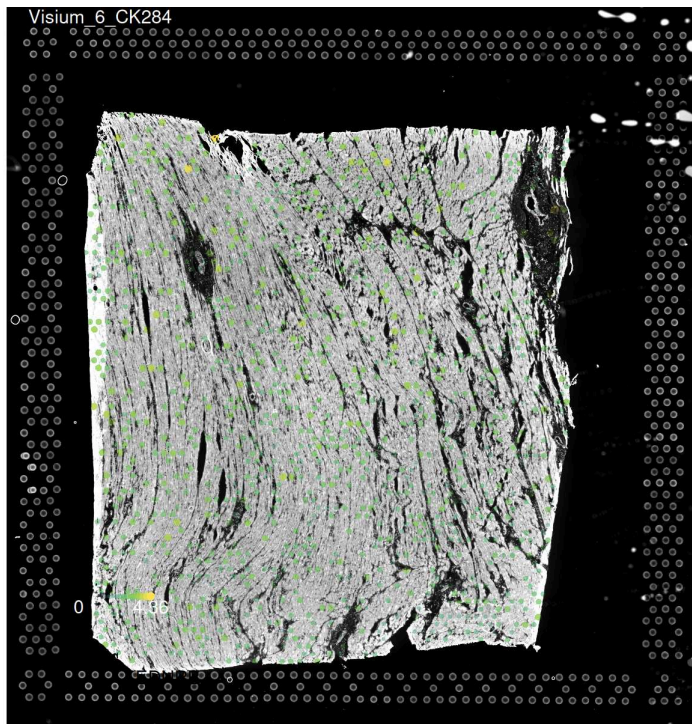


Visium_2_CK280



Visium_17_CK295



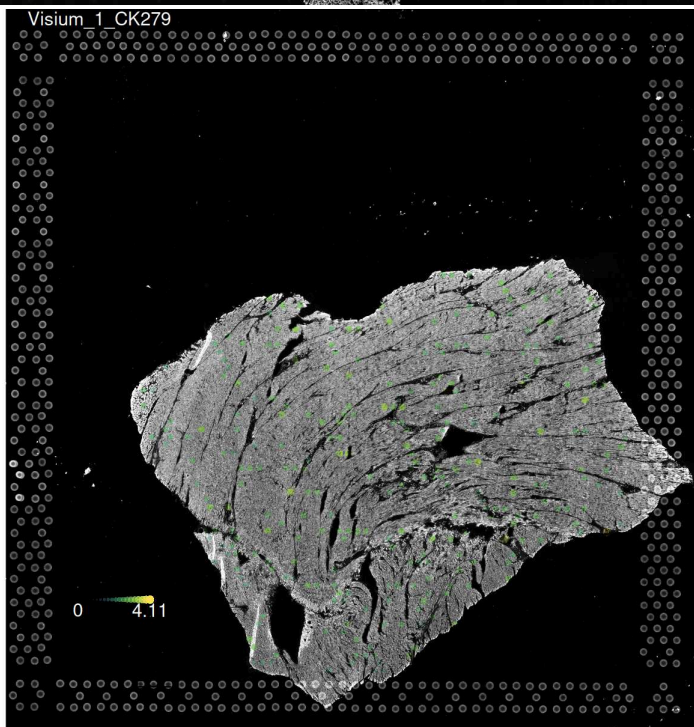
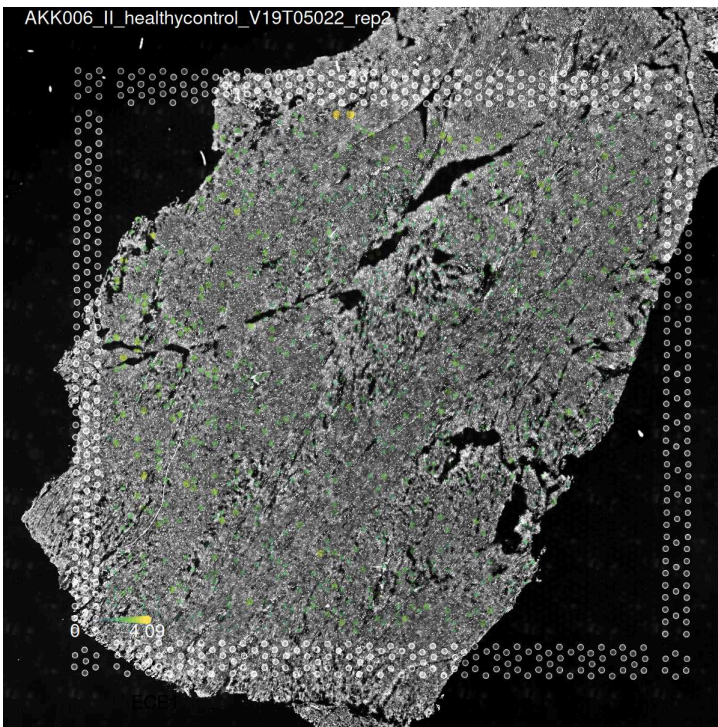


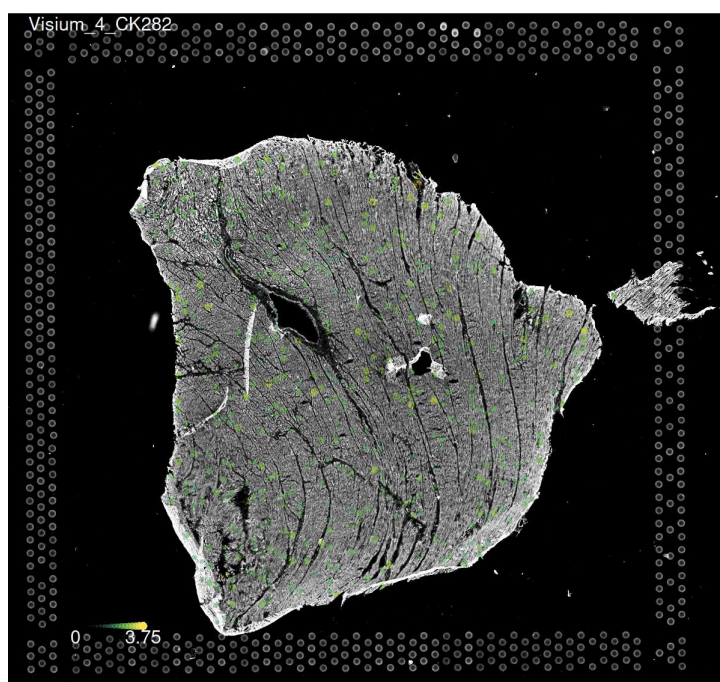
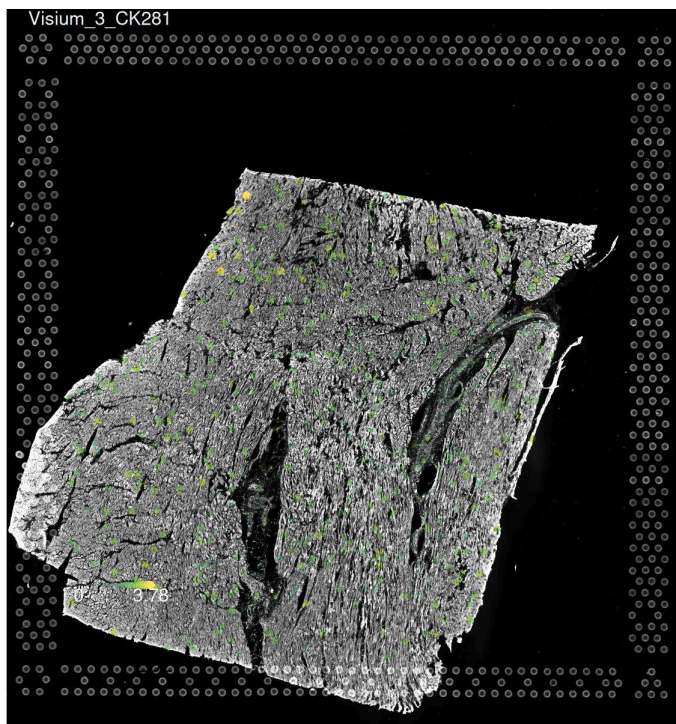
ECE1 only - all

```
In [14]: options(repr.plot.width = 10)
options(repr.plot.height = 10)
gc()
#par(mfrow=c(2,2))
gene.r = "ECE1"
for(i in 1:13){
  samples2plot = sample_id_list[c(i)]
  pdf(paste("figures/panels/results/Kuppe2022/", gene.r, "_", dataset_names[i], "R.pdf", sep=""), height=10, width=10)
  plotVisium1(
    rgb.samples_list = VisiumData1(gene.r, sample_id_list = samples2plot),
    light_or_dark = input$light_or_dark,
    plot_size = input$plot_size,
    mask_overlay = input$mask_overlay,
    min.col = input$min.col,
    opacity_count_dependent = sum(input$opacity_count_dependent),
    opacity = input$opacity,
    dot_or_not = input$dot_or_not,
    sample_id_list = samples2plot
  )
  dev.off()
  plotVisium1(
    rgb.samples_list = VisiumData1(gene.r, sample_id_list = samples2plot),
    light_or_dark = input$light_or_dark,
    plot_size = input$plot_size,
    mask_overlay = input$mask_overlay,
    min.col = input$min.col,
    opacity_count_dependent = sum(input$opacity_count_dependent),
    opacity = input$opacity,
    dot_or_not = input$dot_or_not,
    sample_id_list = samples2plot
  )
}
```

A matrix: 2 x 6 of type dbl

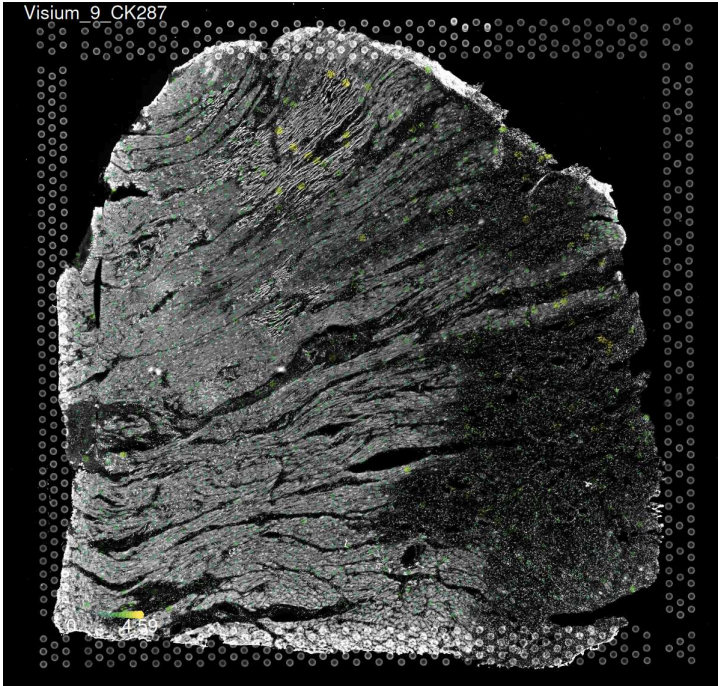
	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	2187979	116.9	4263800	227.8	3586903	191.6
Vcells	406541892	3101.7	748032986	5707.1	748032986	5707.1





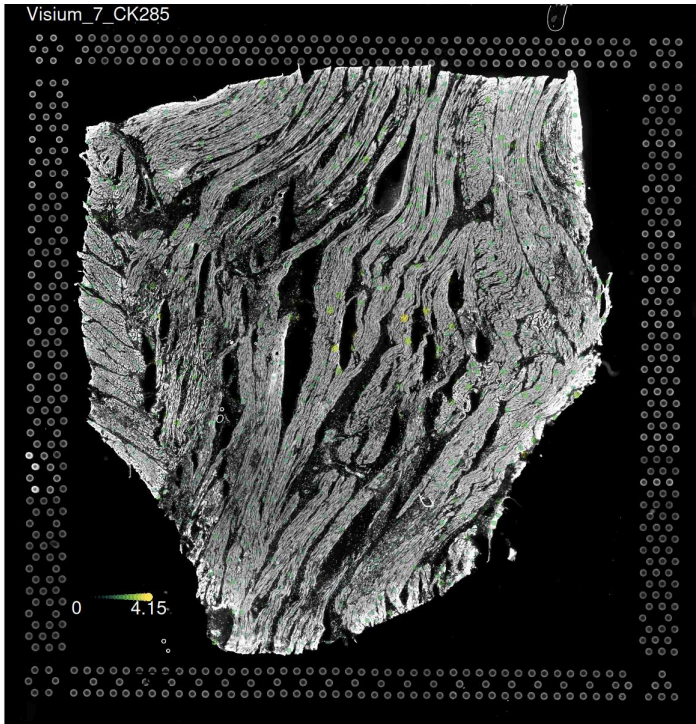
FCF1

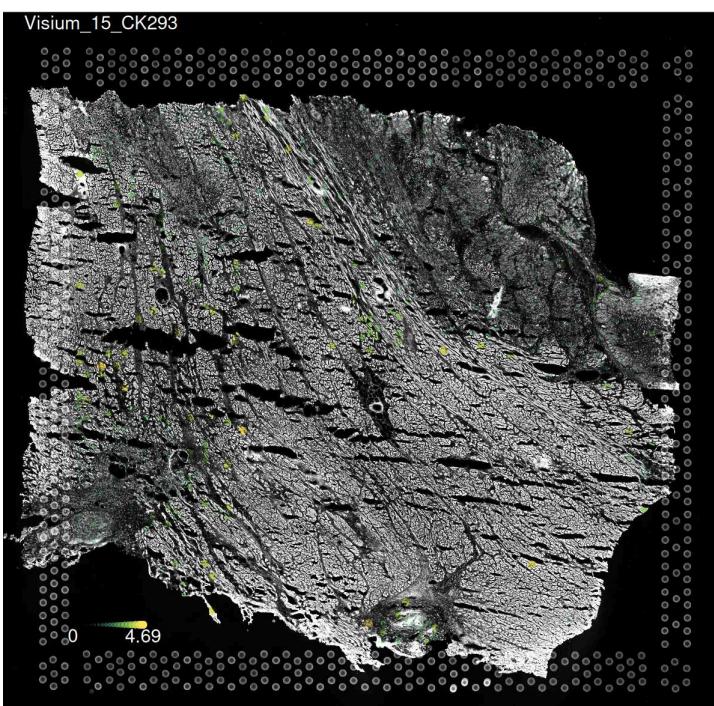
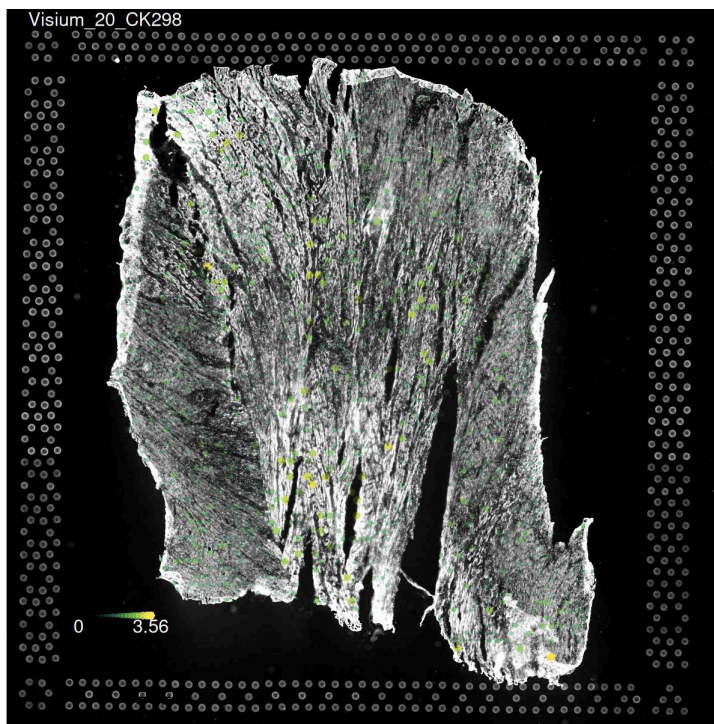
Visium_9_CK287



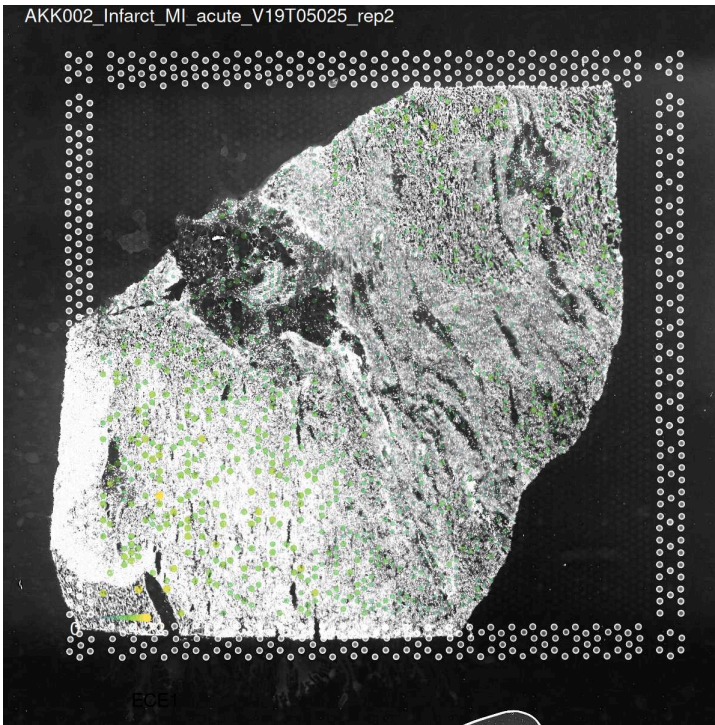
ECE1

Visium_7_CK285

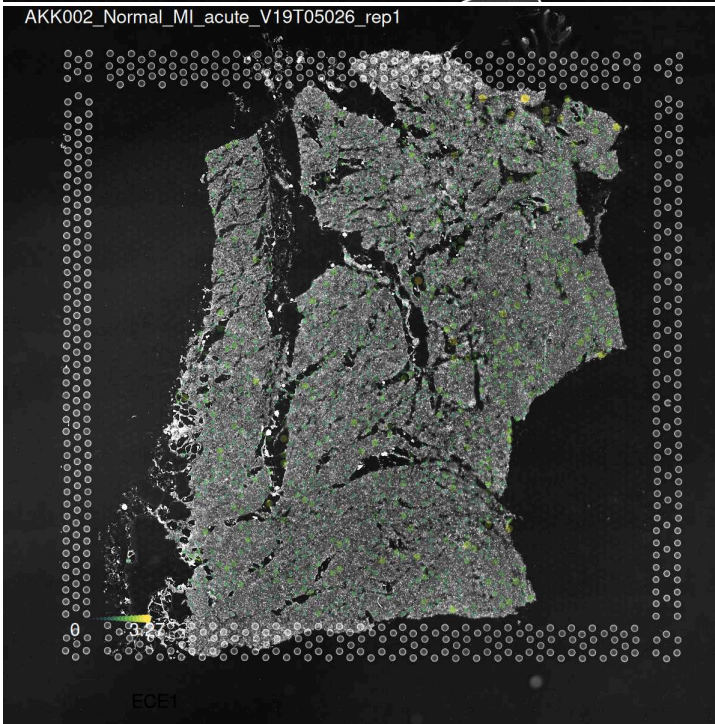




AKK002_Infarct_MI_acute_V19T05025_rep2



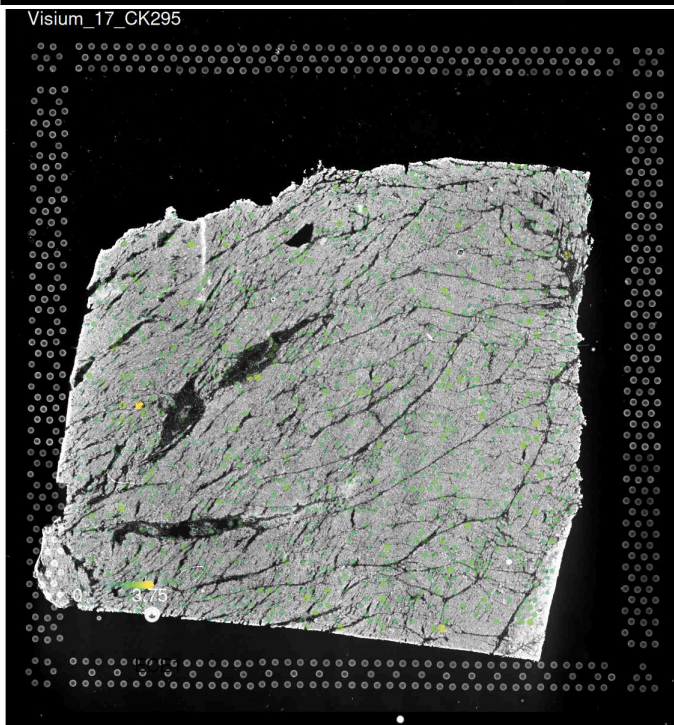
AKK002_Normal_MI_acute_V19T05026_rep1

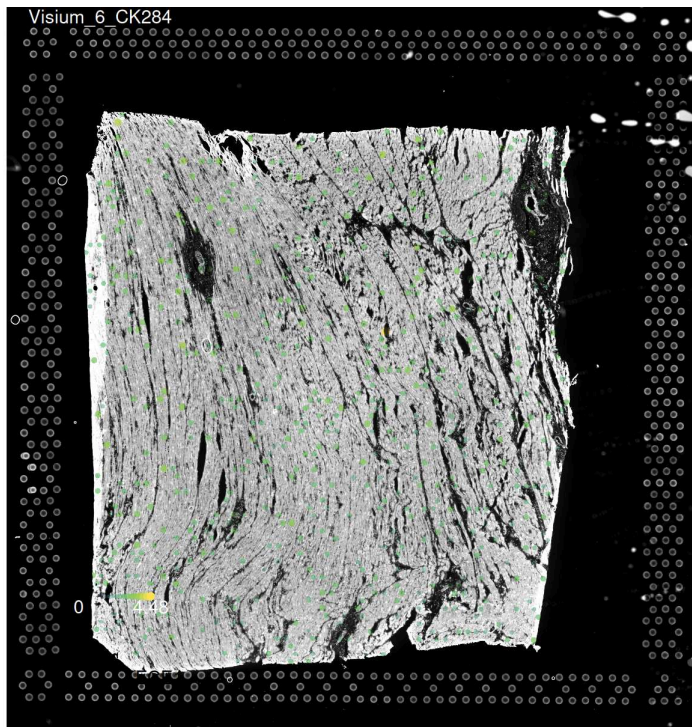


Visium_2_CK280



Visium_17_CK295



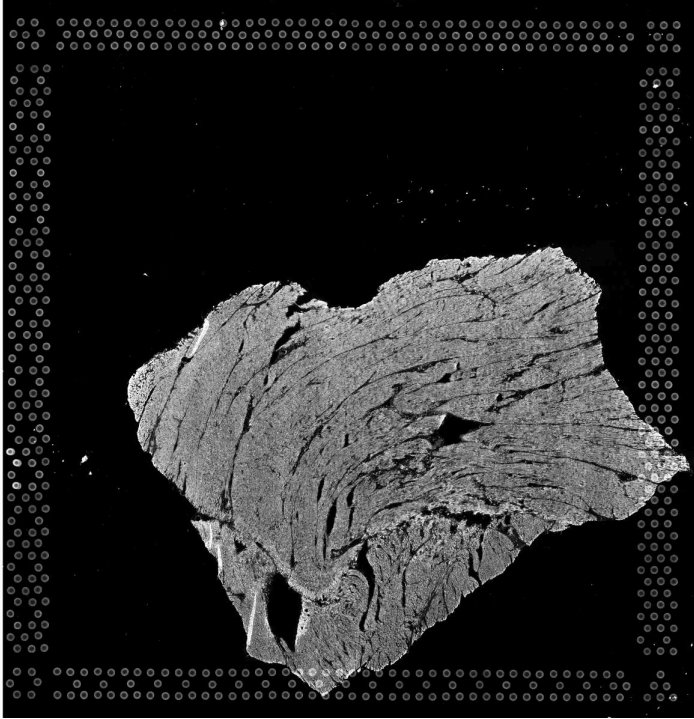
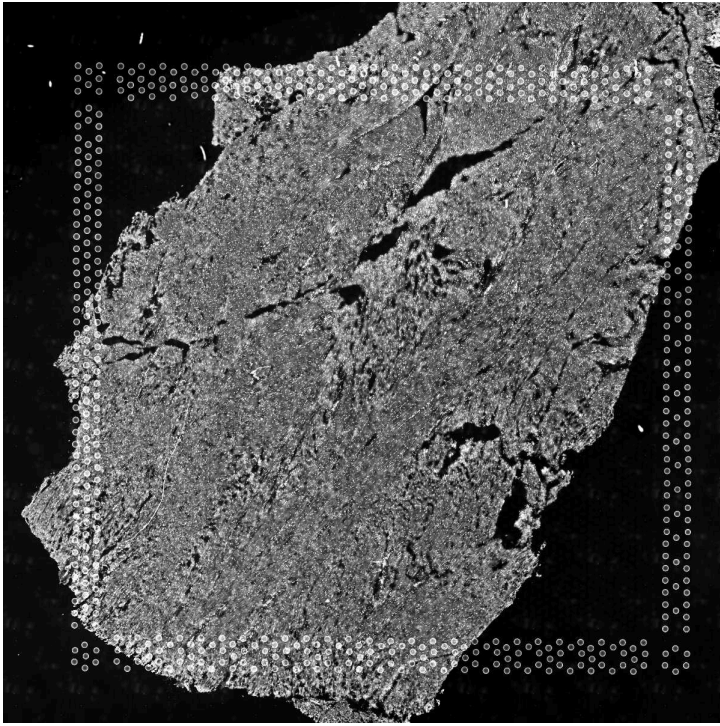


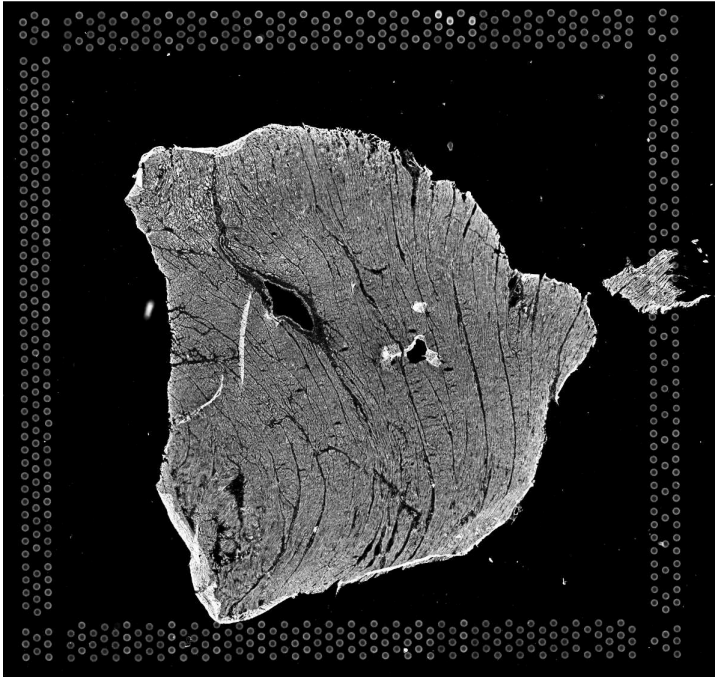
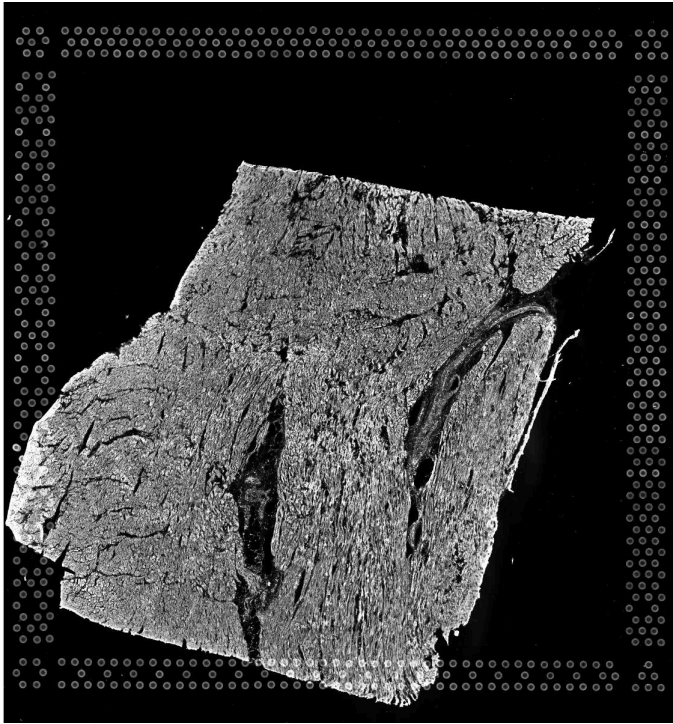
ECE2 only - all

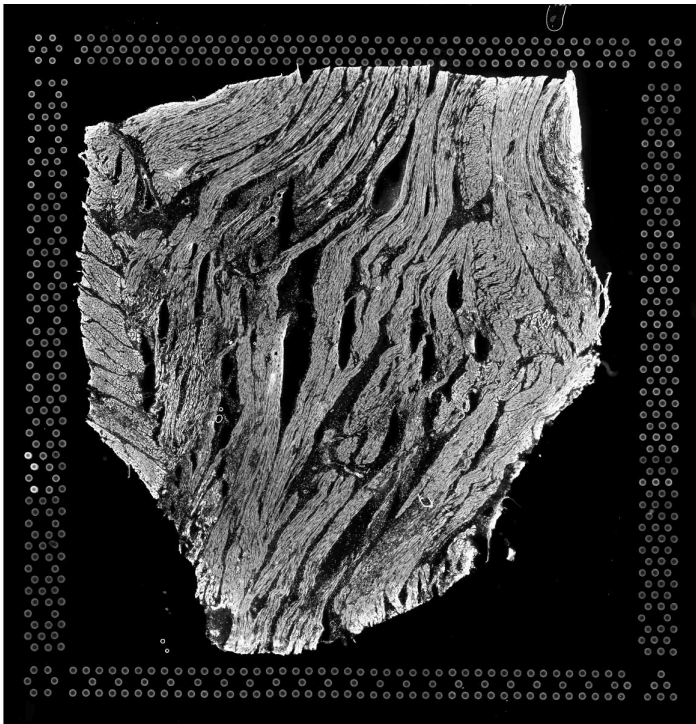
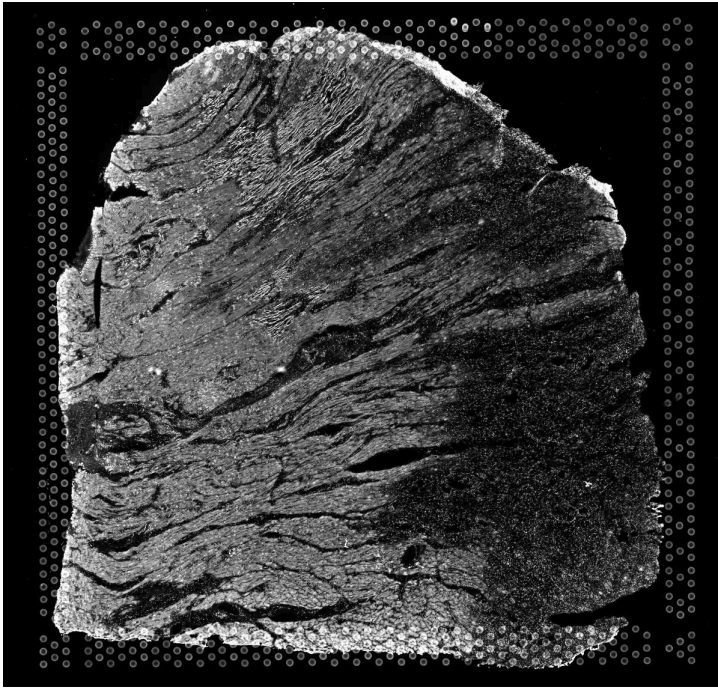
```
In [15]: options(repr.plot.width = 10)
options(repr.plot.height = 10)
gc()
#par(mfrow=c(2,2))
gene.r = "ECE2"
for(i in 1:13){
  samples2plot = sample_id_list[c(i)]
  pdf(paste("figures/panels/results/Kuppe2022/", gene.r, "_", dataset_names[i], "R.pdf", sep=""), height=10, width=10)
  plotVisium1(
    rgb.samples_list = VisiumData1(gene.r, sample_id_list = samples2plot),
    light_or_dark = input$light_or_dark,
    plot_size = input$plot_size,
    mask_overlay = input$mask_overlay,
    min.col = input$min.col,
    opacity_count_dependent = sum(input$opacity_count_dependent),
    opacity = input$opacity,
    dot_or_not = input$dot_or_not,
    sample_id_list = samples2plot
  )
  dev.off()
  plotVisium1(
    rgb.samples_list = VisiumData1(gene.r, sample_id_list = samples2plot),
    light_or_dark = input$light_or_dark,
    plot_size = input$plot_size,
    mask_overlay = input$mask_overlay,
    min.col = input$min.col,
    opacity_count_dependent = sum(input$opacity_count_dependent),
    opacity = input$opacity,
    dot_or_not = input$dot_or_not,
    sample_id_list = samples2plot
  )
}
```

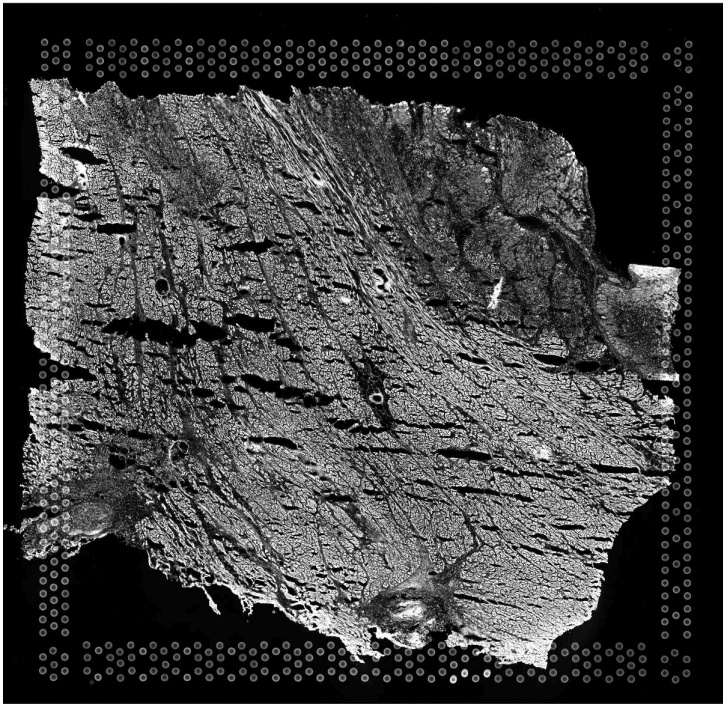
A matrix: 2 × 6 of type dbl

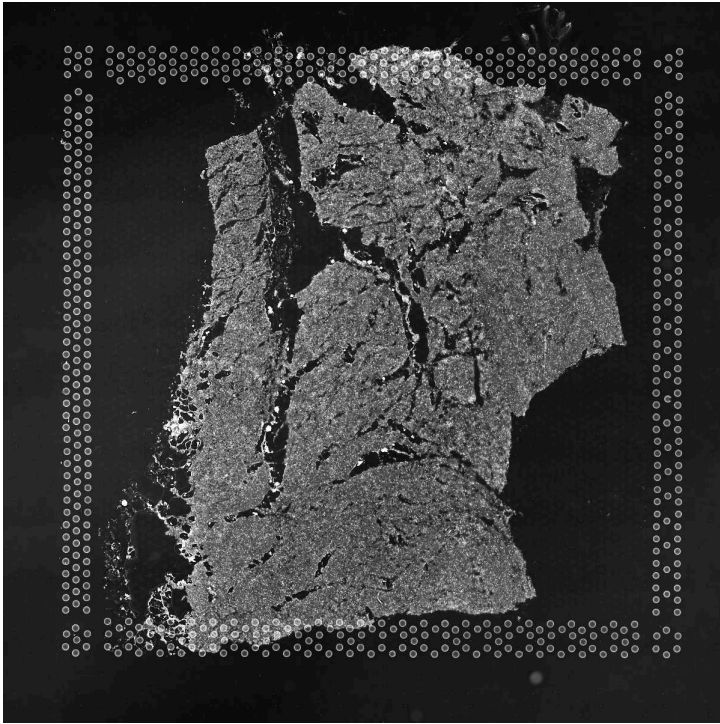
	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	2187968	116.9	4263800	227.8	3586903	191.6
Vcells	406541881	3101.7	748069691	5707.4	748069691	5707.4

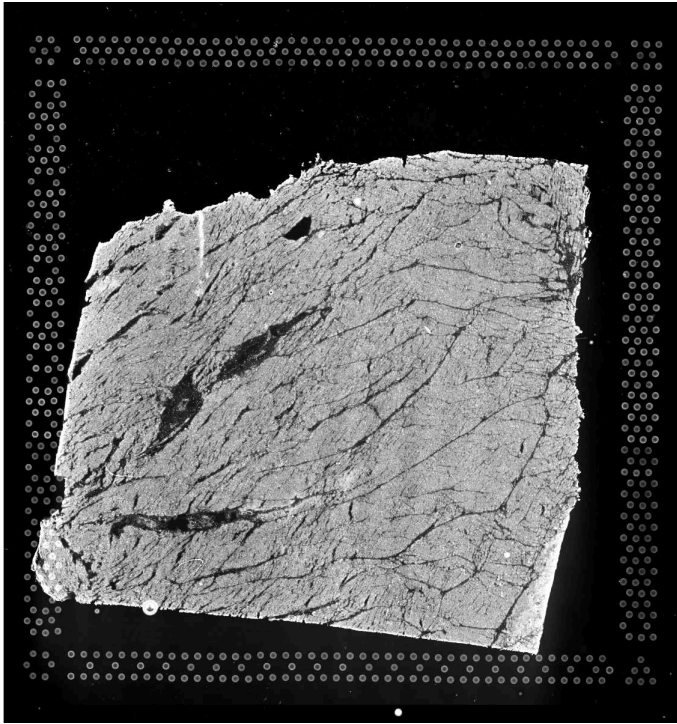
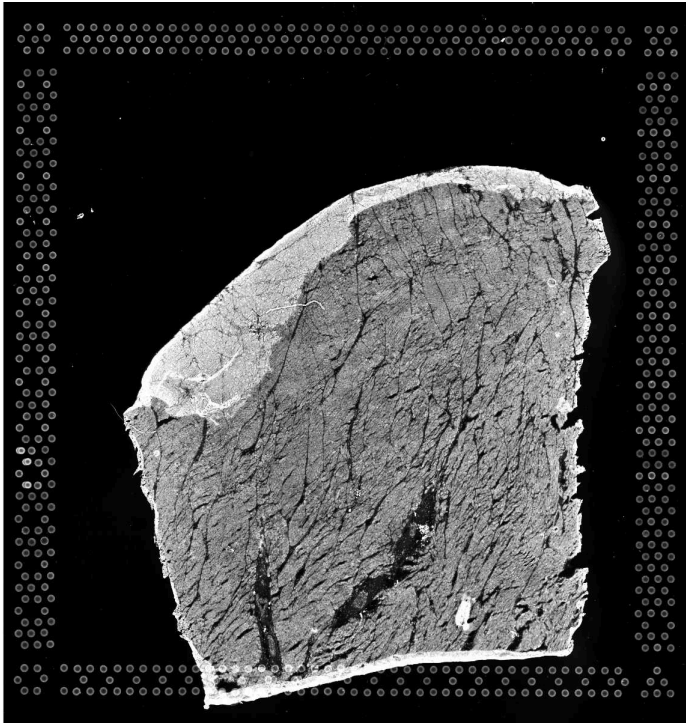


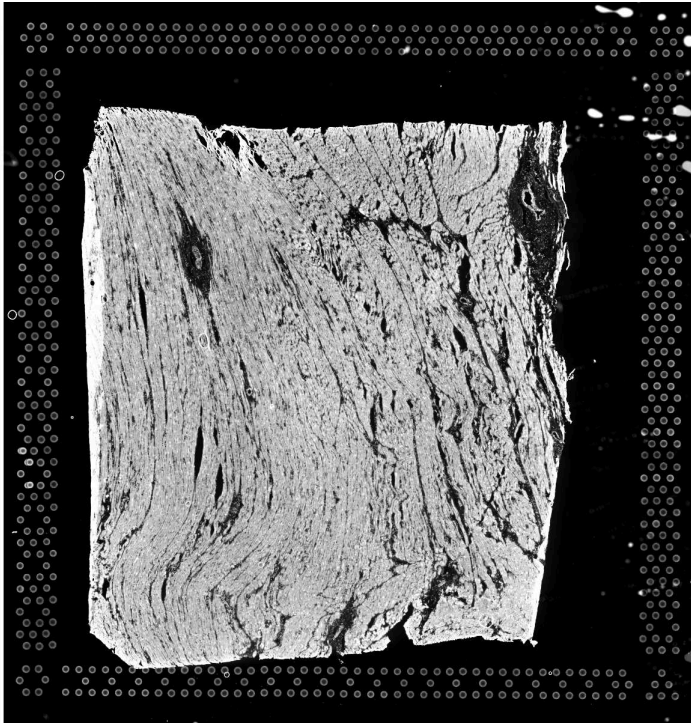












Notes

- EDN1 - quite present in all tissues, sparse. Obvious localisation in some slides around vessels, presumably coronary arteries
- EDN2 - popped up only in 2 sections, not sure what it means
- EDN3 - absent in all sections
- EDNRA - present in all sections, not obvious localisation, sparse all over
- EDNRB - present in all sections, not obvious localisation, sparse all over
- ECE1 - present in all sections, not overly obvious localisation or differences between healthy and MI at a glance.
- ECE2 - absent in all sections